

claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\agent-a5d37002bc231c023.jsonl`  
- SHA-256: `2077b54d4aa858fb594ce5083c8a0fcacfca1e3a40edcd3dc0bc2ee83d0107df9`  
- Source modified: `2026-06-21T22:33:29+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `subagents`  
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`
```

Transcript

1. user (2026-06-21T22:32:04.305Z)

Map the install/first-run/config surface of this repo for a "P2b-install" packaging PR. Work ONLY in the worktree at C:\Users\avidu\Projects\bhi-p2b (this is origin/master, detached). Read-only; report findings with exact file:line citations. Be thorough but return a TIGHT structured map, not file dumps.

Answer these specific questions:

- **Launcher app-home wiring.**** In `backend/main.py`, does the `indexer --app` launcher (and/or `--server`) auto-export/resolve `RALLY_APP_HOME` to the OS app-data dir (`backend/utils/app_home.py::default_os_app_home`) when it's unset? Quote the exact code path in `main()` that handles `args.app`, and state whether a fresh `indexer --app` run (no `RALLY_APP_HOME`, no `config.json`) roots its state in the OS app-data dir or in the CWD. Also: where/when does the server call `load_config()` and `default_config_path()`?
- **Fresh-install config defaults.**** For a fresh `pip install` of the wheel (NO repo-root `config.json` present, NO `RALLY_APP_HOME`): what does `load_config()` (`backend/config/loader.py`) produce when no `config.json` file exists? does it error, or fall back to pydantic `AppConfig()` defaults? Quote the loader's missing-file branch. Then report the DEFAULT pydantic values (the `dataclass/BaseModel` field defaults, NOT the repo-root `config.json` overrides) for: `indexing.default_segmenter`, `indexing.use_gpu`, `deployment.profile` (and `compute_target` if present), `indexing.skip_ai_handoff`, `stitching.min_shot_count`. Cite the model files under `backend/config/`.
- **Truly-local profile preset.**** In `backend/config/presets.py`, list the `PROFILE_PRESETS` keys and quote exactly what the `truly-local` (and `cpu-mock`) preset sets? `segmenter`, `compute_target`, `storage backend`, `skip_ai_handoff`, `use_gpu`, anything else. How is a profile applied to an `AppConfig` (the merge/override function)? Is a profile selectable purely via `config.json` (e.g. a `deployment.profile` field) or only via `env/preset` code?
- **`indexer --setup` / doctor.py.**** Read `scripts/doctor.py`. What does `--setup` do today? does it WRITE a `config.json`, and to what path (does it use `default_config_path()` / `app_home`, or hardcode the repo root)? Does it check `ffmpeg` and print the per-OS install hint (`winget` etc.)? Is `doctor.py` packaged in the wheel (it's under `scripts/`, and the wheel `packages=["backend","training"]`)? i.e. is `indexer --setup` reachable from an installed wheel, or does `--setup` import something from `scripts/`?
- **README / docs install steps.**** Quote the current documented install procedure (`grep README.md + docs` for "pip install", "uv ", "pip install -e", torch index-url, ffmpeg/winget). What does a user run today end-to-end to install?
- **Uninstall footprint.**** Enumerate every piece of mutable state a fresh install + first run creates that a clean uninstall would need to scrub: app-home files (`config.json`, `.env`, `output/`, the SQLite DB name from `config`output.db_name``, the `<db>.lock` from `P0e single_instance`, uploads dir from `security.upload_dir` if any), plus the installed `indexer` console script and the wheel/package. Cite where each path is determined.

7. ****Existing install tooling.**** Is there ANY existing install script, uv usage, uninstall manifest, or `[project.scripts]` beyond `indexer`? Grep the repo (deploy/, scripts/, pyproject.toml, *.sh, *.ps1). Note `deploy/digitalocean/bootstrap.sh` and what it installs (just summarize, don't dump).

8. ****Tests layout for config/doctor/app_home.**** Where do existing tests for app_home, config loading, and doctor live (tests/test_*.py)? Name the files so a new P2b test fits the convention.

Return: a numbered findings list answering 1-8, each with file:line citations and short quotes. Flag anything that would make "ship a truly-local default config.json" + "uninstall manifest" tricky (e.g. config.json not in the wheel, --setup writing to the wrong place, profile only applicable via code not config).

2. assistant (2026-06-21T22:32:06.793Z)

I'll systematically map the install/first-run/config surface. Let me start by exploring the repo structure and the key files you mentioned.

3. user (2026-06-21T22:32:07.415Z)

```
./backend/config/loader.py
./backend/config/models.py
./backend/config/presets.py
./backend/config/startup.py
./backend/config/__init__.py
./backend/eval/gt_loader.py
./backend/main.py
./backend/utils/app_home.py
./backend/worker/__main__.py
./scripts/doctor.py
./tests/test_app_home.py
./tests/test_config.py
./tests/test_config_deployment.py
./tests/test_config_wasb.py
./tests/test_doctor.py
./tests/test_gt_loader.py
```

4. assistant (2026-06-21T22:32:08.875Z)

Now let me read the key files:

5. user (2026-06-21T22:32:12.387Z)

```
1      """Resolved application-data HOME for a packaged, batteries-included install.
2
3      PACKAGING_PLAN.md P0a. Today every piece of mutable state ? ``config.json``, ``.env``,
4      the SQLite DB, ``output/`` ? is resolved RELATIVE TO THE CURRENT WORKING DIRECTORY, and
5      ``scripts/doctor.py`` writes ``config.json`` to the repo root while the server reads it
6      from
7      the CWD. Launched from an OS app menu (CWD ? repo root) those disagree and the user
8      silently
9      "loses" their index. This module introduces ONE resolver every state path roots on.
10     Design contract ? **DEFAULT IS BYTE-IDENTICAL TO TODAY**:
11     * ``RALLY_APP_HOME`` env set ? that directory is the app home (the packaged launcher
12     sets it).
13     * ``RALLY_APP_HOME`` unset ? ``Path(".")`` i.e. the CWD, exactly the pre-P0a
14     behaviour.
15
16     The OS-default location (``%APPDATA%`` / ``~/Library/Application Support`` /
17     ``~/local/share``)
18     is computed by :func:`default_os_app_home` but is **NOT** auto-selected here ? the
19     launcher (P2)
```

```

15     calls it and exports ``RALLY_APP_HOME`` explicitly. Keeping the resolver opt-in means
importing
16     this module has zero behavioural effect on the repo/dev/CI/test paths (no surprise
writes into a
17     real user profile during a test run).
18
19     stdlib-only (os, platform, pathlib) so it is safe to import at ``backend.main`` line 1 ?
before
20     the heavier pydantic-backed ``backend.config`` package ? keeping the early
``load_dotenv`` path light.
21     """
22
23     from __future__ import annotations
24
25     import os
26     import platform
27     from pathlib import Path
28
29     #: Environment variable the packaged launcher sets to pin the app-data home.
30     APP_HOME_ENV = "RALLY_APP_HOME"
31
32     #: OS-app-dir leaf used by :func:`default_os_app_home` (the product brand).
33     APP_DIR_NAME = "Khelsutra"
34
35
36     def app_home() -> Path:
37         """The resolved application-data home.
38
39         ``RALLY_APP_HOME`` (``~`` expanded) when set; otherwise ``Path(".")`` ? the CWD,
byte-identical
40         to the pre-P0a behaviour. Does NOT create the directory (callers create on write).
41         """
42         env = os.environ.get(APP_HOME_ENV)
43         if env and env.strip():
44             return Path(env.strip()).expanduser()
45         return Path(".")
46
47
48     def app_path(*parts: str | os.PathLike[str]) -> Path:
49         """A path under :func:`app_home` (e.g. ``app_path("output")``)."""
50         return app_home().joinpath(*[str(p) for p in parts])
51
52
53     def default_config_path() -> Path:
54         """``<app-home>/config.json`` ? the config file the loader reads and ``--setup``
writes.
55
56         With ``RALLY_APP_HOME`` unset this is ``Path("config.json")`` (CWD-relative),
unchanged.
57         """
58         return app_home() / "config.json"
59
60
61     def default_env_path() -> Path:
62         """``<app-home>/env`` ? the dotenv file the server prefers (e.g.
``GEMINI_API_KEY``)."""
63         return app_home() / ".env"
64
65
66     def env_file_to_load() -> Path | None:
67         """The ``.env`` the server should load, or ``None`` meaning "use python-dotenv's
DEFAULT
68         (frame-relative walk-up) search" ? exactly the pre-P0a bare ``load_dotenv()``
behaviour.
69

```

```

70         Returns the app-home ``.env`` ONLY when ``RALLY_APP_HOME`` is set AND that file
exists. With the
71         env UNSET this returns ``None`` so the caller keeps the historical search and does
NOT newly
72         prefer a stray CWD ``.env`` (bare ``load_dotenv()`` walks up from the caller file,
not the CWD).
73         """
74         home = app_home()
75         if str(home) == ".": # env unset (or pinned to CWD) ? historical load_dotenv()
search
76             return None
77         candidate = home / ".env"
78         return candidate if candidate.exists() else None
79
80
81     def resolve_output_dir(output_dir: str) -> str:
82         """Root a RELATIVE ``output_dir`` under :func:`app_home`; pass an ABSOLUTE one
through.
83
84         With ``RALLY_APP_HOME`` UNSET the input is returned VERBATIM (the default
``"output"`` when
85         empty) ? byte-identical to the pre-P0a code that stored ``--output-dir`` as-is (no
pathlib
86         normalisation of ``./x`` / separators / trailing slash). With it SET, a relative dir
like
87         ``"output"`` becomes ``<app-home>/output`` so the SQLite DB and reels live with the
rest of the
88         app's state; an absolute path ? or a Windows drive-anchored one (``C:\x`` /
``C:x``) ? passes
89         through untouched.
90         """
91         home = app_home()
92         if str(home) == ".": # env unset (or pinned to CWD) ? verbatim, today's behaviour
93             return output_dir or "output"
94         if not output_dir:
95             return str(home / "output")
96         if os.path.isabs(output_dir) or os.path.splitdrive(output_dir)[0]:
97             return output_dir
98         return str(home / output_dir)
99
100
101     def default_os_app_home(app_name: str = APP_DIR_NAME) -> Path:
102         """The OS-conventional per-user app-data dir (for the packaged LAUNCHER to export as
103         ``RALLY_APP_HOME`` ? NOT auto-selected by :func:`app_home`).
104
105         * Windows ? ``%APPDATA%\<app_name>`` (``~/AppData/Roaming/<app_name>`` if unset)
106         * macOS ? ``~/Library/Application Support/<app_name>``
107         * Linux/other ? ``$XDG_DATA_HOME/<app_name>`` (``~/.local/share/<app_name>`` if
unset)
108         """
109         system = platform.system()
110         if system == "Windows":
111             base = os.environ.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
112         elif system == "Darwin":
113             base = str(Path.home() / "Library" / "Application Support")
114         else:
115             base = os.environ.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
116         return Path(base) / app_name
117

```

6. user (2026-06-21T22:32:12.479Z)

```

1     """Configuration loader and saver.
2
3     Provides a single place to load validated AppConfig with built-in defaults

```

```

4     merged with optional file overrides (config.json).
5
6     This replaces scattered json.load + .get() patterns throughout the codebase.
7     """
8
9     from __future__ import annotations
10
11     import json
12     import logging
13     import os
14     from pathlib import Path
15     from typing import Any
16
17     from pydantic import BaseModel
18
19     from backend.utils.app_home import default_config_path
20
21     from .models import AppConfig
22     from .presets import (
23         COMPUTE_TARGET_FOR_PROFILE,
24         PROFILE_PRESETS,
25         is_known_profile,
26         preset_for,
27         suggest_profile,
28     )
29
30     logger = logging.getLogger(__name__)
31
32     # Config path resolution now lives in ``backend.utils.app_home.default_config_path()`` ?
it honours
33     # ``RALLY_APP_HOME`` (PACKAGING_PLAN.md P0a) and equals ``Path("config.json")`` (CWD-
relative) when
34     # the env is unset, exactly the pre-P0a value. (The old module-level
``DEFAULT_CONFIG_PATH`` constant
35     # was dropped: it had no external consumers and a frozen-at-import snapshot would
silently diverge
36     # from a later RALLY_APP_HOME.)
37
38
39     def _deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
40         """Recursively overlay ``override`` onto ``base`` (override wins; dicts merge,
scalars
41         replace). Used ONLY for the profile-preset precedence path so a config.json sub-key
42         overrides a preset sub-key while sibling preset/default sub-keys survive ? the no-
profile
43         path keeps the original shallow ``{**defaults, **file_data}`` merge unchanged."""
44         out = dict(base)
45         for key, val in override.items():
46             if isinstance(val, dict) and isinstance(out.get(key), dict):
47                 out[key] = _deep_merge(out[key], val)
48             else:
49                 out[key] = val
50         return out
51
52
53     def _resolve_explicit_profile(file_data: dict[str, Any]) -> str:
54         """The EXPLICITLY chosen deployment profile, env winning over config.json. ``""`` if
none
55         is set. Auto-detect is deliberately NOT consulted here ? it only suggests (D15), so
a bare
56         environment never silently selects a profile (and never silently spends on cloud).
57
58         An *unrecognised* ``DEPLOYMENT_PROFILE`` env value warns and FALLS THROUGH to the
file's
59         profile (rather than suppressing it) ? so an env typo can't leave the recorded

```

```

profile
60         asserting a preset that was never applied. A bad value in config.json is left to
surface as
61         a hard, typed-Literal error downstream (config-file correctness)."""
62         env_profile = os.environ.get("DEPLOYMENT_PROFILE")
63         if env_profile and env_profile.strip():
64             ep = env_profile.strip()
65             if is_known_profile(ep):
66                 return ep
67             logger.warning(
68                 "[CONFIG] unrecognized DEPLOYMENT_PROFILE env value %r (valid: %s); ignoring
it "
69                 "and falling back to config.json deployment.profile.", ep, ",
".join(PROFILE_PRESETS),
70             )
71             # fall through to the file's profile (env typo must not suppress a valid file
choice)
72         dep = file_data.get("deployment")
73         if isinstance(dep, dict):
74             prof = dep.get("profile")
75             if isinstance(prof, str) and prof.strip():
76                 return prof.strip()
77         return ""
78
79
80     def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str =
"" ) -> list[str]:
81         """Dotted paths in `data` that the typed config will not honor.
82
83         Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
84         kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
85         typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
86         user's intent is lost ? collect every such key so load_config can warn.
87         """
88         unknown: list[str] = []
89         fields = model_cls.model_fields
90         for key, val in data.items():
91             if key not in fields:
92                 unknown.append(prefix + key)
93                 continue
94             ann = fields[key].annotation
95             if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann,
BaseModel):
96                 unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}.")
97         return unknown
98
99
100     def _get_builtin_defaults() -> dict[str, Any]:
101         """Return a plain dict of the current built-in defaults.
102
103         The AppConfig model is the single source of truth for defaults.
104         """
105         return AppConfig().model_dump(mode="json")
106
107
108     def load_config(path: str | Path | None = None) -> AppConfig:
109         """Load configuration with the following precedence:
110
111         1. Built-in defaults defined in the Pydantic models.
112         2. Values from the JSON file (if present and readable).
113         3. (Future) environment / CLI overrides.
114
115         Missing file or unreadable file ? returns a fully defaulted AppConfig.
116         Unknown keys in the file are tolerated (extra="allow" on the model).
117         """

```

```

118     cfg_path = Path(path) if path is not None else default_config_path()
119
120     file_data: dict[str, Any] = {}
121     if cfg_path.exists():
122         try:
123             with open(cfg_path, "r", encoding="utf-8") as f:
124                 file_data = json.load(f)
125         except Exception as exc:
126             # Non-fatal: continue with defaults + whatever we could parse.
127             # In production we might log here.
128             logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")
129
130     # A config.json that is valid JSON but not an OBJECT (e.g. `[ ]`, `42`, `"x"`,
`null`) would
131     # otherwise crash startup: a truthy non-mapping hits `.items()` in
_unknown_key_paths, and any
132     # non-mapping breaks the `{**defaults, **file_data}` unpack. Degrade to defaults +
warn, per the
133     # loader's "bad input is non-fatal" contract.
134     if not isinstance(file_data, dict):
135         logger.warning("%s is not a JSON object (got %s); ignoring it and using
defaults.",
136                        cfg_path, type(file_data).__name__)
137         file_data = {}
138
139     if file_data:
140         unknown = _unknown_key_paths(file_data, AppConfig)
141         if unknown:
142             logger.warning(
143                 "[CONFIG WARNING] %s contains keys the typed config does not "
144                 "recognize (typo'd or removed knobs ? top-level extras are kept "
145                 "but unread; unknown section keys are silently dropped): %s",
146                 cfg_path, ", ".join(sorted(unknown)),
147             )
148
149     # Precedence: built-in defaults < profile preset < config.json (< env/--set, applied
at
150     # consumption sites downstream). A *deployment profile* is opt-in ? it is applied
ONLY when
151     # explicitly selected (DEPLOYMENT_PROFILE env or config.json deployment.profile).
With no
152     # profile the merge is byte-identical to the pre-M1 loader, so the default path is
unchanged.
153     defaults = _get_builtin_defaults()
154     profile = _resolve_explicit_profile(file_data)
155
156     if profile and not is_known_profile(profile):
157         # Only an unrecognised value from config.json reaches here (env typos already
fell back
158         # in _resolve_explicit_profile). Warn + apply no preset; the bad value also hits
the
159         # typed Literal at model_validate below, a hard clear error ? loud, never
silent.
160         logger.warning(
161             "[CONFIG] unrecognized deployment profile %r (valid: %s); applying no
preset.",
162             profile, ", ".join(PROFILE_PRESETS),
163         )
164         profile = ""
165
166     if profile:
167         merged = _deep_merge(_deep_merge(defaults, preset_for(profile)), file_data)
168         # Record the profile actually applied ? the env may have chosen it over
config.json,

```

```

169         # so re-stamp it onto the merged deployment section to keep the record honest.
170         merged.setdefault("deployment", {})
171         if isinstance(merged["deployment"], dict):
172             merged["deployment"]["profile"] = profile
173             ct = merged["deployment"].get("compute_target")
174             canonical = COMPUTE_TARGET_FOR_PROFILE.get(profile)
175             if not ct and canonical:
176                 # An active profile with no explicit compute_target (e.g. config.json
blanked it
177                 # to "") gets its canonical target back ? the record never silently lies
about an
178                 # active profile's compute_target (it stays meaningful for the M2+
ComputeBackend).
179                 merged["deployment"]["compute_target"] = canonical
180             elif ct and canonical and ct != canonical:
181                 logger.warning(
182                     "[CONFIG] deployment profile %r normally uses compute_target=%r but
"
183                     "config.json overrides it to %r ? honoring the explicit override.",
184                     profile, canonical, ct,
185                 )
186                 logger.info("[CONFIG] deployment profile ACTIVE: %s (compute_target=%s)",
187                             profile, merged["deployment"].get("compute_target"))
188         else:
189             # No profile ? the original shallow merge (Pydantic re-defaults nested
sections).
190             merged = {**defaults, **file_data}
191             suggestion = suggest_profile() # cheap, env-only (no torch); SUGGESTS only
(D15)
192             if suggestion:
193                 logger.debug(
194                     "[CONFIG] no deployment.profile set (manual knobs). Environment suggests
%r ? "
195                     "set deployment.profile or DEPLOYMENT_PROFILE to apply (auto-detect
never "
196                     "auto-applies ? D15).", suggestion,
197                 )
198
199         return AppConfig.model_validate(merged)
200
201
202     def save_default_config(path: str | Path | None = None) -> Path:
203         """Write a fresh default config.json.
204
205         Used by scripts/doctor.py diagnostics and the --setup CLI path.
206         Overwrites if the file exists.
207         """
208         cfg_path = Path(path) if path is not None else default_config_path()
209         cfg = AppConfig()
210         cfg_path.parent.mkdir(parents=True, exist_ok=True)
211         cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
212         return cfg_path
213
214
215     # Backwards-compatible alias used by some older call sites during transition.
216     def get_default_config() -> dict[str, Any]:
217         """Return defaults as a plain dict (for transitional use only)."""
218         return _get_builtin_defaults()
219

```

7. user (2026-06-21T22:32:12.584Z)

```

1         """Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING
M1).
2

```



```

3      A *deployment profile* is a single switch that applies a coherent BUNDLE of existing
4      config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
5      knob at a time (the design in ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).
6
7      Precedence (lowest ? highest), realised by the loader::
8
9          built-in model defaults < profile preset < config.json < env < worker --set
10
11      This module is **pure data + pure functions** ? no IO, and (critically) **no torch
import at
12      module load**. The loader applies a preset ONLY when a profile is EXPLICITLY selected
13      (``config.json`` ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); with no
14      profile selected the loader is byte-identical to the pre-M1 behaviour (``profile=""`` ==
15      today). Auto-detect only ever **suggests** a profile ? it never auto-applies one (D15:
cloud
16      spend is never entered silently).
17      ""
18
19      from __future__ import annotations
20
21      import copy
22      import os
23      from typing import Any, Mapping, Optional
24
25      # Canonical enum values, defined ONCE here. ``models.DeploymentConfig``'s ``Literal``
mirrors
26      # these and ``tests/test_config_deployment.py`` pins parity so the two can never drift.
27      PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
28      COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
29
30      # Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to
warn
31      # when config.json explicitly overrides compute_target into a state inconsistent with
the profile.
32      COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
33          "truly-local": "local_gpu",
34          "cloud-serving": "cloud_run",
35          "cpu-mock": "cpu_mock",
36      }
37
38      # A preset is a partial, nested override dict over the model defaults. It touches ONLY
knobs
39      # that EXIST today ? M1 is pure/additive: the new
``deployment.{profile,compute_target}`` plus
40      # ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones
extend
41      # these bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,
42      # Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
43      PROFILE_PRESETS: dict[str, dict[str, Any]] = {
44          "truly-local": {
45              "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
46              # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU
box).
47              # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
48              "indexing": {"skip_ai_handoff": True},
49              "storage": {"backend": "local"},
50          },
51          "cloud-serving": {
52              "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
53              # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor
(D11).
54              "indexing": {"detector_impl": "native", "skip_ai_handoff": False},
55              "storage": {"backend": "gcs"},
56          },
57          "cpu-mock": {

```

```

58         "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
59         # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression
profile.
60         "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
61         "storage": {"backend": "local"},
62     },
63 }
64
65
66 def is_known_profile(profile: str) -> bool:
67     """True for a non-empty, recognised profile name."""
68     return bool(profile) and profile in PROFILE_PRESETS
69
70
71 def preset_for(profile: str) -> dict[str, Any]:
72     """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate
the
73     module-level table), or ``{}`` for ``""`` / an unknown profile."""
74     return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))
75
76
77 def _truthy(val: Optional[str]) -> bool:
78     """Common env-var truthiness (``1/true/yes/on``, case/space-insensitive)."""
79     if val is None:
80         return False
81     return val.strip().lower() in {"1", "true", "yes", "on"}
82
83
84 def _cuda_available() -> bool:
85     """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here
and
86     ONLY when a caller opts into ``probe_cuda``; any import/runtime failure means 'no
GPU'."""
87     try:
88         # Intentionally lazy + local: keeps the config-load path torch-free (torch is
heavy
89         # and optional). Only reached when a caller opts into probe_cuda.
90         import torch
91
92         return bool(torch.cuda.is_available())
93     except Exception:
94         return False
95
96
97 def probe_cuda_available() -> bool:
98     """Public best-effort CUDA probe (lazy torch import; ``False`` on any failure).
99
100     Exposed for the per-profile startup checks (``backend/config/startup.py``) so a
caller
101     that *wants* the GPU heads-up can pass the result in without the startup module
importing
102     torch itself. Best-effort by design: if torch lives only in a detector subprocess
env
103     (native WASB), this returns ``False`` in the main process ? which is why the startup
checks
104     treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a
hard
105     error that could false-fail."""
106     return _cuda_available()
107
108
109 def suggest_profile(
110     env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
111 ) -> str:
112     """SUGGEST (never auto-apply ? D15) a deployment profile from the environment.

```

```

113
114     Signals, in priority order:
115
116     - ``K_SERVICE`` set (Cloud Run)           ? ``"cloud-serving"``
117     - ``CI`` truthy                             ? ``"cpu-mock"``
118     - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
119     - otherwise                                 ? ``""`` (no strong suggestion)
120
121     ``probe_cuda`` defaults to ``False`` so the common config-load path stays cheap and
never
122     imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe
passes
123     ``probe_cuda=True``. The result is advisory only: the caller decides whether to act
on it.
124     """
125     e = os.environ if env is None else env
126     if e.get("K_SERVICE"):
127         return "cloud-serving"
128     if _truthy(e.get("CI")):
129         return "cpu-mock"
130     if probe_cuda and _cuda_available():
131         return "truly-local"
132     return ""
133

```

8. user (2026-06-21T22:32:12.645Z)

```

1     import sys
2     import os
3     import subprocess
4     import shutil
5
6     # This diagnostics/bootstrap script lives in scripts/. REPO_ROOT (resolved from this
file's
7     # location, cwd-independent) is used for the repo-bundled requirements.txt. NOTE:
config.json is
8     # NO LONGER written here ? it follows backend.utils.app_home.default_config_path()
(RALLY_APP_HOME,
9     # else CWD), the SAME path the server READS, so --setup and the server can't disagree
(P0a).
10     REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
11
12     def check_command(cmd):
13         """Check if a system command is available."""
14         return shutil.which(cmd) is not None
15
16     def run_pip_install():
17         """Install required Python packages."""
18         print("Installing Python packages from requirements.txt...")
19
20         # Check if NVIDIA GPU is present
21         has_gpu = check_command("nvidia-smi")
22
23         requirements_path = os.path.join(REPO_ROOT, "requirements.txt")
24         pip_cmd = [sys.executable, "-m", "pip", "install", "--user", "-r",
requirements_path]
25
26         import platform
27         if has_gpu:
28             print("[INFO] NVIDIA GPU detected. Configuring PyTorch to download CUDA-enabled
wheels.")
29             pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cu124"])
30         elif platform.system() == "Darwin":
31             # macOS has no CUDA; the default PyPI wheel already ships CPU + Apple MPS.
32             print("[INFO] macOS detected. Using default PyTorch wheels (CPU + Apple MPS).")

```

```

33     else:
34         print("[INFO] No NVIDIA GPU detected. Defaulting to PyTorch CPU wheels.")
35         pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cpu"])
36
37     try:
38         subprocess.check_call(pip_cmd)
39         print("[SUCCESS] Python packages installed.")
40         return True
41     except subprocess.CalledProcessError as e:
42         print(f"[ERROR] Failed to install Python packages: {e}")
43         return False
44
45 def init_default_config():
46     """Initialize a default config.json if it doesn't exist.
47
48     Now delegates to the canonical backend.config loader so that defaults
49     stay in sync with the Pydantic models (single source of truth).
50     """
51     try:
52         from backend.config import save_default_config
53         from backend.utils.app_home import default_config_path
54     except Exception as e:
55         print(f"[ERROR] Could not import backend.config loader: {e}")
56         print("[INFO] Falling back to no-op for config initialization.")
57         return
58
59     # PACKAGING_PLAN.md P0a: write to the SAME app-home config path the server READS
60     # (RALLY_APP_HOME, else CWD/config.json) ? previously this wrote to the repo root
61     # the server read from the CWD, so --setup and the server disagreed once CWD ? repo
62     root.
63     config_path = str(default_config_path())
64     if os.path.exists(config_path):
65         print(f"[INFO] {config_path} already exists. Skipping initialization.")
66         return
67
68     try:
69         saved_path = save_default_config(config_path)
70         print(f"[SUCCESS] config.json initialized with default settings at {saved_path}.")
71     except Exception as e:
72         print(f"[ERROR] Failed to create config.json via loader: {e}")
73
74 def run_diagnostics():
75     """Perform system-wide checks for FFmpeg and PyTorch/CUDA."""
76     print("-" * 50)
77     print("SPORTS HIGHLIGHT INDEXER SETUP DIAGNOSTICS")
78     print("-" * 50)
79
80     # 1. Check Python Version
81     print(f"Python Version: {sys.version_info.major}.{sys.version_info.minor}.{sys.version_info.micro}")
82     if sys.version_info.major < 3 or (sys.version_info.major == 3 and sys.version_info.minor < 8):
83         print("[WARNING] Python 3.8+ is highly recommended!")
84     else:
85         print("[PASS] Python version check passed.")
86
87     print("-" * 50)
88
89     # 2. Check FFmpeg / FFprobe
90     ffmpeg_ok = check_command("ffmpeg")
91     ffprobe_ok = check_command("ffprobe")
92
93     if ffmpeg_ok:

```

```

93         print("[PASS] FFmpeg is installed and available in PATH.")
94     else:
95         print("[FAIL] FFmpeg is NOT found in your PATH.")
96
97     if ffprobe_ok:
98         print("[PASS] FFprobe is installed and available in PATH.")
99     else:
100         print("[FAIL] FFprobe is NOT found in your PATH.")
101
102     if not (ffmpeg_ok and ffprobe_ok):
103         import platform
104         print("\n" + "!" * 50)
105         print("CRITICAL: FFmpeg and FFprobe are REQUIRED to run this application.")
106         _sys = platform.system()
107         if _sys == "Windows":
108             print("Install on Windows: winget install Gyan.FFmpeg")
109         elif _sys == "Darwin":
110             print("Install on macOS: brew install ffmpeg")
111         else:
112             print("Install on Linux: sudo apt-get install -y ffmpeg (or your
113 distro's package manager)")
114             print("Or download a build from https://ffmpeg.org/download.html and put
115 ffmpeg/ffprobe on PATH.")
116             print("!" * 50 + "\n")
117         else:
118             print("[SUCCESS] All essential media command line tools are ready.")
119
120     print("-" * 50)
121
122     # 3. Check ML / GPU (Optional but good)
123     try:
124         import torch
125         cuda_ok = torch.cuda.is_available()
126         if cuda_ok:
127             print(f"[PASS] PyTorch loaded successfully. CUDA (GPU) is AVAILABLE:
128 {torch.cuda.get_device_name(0)}")
129         else:
130             print("[INFO] PyTorch loaded successfully, but CUDA (GPU) is NOT available.
131 Falling back to CPU.")
132     except ImportError:
133         print("[INFO] PyTorch is not installed. Relevance checks will run using
134 lightweight visual heuristics.")
135
136     print("=" * 50)
137     return ffmpeg_ok and ffprobe_ok
138
139 if __name__ == "__main__":
140     init_default_config()
141     media_ok = run_diagnostics()
142     run_pip_install()
143     print("\n[INFO] Setup complete. If diagnostics passed, you can now run the app!")

```

9. user (2026-06-21T22:32:12.657Z)

```

1     import os
2     import sys
3     import argparse
4     import json
5     import logging
6     import shutil
7     import socket
8     import threading
9     import uuid
10    import webbrowser

```

```

11     from importlib.metadata import PackageNotFoundError, version as _pkg_version
12     from urllib.parse import quote
13     import uvicorn
14     from dotenv import load_dotenv
15
16     # PACKAGING_PLAN.md P0a: prefer an app-home .env ONLY when RALLY_APP_HOME is set (the
packaged
17     # launcher pins it); otherwise env_file_to_load() returns None and load_dotenv(None)
keeps the
18     # exact historical frame-relative walk-up ? byte-identical to the pre-P0a bare
load_dotenv().
19     # stdlib-only import ? keeps this pre-pydantic line light.
20     from backend.utils.app_home import env_file_to_load, resolve_output_dir
21
22     load_dotenv(env_file_to_load())
23
24     # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
25     logging.basicConfig(
26         level=logging.INFO,
27         format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
28         datefmt='%H:%M:%S'
29     )
30     logger = logging.getLogger(__name__)
31     from fastapi import FastAPI, HTTPException, Request
32     from fastapi.middleware.cors import CORSMiddleware
33     from starlette.middleware.trustedhost import TrustedHostMiddleware
34     from pydantic import BaseModel
35     from typing import List, Dict, Any, Optional
36
37     from fastapi.staticfiles import StaticFiles
38     from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
39     from backend.pipeline.validators.base import registry
40     from backend.infrastructure.database import Database
41     from backend.pipeline.segmenters.base import segmenter_registry
42     from backend.pipeline.stitcher.compiler import VideoCompiler
43     from backend.pipeline.stitcher.watermark_i18n import localize_watermark
44     from backend.utils.hashing import compute_video_id # canonical file-identity hashing
45     from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
46     from backend.utils.redact import redact_secrets # P0b: mask secret-shaped fields in
/api/config
47     from backend.utils.single_instance import acquire_lock # P0e: one instance per database
48     from backend.utils.ffmpeg import ffmpeg_bin, ffprobe_bin # P2a: single ffmpeg/ffprobe
resolver
49     from backend.config import ( # new typed config
50         load_config as load_app_config,
51         AppConfig,
52         StitchingConfig,
53         enforce_startup_checks,
54         StartupCheckError,
55     )
56     from backend.results import Failure # standardized error/result contract
57     from backend.reporting import emit_indexer_report
58     from backend import storage # M2: URI-aware input acquisition (local = identity
passthrough)
59     from backend import billing # M8: per-job cost ledger (USD internal / INR display)
60     from backend.api import auth as edge_auth # M9: Cloudflare Access edge-auth (default-
OFF)
61
62     app = FastAPI(title="Sports Highlight Indexer API")
63
64
65     def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
66         """M9: CORS allow-origins from the ``RALLY_CORS_ALLOW_ORIGINS`` env var (comma-

```

```

separated),
67         default ``["*"]``. Read from the ENV (not a cwd ``config.json``) so importing
``backend.main``
68         does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to
its origin.""
69         src = os.environ if env is None else env
70         raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
71         origins = [o.strip() for o in raw.split(",") if o.strip()]
72         return origins or ["*"]
73
74
75     # CORS for the web UI. allow_origins='' with allow_credentials=True is rejected by
browsers per the
76     # fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so
credentials stay
77     # off. The origin list is configurable via the env var above (default ["*"]) so a hosted
deploy can
78     # lock it to its origin; the middleware is fixed at startup.
79     app.add_middleware(
80         CORSMiddleware,
81         allow_origins=_cors_origins_from_env(),
82         allow_credentials=False,
83         allow_methods=["*"],
84         allow_headers=["*"],
85     )
86
87
88     def _allowed_hosts_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
89         """P0c: Host-header allowlist ? a DNS-rebinding guard for the localhost appliance. A
malicious
90         web page can point a domain it controls at 127.0.0.1 and script requests to the
local server;
91         the browser sends ``Host: attacker.com``, which this rejects (the server only
answers loopback
92         Host headers by default). ``RALLY_ALLOWED_HOSTS`` (comma-separated) overrides; the
default keeps
93         Starlette's TestClient host ``testserver`` so the in-process suite isn't broken.
Read from the
94         ENV (not config.json) so importing ``backend.main`` does zero filesystem I/O; a
routable/edge
95         deploy sets its public host (or ``*`` when fronted by an auth gate that validates
the host)."""
96         src = os.environ if env is None else env
97         raw = (src.get("RALLY_ALLOWED_HOSTS", "") or "").strip()
98         hosts = [h.strip() for h in raw.split(",") if h.strip()]
99         # IPv4 loopback (the default bind) + the TestClient host. NB: Starlette's
TrustedHostMiddleware
100         # matches on host.split(":")[0], which mangles a bracketed IPv6 Host ("[::1]:port"),
so IPv6
101         # loopback is NOT supported by this guard ? access the appliance via
127.0.0.1/localhost.
102         return hosts or ["localhost", "127.0.0.1", "testserver"]
103
104
105     # DNS-rebinding guard. Bound to localhost by default, so only loopback Host headers are
honoured
106     # (a remote page can't read responses cross-origin AND its Host header is rejected
here). www_redirect
107     # off so an unexpected ``www.`` host is rejected rather than 307-redIRECTED.
108     app.add_middleware(
109         TrustedHostMiddleware,
110         allowed_hosts=_allowed_hosts_from_env(),
111         www_redirect=False,
112     )
113

```

```

114
115     def _resolve_bind_host(config: Optional[AppConfig] = None, cli_host: Optional[str] =
None) -> str:
116         """P0c: the server bind address. Precedence: ``--host`` > ``RALLY_BIND_HOST`` env >
``0.0.0.0``
117         when an edge-auth profile is active (``security.edge_auth_enabled`` ? i.e. there IS
an auth gate
118         in front) > ``127.0.0.1`` (default: localhost-only, today's behaviour). Binding a
routable
119         interface with no auth gate is unsafe, hence the localhost default; widen it
consciously."""
120         if cli_host and cli_host.strip():
121             return cli_host.strip()
122         env = os.environ.get("RALLY_BIND_HOST")
123         if env and env.strip():
124             return env.strip()
125         sec = getattr(config, "security", None)
126         if getattr(sec, "edge_auth_enabled", False):
127             return "0.0.0.0" # behind a Cloudflare-Access-style edge gate (M9) ? bind all
interfaces
128         return "127.0.0.1"
129
130
131     # --- P2: the friendly `indexer --app` launcher (cross-platform, no native component ?
01) ---
132
133     def _port_is_free(host: str, port: int) -> bool:
134         """True if (host, port) can be bound right now (best-effort; a TOCTOU race is
acceptable ?
135         uvicorn surfaces a real bind failure loudly)."""
136         with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
137             try:
138                 s.bind((host, port))
139                 return True
140             except OSError:
141                 return False
142
143
144     def _find_free_port(host: str) -> int:
145         """An OS-assigned free port on host."""
146         with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
147             s.bind((host, 0))
148             return int(s.getsockname()[1])
149
150
151     def _resolve_app_port(host: str, port: int) -> int:
152         """For --app: keep `port` if free, else fall back to a free one so a busy 8000 never
blocks the app."""
153         return port if _port_is_free(host, port) else _find_free_port(host)
154
155
156     def _open_browser_when_ready(url: str, health_url: str, timeout: float = 60.0) -> bool:
157         """Poll `health_url` until the server is up, then open `url` in the user's default
browser.
158         Returns True if a browser open was attempted. No-op (returns False) when
RALLY_NO_BROWSER is set
159         (tests / headless) or the server never becomes healthy (don't open a dead page).
Never raises."""
160         if os.environ.get("RALLY_NO_BROWSER"):
161             return False
162         import time as _time
163         import urllib.request
164         deadline = _time.monotonic() + timeout
165         while _time.monotonic() < deadline:
166             try:

```



```

167         with urllib.request.urlopen(health_url, timeout=2) as r:
168             if getattr(r, "status", 200) == 200:
169                 break
170             except Exception:
171                 _time.sleep(0.3)
172     else:
173         return False
174     try:
175         webbrowser.open(url)
176     except Exception:
177         pass
178     return True
179
180
181     # Serves static frontend files directly (now under api/ per approved structure).
182     # PACKAGING_PLAN.md P0a: resolve PACKAGE-relative (not CWD-relative) so launching from
183     # an arbitrary directory finds the shipped assets instead of creating a stray
184     # (The legacy in-engine dashboard; the real SPA gets bundled here in P1a.)
185     _STATIC_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "api", "static")
186     try:
187         os.makedirs(_STATIC_DIR, exist_ok=True)
188     except OSError:
189         # The dir ships with the package (a no-op create); guard against a read-only install
190         # tree (system-installed Python) since exist_ok suppresses FileExistsError, not
191         # PermissionError.
192         pass
193     app.mount("/static", StaticFiles(directory=_STATIC_DIR), name="static")
194
195     # P1b: serve the bundled KhelSutra SPA single-origin. `backend/ui/` is the committed
196     # snapshot
197     # (produced by scripts/bundle_ui.sh); fall back to the legacy in-engine dashboard only
198     # if it's absent (a source checkout with no vendored UI). The SPA calls same-origin `/api`, so
199     # no SPA change is needed; it is query-param driven (?video=?), so serving index.html at `/` + /assets
200     # is enough
201     # (no client-side path routing ? no history-fallback catch-all, which would also be a
202     # route-order hazard).
203     _BUNDLED_UI_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "ui")
204     _UI_DIR = _BUNDLED_UI_DIR if os.path.isfile(os.path.join(_BUNDLED_UI_DIR, "index.html"))
205     else _STATIC_DIR
206     _UI_ASSETS_DIR = os.path.join(_UI_DIR, "assets")
207     if os.path.isdir(_UI_ASSETS_DIR):
208         app.mount("/assets", StaticFiles(directory=_UI_ASSETS_DIR), name="assets")
209
210
211     @app.get("/", response_class=HTMLResponse)
212     def serve_index():
213         index_path = os.path.join(_UI_DIR, "index.html")
214         if os.path.exists(index_path):
215             with open(index_path, "r", encoding="utf-8") as f:
216                 return f.read()
217         return "<h3>Sports Highlight Indexer Server is Running. (No bundled UI found.)</h3>"
218
219
220     def _bundled_ui_version() -> Optional[Dict[str, Any]]:
221         """Provenance of the vendored SPA (backend/ui/ui_version.json), or None if no UI is
222         bundled.
223         Includes `engine_api_version` ? the API_VERSION the snapshot was built against (skew
224         guard)."""
225         path = os.path.join(_BUNDLED_UI_DIR, "ui_version.json")
226         try:
227             with open(path, "r", encoding="utf-8") as f:

```

```

221         return json.load(f)
222     except (OSError, ValueError):
223         return None
224
225     # Global variables initialized at startup
226     db_instance: Optional[Database] = None
227     global_config: Optional[AppConfig] = None # now typed (Pydantic model)
228     # P0e: the single-instance lock handle. Kept module-global for the process lifetime ? if
it were
229     # GC'd/closed the OS would release the lock. None until main() acquires it (server/CLI
init).
230     _instance_lock: Optional[Any] = None
231
232     # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
233     # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
234     # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
235     # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
236     # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
237     # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
238     # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
239     from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable
on backend.main)
240         JobWaiting,
241         _arm_wake_timer,
242         _drain_due_jobs,
243         _get_executor,
244         _job_to_request,
245         _MAX_DRAIN_WAKE_SEC,
246         _maybe_record_job_cost,
247         _run_claimed_job,
248         _schedule_next_drain_if_waiting,
249     )
250
251     # Legacy thin wrapper for any remaining direct calls during transition.
252     # New code should import load_app_config / AppConfig from backend.config directly.
253     def load_config(config_path: str = "config.json") -> Dict[str, Any]:
254         cfg = load_app_config(config_path)
255         return cfg.model_dump(mode="json")
256
257     # --- API Models ---
258     class ProcessRequest(BaseModel):
259         video_path: str
260         player_pool: Optional[List[str]] = None
261         max_frames: Optional[int] = None
262         segmenter: Optional[str] = None
263         # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Recorded on the job
for PMF
264         # analytics (count_jobs_by_locale). Optional ? NULL when unrecorded (CLI / older
clients).
265         locale: Optional[str] = None
266
267     class QueryRequest(BaseModel):
268         video_id: str
269         server: Optional[str] = None
270         receiver: Optional[str] = None
271         duration_min: Optional[float] = None
272         event_type: Optional[str] = None
273
274     class CompileRequest(BaseModel):
275         video_id: str
276         segment_ids: List[int]
277         output_filename: str
278         # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Localizes the reel

```

```

watermark
279     # (text + script font). Optional ? absent/unknown keeps the configured default (en
behavior).
280     locale: Optional[str] = None
281
282     def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
283         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
284
285         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
286         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
287         and
288         keep the prior good results intact ? a failed/empty re-run never destroys prior
289         data.
290         """
291         if segments:
292             db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
293         else:
294             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
295
296     # --- API Endpoints ---
297     @app.get("/api/config")
298     def get_config():
299         # P0b: this endpoint is unauthenticated, so never serialize a credential-shaped
300         field. Today
301         # AppConfig holds no secrets (the Gemini key is env-only; cf_access_aud is a public
302         id), so this
303         # is byte-identical for the real fields ? but it fails-closed if a secret is ever
304         added (guarded
305         # by tests/test_redact.py + the /api/config redaction test).
306         if global_config is None:
307             return {}
308         return redact_secrets(global_config.model_dump(mode="json"))
309
310     @app.get("/api/health")
311     def health():
312         """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
313         (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
314         confirm the engine is reachable and which detector is wired. Never raises."""
315         sport = getattr(global_config, "sport", None)
316         indexing = getattr(global_config, "indexing", None)
317         return {
318             "status": "ok",
319             "sport": sport,
320             "default_segmenter": getattr(indexing, "default_segmenter", None),
321         }
322
323     # P1: the bundled SPA pins this ? the contract version of the /api surface. BUMP when an
324     existing
325     # endpoint's request/response shape changes incompatibly (purely additive endpoints need
326     no bump).
327     API_VERSION = 1
328
329     def _engine_version() -> str:
330         """Installed package version (wheel/editable); 'unknown' if not importable as a
331         distribution."""
332         try:
333             return _pkg_version("badminton-highlight-indexer")
334         except PackageNotFoundError:
335             return "unknown"
336
337     @app.get("/api/version")

```

```

334     def api_version():
335         """Version handshake so a bundled SPA can detect engine skew (PACKAGING_PLAN.md
336         P1)."""
337         return {"engine_version": _engine_version(), "api_version": API_VERSION, "ui":
338         _bundled_ui_version()}
339
340     @app.get("/api/capabilities")
341     def capabilities():
342         """Honest probe of what THIS box can do ? the first-run wizard renders the compute
343         step from
344         it (PACKAGING_PLAN.md P1). No secrets: reports only whether a Gemini key is PRESENT,
345         never its
346         value. Best-effort + never raises (the wizard degrades gracefully)."""
347         indexing = getattr(global_config, "indexing", None)
348         deployment = getattr(global_config, "deployment", None)
349         ffmpeg = shutil.which(ffmpeg_bin()) # P2a: honor RALLY_FFMPEG / RALLY_FFPROBE
350         ffprobe = shutil.which(ffprobe_bin())
351         # WASB weights are best-effort: env first (matches native_wasb_runner), then config.
352         wasb = getattr(indexing, "wasb", None)
353         weights_path = os.environ.get("WASB_WEIGHTS") or getattr(wasb, "weights_path", "")
354         or ""
355         # Cheap GPU heuristic: nvidia-smi on PATH (matches scripts/doctor.py). Deliberately
356         NOT
357         # torch.cuda.is_available() ? torch is absent in the LIGHT tier (so it would false-
358         negative on a
359         # real GPU box) and importing it would add seconds to this probe. The detector
360         healthcheck is the
361         # authority on whether GPU inference actually works; this is just the wizard's
362         first-order signal.
363         nvidia_smi = shutil.which("nvidia-smi")
364         return {
365             "api_version": API_VERSION,
366             "engine_version": _engine_version(),
367             "segmenters": sorted(segmenter_registry.list_available().keys()),
368             "default_segmenter": getattr(indexing, "default_segmenter", None),
369             "ffmpeg": {"ffmpeg": bool(ffmpeg), "ffprobe": bool(ffprobe),
370                       "ffmpeg_path": ffmpeg, "ffprobe_path": ffprobe},
371             "gpu": {"nvidia_smi_present": bool(nvidia_smi)},
372             "gemini_key_present": bool(os.environ.get("GEMINI_API_KEY")),
373             "wasb_weights_present": bool(weights_path) and os.path.exists(weights_path),
374             "deployment": {
375                 "profile": getattr(deployment, "profile", "") or "",
376                 "compute_target": getattr(deployment, "compute_target", "") or "",
377             },
378         }
379
380     @app.get("/api/videos")
381     def list_videos(limit: Optional[int] = None):
382         """List indexed videos (most recent first) so the console survives a reload ?
383         previously only a
384         single-row get existed (PACKAGING_PLAN.md P1)."""
385         return {"videos": db_instance.list_videos(limit=limit)}
386
387     def _reels_dir() -> str:
388         return getattr(getattr(global_config, "output", None), "output_dir", "output") or
389         "output"
390
391     _REEL_EXTS = (".mp4", ".mov")
392
393     @app.get("/api/reels")

```

```

388     def list_reels():
389         """List compiled highlight reels present in the output dir, newest first, with a
download URL.
390
391         NB: reels are stored FLAT in output_dir under the bare name passed to /api/compile ?
the engine
392         does NOT track a video_id?reel association today (a per-video listing would be a
phantom), so
393         this is a global list. (Per-video reel tracking would need a DB record + compile
change ? a
394         later slice.) Only top-level video files are listed (per-video scratch subdirs are
skipped)."""
395         base = _reels_dir()
396         reels = []
397         try:
398             entries = os.scandir(base)
399         except OSError:
400             return {"reels": []}
401         with entries:
402             for e in entries:
403                 if e.is_file() and e.name.lower().endswith(_REEL_EXTS):
404                     try:
405                         st = e.stat()
406                     except OSError:
407                         continue
408                     reels.append({
409                         "filename": e.name,
410                         "size_bytes": st.st_size,
411                         "modified_at": st.st_mtime,
412                         "download_url": f"/api/reels/{quote(e.name)}",
413                     })
414         reels.sort(key=lambda r: r["modified_at"], reverse=True)
415         return {"reels": reels}
416
417
418     @app.get("/api/reels/{filename}")
419     def download_reel(filename: str):
420         """Stream a compiled reel by its bare filename (no video_id ? reels are flat in
output_dir).
421         Mirrors /api/highlights' safety (bare-name + containment checks)."""
422         try:
423             name = safe_output_filename(filename)
424         except ValueError as e:
425             raise HTTPException(status_code=400, detail=str(e)) from e
426         base = _reels_dir()
427         path = os.path.join(base, name)
428         try:
429             path = resolve_under_root(path, base)
430         except ValueError as e:
431             raise HTTPException(status_code=400, detail=str(e)) from e
432         if not os.path.isfile(path) or not name.lower().endswith(_REEL_EXTS):
433             raise HTTPException(status_code=404, detail=f"No compiled reel named '{name}'.")
434         media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
435         return FileResponse(path, media_type=media, filename=name)
436
437
438     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
439         """The full hash ? validate ? segment ? report pipeline for one video.
440
441         This is the former synchronous /api/process body, unchanged in behavior, now run on
the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
the sync endpoint used to return (UI compatibility); the per-video observability
report is still emitted in `finally:` and grafted as response["reports"].
444
445         `progress` is an optional callable(stage: str) used to surface coarse job progress.
446

```

```

447     Raises on unexpected internal errors ? the caller records those as a job failure
448     (after this function has already restored the pre-run segments, see below).
449     ""
450     notify = progress or (lambda stage: None)
451
452     notify("hashing")
453     # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
454     # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
455     # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
456     # consumers (hash / validators / segmenter) use the resolved local path.
457     local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
458     video_id = compute_video_id(local_video)
459     was_reingest = db_instance.get_video(video_id) is not None
460
461     # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
462     notify("validating")
463     logger.info(f"Running validations for {req.video_path}...")
464     val_results, val_context = registry.run_all_with_context(local_video, global_config)
465     failures = [r for r in val_results if not r.passed]
466
467     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
468     default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
469     seg_name = req.segmenter or default_seg
470     segmenter_actual = None
471     segmentation_ran = False
472     response: Optional[Dict[str, Any]] = None
473     try:
474         if failures:
475             # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
476             # status; it no longer destroys the video's prior good segments.
477             db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
478             response = {
479                 "video_id": video_id,
480                 "status": "failed",
481                 "message": "Video failed validation checks.",
482                 "results": [r.model_dump() for r in val_results],
483             }
484             return response
485
486     # 2. Run segmenter & indexer
487     logger.info("[API] Video passed checks. Segmenting and indexing...")
488     db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
489
490     segmenter_cls = segmenter_registry.get(seg_name)
491     if not segmenter_cls:
492         # Submit-time validation already 400s unknown names; this is the defensive
493         # in-worker path (e.g. config default pointing at an unregistered
segmenter).
494         available = list(segmenter_registry.list_available().keys())
495         response = {
496             "video_id": video_id,
497             "status": "failed",
498             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
499             "results": [r.model_dump() for r in val_results],
500             "play_segments": [],
501         }

```

```

502         return response
503
504         logger.info(f"[API] Using segmenter: {seg_name}")
505         notify(f"segmenting ({seg_name})")
506         segmenter_actual = seg_name
507         # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
508         only
509         # if the run yields segments; otherwise drop the new partial rows and keep the
510         old.
511         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
512         segmenter = segmenter_cls(db_instance, global_config)
513         try:
514             result = segmenter.process_video(video_id, local_video, req.player_pool,
515             req.max_frames)
516         except Exception:
517             # A raising segmenter must not leave half-written rows: restore the pre-run
518             # state (same destroy-after-confirm contract as the Failure branch), then
519             let
520             # the job worker record the failure.
521             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
522             raise
523         segmentation_ran = True
524
525         if isinstance(result, Failure):
526             logger.error(f"[API] Segmenter failed: {result.message}")
527             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
528             response = {
529                 "video_id": video_id,
530                 "status": "failed",
531                 "message": f"Segmenter '{seg_name}' failed: {result.message}",
532                 "results": [r.model_dump() for r in val_results],
533                 "play_segments": [],
534             }
535             return response
536
537         segments = result.value if hasattr(result, "value") else result
538         notify("finalizing")
539         _finalize_reingest(video_id, segments, play_wm, cand_wm)
540
541         response = {
542             "video_id": video_id,
543             "status": "success",
544             "message": f"Successfully indexed {len(segments)} play segments using
545             '{seg_name}' segmenter.",
546             "results": [r.model_dump() for r in val_results],
547             "play_segments": segments,
548         }
549         return response
550     finally:
551         # Always emit the per-video observability report (next to the video, or to
552         # report.output_dir). Never raises. Surface the paths in the response.
553         paths = emit_indexer_report(
554             db=db_instance, video_id=video_id, video_path=req.video_path,
555             config=global_config,
556             val_results=val_results, val_context=val_context,
557             segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
558             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
559         )
560         if paths and isinstance(response, dict):
561             response["reports"] = paths
562
563     @app.post("/api/process", status_code=202)
564     def process_video(req: ProcessRequest, request: Request):
565         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.

```

```

561
562     Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
563     client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
564     runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
565     worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
566     ""
567     # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
568     # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
569     # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
570     try:
571         owner_id = edge_auth.resolve_tenant(request.headers, global_config)
572     except edge_auth.EdgeAuthError as e:
573         raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
574
575     allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
576     try:
577         validate_input_path(req.video_path, allowed_root=allowed_root)
578         # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
579         # scheme, which we map to a clean 400 (an unknown scheme must never 500).
580         input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
581     except ValueError as e:
582         raise HTTPException(status_code=400, detail=str(e)) from e
583     # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
584     # (input_ref.key ? e.g. local:/// stripped to the bare path), not the raw URI
string. A
585     # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
586     # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
587     # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
588     # rejects a non-local URI as out-of-root.
589     if input_ref.backend == "local" and not os.path.exists(input_ref.key):
590         raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
591     if req.segmenter and not segmenter_registry.get(req.segmenter):
592         available = list(segmenter_registry.list_available().keys())
593         raise HTTPException(
594             status_code=400,
595             detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
596         )
597
598     job_id = uuid.uuid4().hex
599     if not db_instance.create_job(job_id, req.video_path, req.segmenter,
600                                req.player_pool, req.max_frames, owner_id=owner_id,
601                                locale=req.locale):
602         raise HTTPException(status_code=500, detail="Failed to persist job record.")
603     _get_executor().submit(_drain_due_jobs)
604     return JSONResponse(
605         status_code=202,
606         content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
607     )
608
609
610 @app.get("/api/jobs/{job_id}/cost")
611 def get_job_cost(job_id: str):
612     """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of

```



```

truth (matches
613         GCP billing); ``cost_inr`` is a display conversion at the configured FX rate.
``cost_usd`` is
614         null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
615         cost = db_instance.get_job_cost(job_id)
616         if cost is None:
617             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
618         fx = global_config.billing.fx_inr_per_usd
619         usd = cost.get("cost_usd")
620         cost["cost_inr"] = billing.to_inr(usd, fx) if usd is not None else None
621         cost["fx_inr_per_usd"] = fx
622         return cost
623
624
625     @app.get("/api/videos/{video_id}/cost")
626     def get_video_cost(video_id: str):
627         """Aggregate per-video cost across its jobs (M8): ``total_cost_usd`` +
``total_cost_inr`` + the
628         per-job rows. A video is 1:1 with an infer job, but a re-ingest can produce
several."""
629         agg = db_instance.get_video_cost(video_id)
630         fx = global_config.billing.fx_inr_per_usd
631         agg["total_cost_inr"] = billing.to_inr(agg["total_cost_usd"], fx)
632         agg["fx_inr_per_usd"] = fx
633         return agg
634
635
636     @app.get("/api/jobs/{job_id}")
637     def get_job(job_id: str):
638         """Job state: {status, progress, result, reports}. `status` = execution state
639         (queued|running|done|failed); the pipeline verdict is result["status"]."""
640         job = db_instance.get_job(job_id)
641         if not job:
642             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
643         result = job.get("result")
644         return {
645             "job_id": job["id"],
646             "status": job["status"],
647             "progress": job.get("progress"),
648             "video_id": job.get("video_id"),
649             "error": job.get("error"),
650             "result": result,
651             "reports": (result or {}).get("reports"),
652             "created_at": job.get("created_at"),
653             "started_at": job.get("started_at"),
654             "finished_at": job.get("finished_at"),
655         }
656
657     # --- Chunked upload (B2, PR-2) -----
658     # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
in
659     # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
via
660     # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
661     # sequential-part logic, and abort cleanup ? is unchanged.
662     from backend.api import uploads as uploads_api
663
664     app.include_router(uploads_api.router)
665
666
667     # --- Highlight download (B3, PR-2) -----
668
669     @app.get("/api/highlights/{video_id}/{filename}")
670     def download_highlight(video_id: str, filename: str):
671         """Stream a compiled highlight reel ? `filename` is the bare name given to

```

```

/api/compile
672     (which only writes into output_dir; the browser previously got back an unreachable
673     server-local path)."""
674     try:
675         name = safe_output_filename(filename)
676     except ValueError as e:
677         raise HTTPException(status_code=400, detail=str(e)) from e
678     if not db_instance.get_video(video_id):
679         raise HTTPException(status_code=404, detail="Indexed video record not found.")
680     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
681     path = os.path.join(output_dir, name)
682     try:
683         path = resolve_under_root(path, output_dir)
684     except ValueError as e:
685         raise HTTPException(status_code=400, detail=str(e)) from e
686     if not os.path.isfile(path):
687         raise HTTPException(status_code=404,
688                             detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
689     media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
690     return FileResponse(path, media_type=media, filename=name)
691
692
693 @app.post("/api/query")
694 def query_play_segments(req: QueryRequest):
695     filters = {
696         "server": req.server,
697         "receiver": req.receiver,
698         "duration_min": req.duration_min,
699         "event_type": req.event_type
700     }
701     segments = db_instance.query_play_segments(req.video_id, filters)
702     return {"play_segments": segments}
703
704 @app.post("/api/compile")
705 def compile_highlights(req: CompileRequest):
706     video = db_instance.get_video(req.video_id)
707     if not video:
708         raise HTTPException(status_code=404, detail="Indexed video record not found.")
709
710     # Retrieve matching play segments from db
711     all_segments = db_instance.query_play_segments(req.video_id, {})
712     matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
713
714     if not matched_segments:
715         raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")
716
717     # Reject path traversal / absolute output names (os.path.join would otherwise let an
718     # absolute or '..'-laden output_filename write anywhere on disk).
719     try:
720         out_name = safe_output_filename(req.output_filename)
721     except ValueError as e:
722         raise HTTPException(status_code=400, detail=str(e)) from e
723
724     # The configured shot-count floor (stitching.min_shot_count) applies on the API
725     # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
shot_count
726     # metadata are exempt: the floor filters known counts only, so explicitly
727     # requested manual/legacy segments can't silently vanish under the default floor.
728     stitching = getattr(global_config, "stitching", None) or StitchingConfig()
729     kept = []
730     for s in matched_segments:
731         meta = s.get("metadata") or {}

```

```

732         sc = meta.get("shot_count") if isinstance(meta, dict) else None
733         try:
734             if sc is not None and int(sc) < stitching.min_shot_count:
735                 continue
736         except (TypeError, ValueError):
737             pass # unparseable count -> treat as unknown, keep
738         kept.append(s)
739     if len(kept) < len(matched_segments):
740         logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
dropped "
741                     f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
segments below the floor.")
742     if not kept:
743         raise HTTPException(status_code=400,
744                             detail=f"All {len(matched_segments)} matching segments fall
below "
745                                 f"stitching.min_shot_count={stitching.min_shot_count}.")
746
747     # Extract start/end time pairs
748     time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
749
750     # Run compiler with the configured stitching paddings + optional branding watermark,
localized to
751     # the UI locale the SPA rendered in (req.locale): the reel's watermark reads in the
user's language
752     # with a script-appropriate font. Unknown/absent locale keeps the configured default
unchanged.
753     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
754     localized_watermark = localize_watermark(stitching.watermark, req.locale)
755     compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
756                             end_padding=stitching.end_padding,
watermark=localized_watermark)
757     try:
758         # The stored `filepath` is the SOURCE ref ? a bucket/Drive URI (gs://, s3://,
gdrive://) for a
759         # video ingested via /api/process from a URI, or a plain local path. Resolve it
to a LOCAL file
760         # the stitcher can read: identity for a local path (byte-identical to before),
fetch-to-scratch
761         # for a bucket URI (the same seam the process path uses, _execute_processing
main.py:159).
762         # Without this, compiling a bucket-ingested video 500s ? ffmpeg can't open a
gs:// path.
763         local_src = storage.acquire_local_input(video["filepath"],
getattr(global_config, "storage", None))
764         out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
765         # `filepath` is server-local (not reachable by the browser); also return the
ready-to-use
766         # download URL so the SPA needn't hand-build it (P6 / PER_RALLY_SELECTION.md
§4.3).
767         download_url = f"/api/highlights/{quote(req.video_id)}/{quote(out_name)}"
768         return {"status": "success", "filepath": out_path, "download_url": download_url}
769     except Exception as e:
770         raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e
771
772     # --- CLI Debug Utility & Orchestration ---
773     def run_cli_diagnose_clip(video_path: str, timestamp: float):
774         """CLI debugging utility to examine segment properties around a timestamp."""
775         print("=" * 60)
776         print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
777         print("=" * 60)
778

```

```

779     cfg = load_config() # legacy wrapper still works, returns dict for now
780
781     # 1. Validation check diagnostics
782     print("[DIAGNOSTIC] Running validation framework...")
783     results = registry.run_all(video_path, cfg)
784     for r in results:
785         status_symbol = "[PASS]" if r.passed else "[FAIL]"
786         print(f" {status_symbol} {r.validator_name}: {r.message}")
787         if r.details:
788             print(f"         Details: {r.details}")
789
790     # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
791     print("-" * 60)
792     print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
793     import cv2
794     cap = cv2.VideoCapture(video_path)
795     if not cap.isOpened():
796         print("[FAIL] OpenCV could not open video.")
797         return
798
799     fps = cap.get(cv2.CAP_PROP_FPS)
800     total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
801
802     start_frame = max(0, int((timestamp - 3.0) * fps))
803     end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
804
805     # Measure frame-by-frame changes in sample region
806     cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
807     prev_gray = None
808     motions = []
809
810     for _ in range(start_frame, end_frame):
811         ret, frame = cap.read()
812         if not ret:
813             break
814         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
815         gray = cv2.GaussianBlur(gray, (21, 21), 0)
816
817         if prev_gray is not None:
818             diff = cv2.absdiff(prev_gray, gray)
819             _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
820             motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
821             motions.append(motion_ratio)
822             prev_gray = gray
823
824     cap.release()
825
826     if motions:
827         avg_motion = sum(motions) / len(motions)
828         # Support both legacy dict (from wrapper) and future direct model
829         if hasattr(cfg, "indexing"):
830             threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
831         else:
832             threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08) # type:
ignore[attr-defined]
833         print(f" Sample Frame Count: {len(motions)}")
834         print(f" Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
835         if avg_motion > threshold:
836             print(" Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
837         else:
838             print(" Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")

```

```

839         else:
840             print(" [ERROR] No visual frames were extracted in the sample range.")
841
842         print("=" * 60)
843
844
845     def _enforce_startup_or_exit(config) -> None:
846         """M5: run the startup checks for the server path and ``sys.exit(1)`` on a FATAL
finding BEFORE
847         any work ? a fatal incoherent profile (e.g. cloud-serving pointed at local-only
storage) OR, for
848         the exposed server (``include_exposure=True``, M9 hardening), a half-configured
exposure (edge
849         auth on without the ``allowed_video_root`` jail / CF team+AUD). A strict no-op when
no
850         ``deployment.profile`` is selected AND edge auth is off (today's behaviour). No GPU
probe here ?
851         the server stays torch-light; the detector healthcheck is the GPU authority.
Extracted so the
852         fail-fast wiring + the default-OFF no-op are unit-testable."""
853         try:
854             enforce_startup_checks(config, logger=logger, include_exposure=True)
855         except StartupCheckError as e:
856             logger.error("[STARTUP] refusing to start: %s", e)
857             sys.exit(1)
858
859
860     def main():
861         parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
862         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
863         parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
864         parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
865         parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
866         parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
867         parser.add_argument("--app", action="store_true",
868                             help="Launch the app: start the local server (localhost-only)
and open the "
869                             "bundled UI in your browser. Falls back to a free port if
--port is busy.")
870         parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
871         parser.add_argument("--host", default=None,
872                             help="Bind address. Default 127.0.0.1 (localhost-only); 0.0.0.0
when "
873                             "security.edge_auth_enabled. Expose on a network ONLY
behind an auth gate.")
874         parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
875
876         available_segmenters = list(segmenter_registry.list_available().keys())
877         parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
878         parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
879         parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
880         parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
881
882         args = parser.parse_args()

```

```

883
884     if args.setup:
885         # Invoke the diagnostics/bootstrap script (renamed from the old root
886         # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
887         # build system). Resolve its path from this file so `indexer --setup`
888         # works regardless of the caller's cwd.
889         import subprocess
890         doctor_path = os.path.join(
891             os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
892             "scripts", "doctor.py",
893         )
894         print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
895         subprocess.run([sys.executable, doctor_path])
896         return
897
898     if args.trim_start is not None and args.trim_end is not None:
899         if not args.video_path:
900             print("[ERROR] Please provide --video-path to extract a segment.")
901             return
902
903         # Determine output path
904         out_filename = args.trim_output or "trimmed_segment.mp4"
905         if os.path.isabs(out_filename):
906             out_path = out_filename
907             out_dir = os.path.dirname(out_path)
908             out_name = os.path.basename(out_path)
909         else:
910             out_dir = resolve_output_dir(args.output_dir) # P0a: app-home aware (CWD-
identical when unset)
911             out_path = os.path.join(out_dir, out_filename)
912             out_name = out_filename
913
914             os.makedirs(out_dir, exist_ok=True)
915             print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
916
917             # global_config isn't initialized this early; load the typed config so the trim
918             # clip carries the same configured stitching paddings + watermark as a real
919             # highlight (watermark default-on).
920             stitching = load_app_config().stitching
921             compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
922                                     end_padding=stitching.end_padding,
watermark=stitching.watermark)
923             try:
924                 res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
925                 print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
926             except Exception as e:
927                 print(f"[ERROR] Failed to compile trimmed segment: {e}")
928             return
929
930     if args.debug_clip is not None:
931         if not args.video_path:
932             print("[ERROR] Please provide --video-path to debug a specific clip.")
933             return
934         run_cli_diagnose_clip(args.video_path, args.debug_clip)
935         return
936
937     # Initialize Global Config and DB
938     global global_config, db_instance
939     global_config = load_app_config() # returns typed AppConfig
940     _enforce_startup_or_exit(global_config) # M5: fail fast on an incoherent ACTIVE
profile
941
942     # The CLI --output-dir overrides the configured output directory. It now lives

```

```

943     # on the typed model (OutputConfig.output_dir), so every consumer ? including
944     # /api/compile ? reads the same value instead of a write-only runtime hack.
945     # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
946     # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to
before.
947     global_config.output.output_dir = resolve_output_dir(args.output_dir)
948     os.makedirs(global_config.output.output_dir, exist_ok=True)
949
950     db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
951
952     # P0e: refuse a SECOND instance against the same DB BEFORE the destructive stale-job
sweep below.
953     # The job worker is in-process, so a concurrent process opening this DB would sweep
THIS one's
954     # in-flight jobs to 'failed'. The OS lock (held for the process lifetime; auto-
released on exit
955     # or crash) is keyed to db_path, so a different --output-dir/DB still runs
concurrently.
956     global _instance_lock
957     try:
958         _instance_lock = acquire_lock(db_path + ".lock")
959     except OSError as exc:
960         # The lock path isn't writable (read-only dir / perms). Don't crash and don't
falsely claim a
961         # second instance ? warn and proceed WITHOUT the guard (a lock-infra failure
must not block a
962         # legitimate single instance).
963         logger.warning("[JOBS] Could not create the single-instance lock at %s.lock
(%s); proceeding "
964                        "WITHOUT the second-instance guard.", db_path, exc)
965         _instance_lock = None
966     else:
967         if _instance_lock is None:
968             print(f"[FATAL] Another instance is already using this database ({db_path}).
Refusing to "
969                  f"start ? a second instance would corrupt in-flight job state. Stop
the other one, or "
970                  f"use a different --output-dir (RALLY_APP_HOME) to run an independent
instance.")
971             sys.exit(1)
972
973     db_instance = Database(db_path)
974
975     # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
976     # the executor is in-process. Mark them failed so pollers see a terminal state.
977     swept = db_instance.fail_stale_jobs()
978     if swept:
979         logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
980
981     # CLI processing mode (no web UI needed). --app/--server always run the server,
never CLI single-shot.
982     if args.video_path and not args.server and not args.app:
983         print(f"[CLI] Processing video file: {args.video_path}")
984         # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
fetched to scratch.
985         # The local-existence fast-fail only applies to a local input (stat the RESOLVED
key, not the
986         # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
987         storage_cfg = getattr(global_config, "storage", None)
988         try:
989             input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
990         except ValueError as e:

```

```

991         print(f"[FAIL] {e}")
992         return
993     if input_ref.backend == "local" and not os.path.exists(input_ref.key):
994         print(f"[FAIL] Video file not found: {args.video_path}")
995         return
996     local_video = storage.acquire_local_input(args.video_path, storage_cfg)
997
998     video_id = compute_video_id(local_video)
999     was_reingest = db_instance.get_video(video_id) is not None
1000
1001     # Validate (with-context variant surfaces ffprobe_meta for the report)
1002     print("[CLI] Validating...")
1003     val_results, val_context = registry.run_all_with_context(local_video,
global_config)
1004     failures = [r for r in val_results if not r.passed]
1005
1006     # Save video status
1007     status = "failed" if failures else "processed"
1008     db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
in val_results])
1009
1010     seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
1011     segmenter_actual = None
1012     segmentation_ran = False
1013     try:
1014         print("\n" + "="*40)
1015         print("VALIDATION SUMMARY")
1016         print("="*40)
1017         for r in val_results:
1018             status_symbol = "[PASS]" if r.passed else "[FAIL]"
1019             print(f"{status_symbol} {r.validator_name}: {r.message}")
1020         print("="*40 + "\n")
1021
1022         if failures:
1023             print("[FAIL] Video failed checks. Indexing halted.")
1024             return
1025
1026         # Index
1027         segmenter_cls = segmenter_registry.get(seg_name)
1028         if not segmenter_cls:
1029             print(f"[FAIL] Segmenter '{seg_name}' not found.")
1030             return
1031
1032         print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
1033         segmenter_actual = seg_name
1034         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
1035         segmenter = segmenter_cls(db_instance, global_config)
1036         result = segmenter.process_video(video_id, local_video,
max_frames=args.max_frames)
1037         segmentation_ran = True
1038
1039         if isinstance(result, Failure):
1040             print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
1041             print("[FAIL] Indexing halted for video.")
1042             segments = []
1043         else:
1044             segments = result.value if hasattr(result, "value") else result
1045             print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
1046             # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
1047             _finalize_reingest(video_id, segments, play_wm, cand_wm)
1048         finally:
1049             # Always emit the per-video observability report. Never raises.

```



```

1050         paths = emit_indexer_report(
1051             db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
1052             val_results=val_results, val_context=val_context,
1053             segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
1054             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
1055         )
1056         if paths:
1057             print(f"[REPORT] Technical report : {paths.get('technical_md')}")
1058             print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
1059             print(f"[REPORT] Machine JSON      : {paths.get('json')}")
1060         return
1061
1062     # Start Local Server Mode
1063     if args.app or args.server or not args.video_path:
1064         if args.app:
1065             # P2: the friendly launcher ? always localhost-only (use --server for
network exposure),
1066             # auto-pick a free port if --port is busy, and open the bundled UI once the
server is up.
1067             bind_host = "127.0.0.1"
1068             port = _resolve_app_port(bind_host, args.port)
1069             if port != args.port:
1070                 print(f"[APP] Port {args.port} is busy ? using {port} instead.")
1071             url = f"http://{bind_host}:{port}/"
1072             # The browser-open health poll sends Host: 127.0.0.1 ? it relies on loopback
staying in
1073             # the Host allowlist (the default; don't set RALLY_ALLOWED_HOSTS to a non-
loopback-only set
1074             # with --app, or the poll 400s and the browser just won't auto-open ? the
server still runs).
1075             print(f"[APP] Starting KhelSutra ? opening {url} in your browser "
1076                 f"(set RALLY_NO_BROWSER=1 to skip). Press Ctrl+C to stop.")
1077             threading.Thread(target=_open_browser_when_ready, args=(url, url +
"api/health"),
1078                             daemon=True).start()
1079         else:
1080             bind_host = _resolve_bind_host(global_config, args.host) # P0c:
config/profile-driven (default 127.0.0.1)
1081             port = args.port
1082             print(f"[SERVER] Starting local FastAPI server on
http://{bind_host}:{port}...")
1083             if bind_host not in ("127.0.0.1", "localhost", "::1"):
1084                 print(f"[SERVER] ? Binding {bind_host} exposes the API on the network ?
ensure an auth "
1085                     f"gate (edge_auth_enabled / a reverse proxy) is in front, and set
"
1086                     f"RALLY_ALLOWED_HOSTS to your public host(s) (the Host-header
guard defaults to "
1087                     f"loopback-only and will 400 forwarded requests otherwise).")
1088             # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
1089             # in-process wake-timer for any parked 'waiting' job died with the old process,
so
1090             # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
1091             _get_executor().submit(_drain_due_jobs)
1092             uvicorn.run(app, host=bind_host, port=port)
1093
1094     if __name__ == "__main__":
1095         # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads`
resolve the
1096         # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ?
a SEPARATE
1097         # module object. If `main()` runs here it configures `__main__`'s globals, leaving
1098         # `backend.main.db_instance` = None; the async-job drain then crashes silently on

```

```

1099         # `None.claim_due_job()` and every submitted job hangs in 'queued' forever. Dispatch
through
1100         # `backend.main` so `main()` configures the module everyone actually reads. (No
recursion:
1101         # imported as `backend.main`, `__name__` != "__main__", so this block doesn't re-
fire.)
1102         import backend.main as _bm
1103
1104         _bm.main()
1105

```

10. user (2026-06-21T22:32:13.000Z)

```

1      """Pydantic models for the application configuration.
2
3      This module defines the canonical, validated, typed configuration for the
4      Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6      The models support graceful migration from old config.json files via extra="allow".
7      """
8
9      from __future__ import annotations
10
11      from typing import Any, List, Literal, Optional
12
13      from pydantic import BaseModel, Field, field_validator, model_validator
14
15      Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18      class ValidationRelevanceConfig(BaseModel):
19          court_green_lower: list[int] = Field(default=[35, 40, 40])
20          court_green_upper: list[int] = Field(default=[85, 255, 255])
21          court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24      class ValidationConfig(BaseModel):
25          min_resolution_width: int = 1280
26          min_resolution_height: int = 720
27          min_fps: float = 29.9
28          min_blur_threshold: float = 20.0
29          max_scene_cuts_per_minute: float = 0.5
30          relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31
32
33      class TrackNetConfig(BaseModel):
34          wsl_distro: str = "Ubuntu"
35          repo_dir: str = "~/models/TrackNetV4"
36          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37          conda_env: str = "TrackNetV4"
38          weights_path: str = ""
39          queue_length: int = 5
40          stage_in_wsl: bool = True
41          wsl_stage_dir: str = "~/clips"
42          timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43          velocity_threshold: float = 0.02
44          # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45          # decoded frame cache) are deleted automatically ? they are pure intermediates,
46          # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47          # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48          keep_frames: bool = False
49          # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50          # 0 = disabled.
51          min_free_gb: float = 0.0

```

```

52
53
54 class WasbIndexConfig(BaseModel):
55     """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57     Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58     these knobs actually take effect ? previously this whole block was silently dropped
59     because nested config sections defaulted to extra='ignore'.
60     """
61     wsl_distro: str = "Ubuntu"
62     repo_dir: str = "~/models/WASB-SBDT"
63     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64     conda_env: str = "wasb"
65     weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66     sport: str = "badminton"
67     wsl_stage_dir: str = "~/clips"
68     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69     velocity_threshold: float = 0.02
70     # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
71     # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
72     # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
73     # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
74     device: str = "cuda" # auto | cuda | mps | cpu
75     python_bin: str = "python" # the `wasb` conda env python (invoked by path, no
`conda activate`)
76     # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
77     # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
78     # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
79     keep_frames: bool = False
80     # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
81     min_free_gb: float = 0.0
82     # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
83     # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
84     # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
85     # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
86     # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
87     # Tri-state:
88     # None = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
89     # behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
90     # True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
91     stream_video: Optional[bool] = None
92
93
94 class FusionConfig(BaseModel):
95     """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
96
97     Every default reproduces control behaviour: the fusion segmenter persists the
98     shuttle net-crossing count (Gate A signal) and ? only when `cue_flags['person']`
is
99     true ? the person-cue features, but it does NOT gate the served handoff unless a

```

```

100     threshold is raised (`min_crossings`/`min_players` default 0 = OFF). The court
mask
101     is optional: `court_polygon=None` ? Gate B counts every detection (player-count-
only);
102     supplied ? off-court persons (spectators / adjacent courts) are excluded.
103     ""
104     # Which trajectory substrate seeds the windows (shuttle stays primary).
105     substrate: Literal["wasb", "tracknet"] = "wasb"
106     # Windowing velocity threshold for the fusion family (the engine reads
107     # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
108     # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
109     velocity_threshold: float = 0.02
110     # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
111     # enabled ? so the default path never imports torch / touches the GPU.
112     cue_flags: dict[str, bool] = Field(default_factory=dict)
113     # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
114     # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
115     min_crossings: int = Field(default=0, ge=0)
116     # Served Gate B: when the person cue is on, drop windows with median in-court
players
117     # < this. 0 = OFF.
118     min_players: int = Field(default=0, ge=0)
119     # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
120     court_polygon: Optional[list[list[float]]] = None
121     # Net line for the person two-sided-motion split (person features only ? the shuttle
122     # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
123     net_axis: Literal["x", "y"] = "y"
124     net_position: float = Field(default=0.5, ge=0.0, le=1.0)
125
126
127     class IndexingConfig(BaseModel):
128         default_segmenter: str = "yolo_hybrid"
129         use_gpu: bool = True
130         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
131         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
132         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
133         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
134         # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
135         detector_impl: str = "auto"
136         motion_threshold: float = 0.08
137         min_segment_duration: float = 3.0
138         dead_time_merge_gap: float = 2.0
139         chunk_duration: float = 90.0
140         chunk_overlap: float = 10.0
141         detector_model: str = "fasterrcnn_resnet50_fpn"
142         detector_score_threshold: float = 0.5
143         yolo_velocity_threshold: float = 0.15
144         yolo_frame_skip: int = 3
145         yolo_tracker: str = "bytetrack.yaml"
146         yolo_prefetch_workers: int = 2
147         min_action_window_duration: float = 2.0
148         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
=a window
149         # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
150         # `window_min_split_gap` s ? the most likely rally boundary), so one window can't
swallow
151         # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
152         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
153         # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,

```

```

p=0.0005; cap=6
154      # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
155      max_window_duration: float = 6.0
156      window_min_split_gap: float = 0.5 # min internal gap (s) that qualifies as a split
point
157      # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
158      # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
159      # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
160      # Only cuts (recall can't drop). OFF by default until measured/promoted.
161      window_hard_cap: bool = False
162      # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
163      # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
164      # [|?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
165      # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
166      # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
167      # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, [|?end| 4.96?3.31s (n=13).
The W1 cap
168      # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
169      window_inactivity_end: float = 1.5
170      inactivity_rally_thresh: float = 0.20
171      inactivity_min_density: float = 0.30
172      # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
173      # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
174      # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
175      # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
176      # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
177      # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
178      # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
179      # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
180      window_blob_fallback: bool = False
181      # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
182      # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
183      # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
184      # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
185      window_blob_factor: float = 4.0
186      log_yolo_candidates: bool = True
187      store_yolo_candidates: bool = True
188      ai_handoff_padding: float = 2.0
189      ai_max_workers: int = 5
190      # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
191      # chunks of a high-bitrate (4K) source blow past the provider upload cap
192      # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
193      # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
194      # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.

```

```

195     # Matches gemini_refine's --clip-scale-width default.
196     ai_chunk_scale_width: int = 1280
197     # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
198     # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
199     # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
200     # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
201     # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
202     skip_ai_handoff: bool = False
203     # Optional debug knob: trim every video to the first N seconds before processing.
204     debug_duration: Optional[float] = None
205
206     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
207     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
208     fusion: FusionConfig = Field(default_factory=FusionConfig)
209
210     @field_validator("default_segmenter")
211     @classmethod
212     def segmenter_must_be_registered(cls, v: str) -> str:
213         # Registry check happens at runtime in main/segmenter selection.
214         # Here we just allow any string for forward compatibility.
215         return v
216
217     @model_validator(mode="after")
218     def _chunk_step_must_be_positive(self) -> "IndexingConfig":
219         # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
220         # step makes `while t < duration: t += step` loop forever, growing memory before
any
221         # GPU work. Reject the bad config at load time instead.
222         if self.chunk_overlap >= self.chunk_duration:
223             raise ValueError(
224                 f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
225                 f"({self.chunk_duration}); otherwise chunking never advances."
226             )
227         return self
228
229
230     class OutputConfig(BaseModel):
231         default_highlights_name: str = "highlights.mp4"
232         db_name: str = "sports_indexer.db"
233         # Directory where the DB, highlight reels, and processed clips are written.
234         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
235         output_dir: str = "output"
236
237
238     class WatermarkConfig(BaseModel):
239         """Branding overlay burned into each highlight clip during the per-clip re-encode
240         (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
241         (under `stitching.watermark` in config.json) to turn it off. The text is fully
242         configurable. The concat stage stays stream-copy (-c copy)."""
243         enabled: bool = True
244         text: str = "Generated by Khelsutra.guru"
245         logo_path: str = "" # optional transparent PNG overlay; "" = no logo
246         font_path: str = "" # optional .ttf; "" = use the fontconfig `font` family
below
247         font: str = "Sans" # fontconfig family used when font_path is empty
248         font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5) # text height as a
fraction of frame height
249         opacity: float = Field(default=0.85, ge=0.0, le=1.0)
250         box: bool = True # faint backing box behind the text for legibility

```

```

251     margin: int = Field(default=24, ge=0) # px inset from the chosen corner
252     position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"
253
254
255     class StitchingConfig(BaseModel):
256         begin_padding: float = 0.5
257         end_padding: float = 0.5
258         use_cache: bool = False
259         min_shot_count: int = 1
260         # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
261         # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
262         # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
263         # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
264         # local detector over-fires and its false positives are mostly SHORT (motion blips,
265         # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
266         # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
267         min_rally_duration: float = Field(default=0.0, ge=0.0)
268         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
269
270
271     class LoggingConfig(BaseModel):
272         """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
273         # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
274         # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
275         # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
276         # them to WARNING; set true to restore full chatter for request-flow debugging.
277         verbose_http: bool = False
278
279
280     class SecurityConfig(BaseModel):
281         # When set, /api/process video_path must resolve under this directory (hosted/tenant
282         # mode). Default None preserves the single-user local 'any local file' workflow.
283         allowed_video_root: Optional[str] = None
284
285         # --- Chunked upload (B2, PR-2) ---
286         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
287         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
288         # contain upload_dir or /api/process will reject the uploaded paths.
289         upload_dir: str = "output/uploads"
290         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
291         # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).
292         max_upload_mb: int = 4096
293         max_part_mb: int = 95
294         allowed_upload_extensions: List[str] = ["mp4", "mov"]
295
296         # --- M9 edge auth (Cloudflare Access, D9) ---
297         # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local
single-user
298         # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-
Jwt-Assertion`
299         # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
identity
300         # header) and tags the job with the verified user_id (owner_id). The team domain
(e.g.
301         # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is
the CF Access
302         # application's audience tag. NO secrets here ? these are public identifiers.
303         edge_auth_enabled: bool = False
304         cf_team_domain: str = "" # e.g. https://<team>.cloudflareaccess.com (issuer +

```

```

/certs JWKS)
305         cf_access_aud: str = ""          # the CF Access application AUD tag the token must
carry
306         # (CORS allow-origins is set via the RALLY_CORS_ALLOW_ORIGINS env var ? read at
import in
307         # backend/main.py so importing the app does no filesystem I/O; default ["*"].
Credentials stay
308         # OFF ? Cf-Access is a header, not a cookie.)
309
310
311     class StorageConfig(BaseModel):
312         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
313         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
314         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
315         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
316         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
317         (boto3/google auth chains supply credentials). See ``backend/storage/``. """
318         backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
319         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
320         output_root: str = ""
321         # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
322         # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
323         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
324         # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
325         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
326         # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
327         # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
328         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
329         input_backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
330         input_root: str = ""
331         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
332         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>]/[<video_id>/...
333         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
334         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
335         # the legacy flat layout is used unchanged.
336         single_folder_per_video: bool = False
337         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
338         workspace_root: str = ""
339         # Cloud namespace prefix under the root so per-video folders sit tidily beside
340         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
341         workspace_prefix: str = "workspaces"
342         local: dict[str, Any] = Field(default_factory=dict)
343         s3: dict[str, Any] = Field(default_factory=dict)
344         gcs: dict[str, Any] = Field(default_factory=dict)
345         gdrive: dict[str, Any] = Field(default_factory=dict)
346
347
348     class DeploymentConfig(BaseModel):
349         """Deployment profile ? one switch that selects a coherent bundle of compute +

```



```

storage
350     knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
351
352     **Pure / additive:** the empty default profile (``````) applies NO preset, so the
pipeline
353     behaves exactly as before this section existed. A profile is opt-in (``config.json``
354     ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
355     *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
356     these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
357     (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
358
359     Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
360
361     # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
362     profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
363     # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
364     # (recorded for observability; consumed by the ComputeBackend seam from M2+).
365     compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
366
367
368     class BillingConfig(BaseModel):
369         """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
370
371         **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay
NULL =
372         today's behaviour). When enabled, a job's cost is computed + stored in **USD** via
the versioned
373         ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
pricebook
374         version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the
owner
375         inserts a real, calibrated version and points ``pricebook_version`` at it.
376
377         ? Even with ``enabled=True`` + a real pricebook, recorded cost stays **0** until a
follow-up
378         wires the worker to emit per-job ``usage`` (gpu/wall/egress/gemini) ? M8 lands the
ledger +
379         pricebook + the recording seam; usage emission (the metering hooks) is a separate
step."""
380
381         enabled: bool = False
382         pricebook_version: str = "placeholder-v0" # the seeded $0 version; owner flips to
a real one
383         fx_inr_per_usd: float = 83.0 # USD?INR display rate (Bangalore
market)
384         margin: float = 0.0 # resale markup (?0); 0 = bill at
cost
385
386
387     class AppConfig(BaseModel):
388         """Root configuration object.
389
390         All runtime configuration should be accessed via an instance of this class
391         (or the loader) rather than raw dict lookups.
392         """
393
394         sport: Sport = "badminton"
395         ai_provider: str = "gemini"
396         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;

```

```

397         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
398         ai_model: str = "gemini-flash-latest"
399
400         validation: ValidationConfig = Field(default_factory=ValidationConfig)
401         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
402         output: OutputConfig = Field(default_factory=OutputConfig)
403         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
404         security: SecurityConfig = Field(default_factory=SecurityConfig)
405         logging: LoggingConfig = Field(default_factory=LoggingConfig)
406         storage: StorageConfig = Field(default_factory=StorageConfig)
407         deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
408         billing: BillingConfig = Field(default_factory=BillingConfig)
409
410         # Allow extra keys during transition so existing config.json files do not break.
411         model_config = {
412             "extra": "allow",
413             "validate_assignment": True,
414         }
415
416         # Temporary migration shim for legacy dict-style access (config.get("section",
417         {}).get(...)).
418         # Will be removed once all call sites (validators, segmenters, etc.) move to typed
419         # attribute access / the typed config. A nested *section* is returned as a plain
dict
420         # via model_dump, so legacy chained access keeps working ? returning the Pydantic
421         # section model instead breaks it (a model has no .get), which bit every validator
on
422         # real runs. Scalar leaves are returned as-is.
423         def get(self, key: str, default: Any = None) -> Any:
424             _MISSING = object()
425             val: Any = _MISSING
426             if key in type(self).model_fields or hasattr(self, key):
427                 val = getattr(self, key, _MISSING)
428             if val is _MISSING:
429                 # Legacy convenience: find a leaf key inside any nested section. Sections
are
430                 # discovered dynamically from the model fields, so a future top-level
section
431                 # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
432                 for fname in type(self).model_fields:
433                     section = getattr(self, fname, None)
434                     if isinstance(section, BaseModel) and hasattr(section, key):
435                         val = getattr(section, key)
436                         break
437                 if val is _MISSING:
438                     return default
439                 if isinstance(val, BaseModel):
440                     return val.model_dump(mode="json")
441                 return val
442         def __getitem__(self, key: str) -> Any:
443             val = self.get(key)
444             if val is None and key not in type(self).model_fields:
445                 # Re-raise like a real dict for unknown top-level
446                 raise KeyError(key)
447             return val
448
449         # Convenience type alias for callers that want to annotate
450         Config = AppConfig
451
452

```

Now let me read the README and search for install-related docs:

12. user (2026-06-21T22:32:16.081Z)

```
./backend/assets/fonts/README.md
./deploy/cloudrun/README.md
./deploy/digitalocean/README.md
./deploy/gcp/README.md
./docs/archives/checkpoints/2026-06-eval-baseline/README.md
./docs/archives/checkpoints/2026-06-logging-reporting/README.md
./docs/archives/checkpoints/2026-06-scaffolding-complete/README.md
./docs/archives/checkpoints/2026-06-strategy-foundation/README.md
./docs/archives/field-pmf/README.md
./docs/archives/past_projects/DECOUPLED_COMPUTE/README.md
```

13. user (2026-06-21T22:32:19.021Z)

```
1      # Badminton Highlight & Rally Indexer
2
3      A local, AI-powered web application and CLI utility designed to process long-form
badminton recordings (e.g., from GoPro), run comprehensive pluggable validations (quality,
integrity, and relevance checks), index all rallies into a relational SQLite database, and
dynamically compile custom highlight reels using high-speed FFmpeg stitching.
4
5      The pipeline is built around three pluggable registries ? validators,
segmenters, and AI providers ? plus a sport profile registry so the same engine can
target other racket/court sports in the future. Badminton is the default profile.
6
7      > Where this is heading: the project is evolving from an AI-API-dependent prototype
into a self-improving tool that learns rally detection offline from a commercially-licensed,
annotated corpus. See [docs/ROADMAP.md](docs/ROADMAP.md) for the four-phase plan and
[docs/archives/decisions/DECISIONS.md](docs/archives/decisions/DECISIONS.md) (historical
ADRs) for the rationale.
8
9      ---
10
11     ## ? Key Features
12
13     * Pluggable Video Validation Framework: Easily extendable base-class architecture
that automatically runs checks on:
14         * Static Format & Resolution: Verifies container is MP4, resolution >= 720p, and
framerate >= 30 FPS.
15         * PTS Timeline Continuity: Scans keyframe timestamps for timeline gaps, jumps,
or editing splices to detect tampering or splices.
16         * Scene Cut Density Check: Measures sub-sampled HSV frame histogram correlation
to verify visual cuts. A continuous raw camera stream should have near 0 cuts.
17         * Blurriness Check: Calculates average frame Laplacian variance to reject out-
of-focus footage.
18         * Relevance (Is it Badminton?): Uses high-performance HSV color court matching
to identify court layouts.
19     * Six Pluggable Rally Segmenters (`backend/pipeline/segmenters/` ? `motion`,
`gemini`, `yolo_hybrid`, `tracknet_hybrid`, `wasb_hybrid`, `fusion_hybrid`). The three core
ones:
20         * `yolo_hybrid` ? two-stage pipeline. Stage 1 runs torchvision person
detection (Faster R-CNN, BSD-licensed) with ByteTrack tracking (supervision, MIT-licensed)
to cheaply filter the video down to a small set of high-motion candidate windows. Stage 2 hands
each candidate (with a small padding) to the AI provider for high-precision semantic boundaries
and shot counts. Massively cheaper than running the LLM on the full video. *(The registry key
remains `yolo_hybrid` for back-compat; the AGPL-licensed YOLOv8 was replaced with permissive
components ? see [docs/archives/MONETIZATION_AUDIT.md](docs/archives/MONETIZATION_AUDIT.md) and
ADR-005.)*
21         * `gemini` ? sends the entire video (or a `chunk_duration` slice) to Google Gemini
directly. Highest precision per call, highest cost.
22         * `motion` ? zero-dependency OpenCV pixel-difference heuristic. No API key
required. Useful as a baseline or for offline/air-gapped use.
```

```

23      *   **Pluggable AI Provider Registry** (`backend/providers/`): The Gemini provider ships
in-tree; new providers (e.g. an OpenAI or local-model backend) can be registered without
touching the segmenters.
24      *   **Pluggable Sport Profile Registry** (`backend/sports/`): The per-sport LLM prompt
and relevance config live in a `SportProfile`. Add a new sport by subclassing `SportProfile` and
registering it.
25      *   **SQLite Indexing**: Each processed video is keyed by MD5 hash. Play segments
(start/end/duration/server/receiver/metadata) and per-segment events are persisted in
`output/sports_indexer.db`. Re-running on the same file is a cache hit.
26      *   **Query System**: Filter indexed play segments by `server`, `receiver`,
`duration_min`, and `event_type` (the schema supports rich event types like
smashes/serves/fouls; today the shipped segmenters mainly populate timing + estimated
`shot_count`).
27      *   **Zero-Dependency Local UI**: Served statically directly by FastAPI (Vite + Node.js
NOT required!) featuring a glassmorphic dashboard, responsive filtering, and immediate clip
stitching.
28      *   **High-Speed FFmpeg Stitching** with configurable `begin_padding`, `end_padding`,
and `min_shot_count` filtering, plus an optional **branding watermark** burned into every reel
(**on by default** ? `"Generated by Khelsutra.guru"`, fully configurable, with an off switch).
29      *   **Detailed CLI Debugging Tools**: `--debug-clip <t>` inspects frame metrics,
validator scores, and motion values around a timestamp. `--trim-start/--trim-end` extracts an
arbitrary segment via stream-copy ffmpeg for fast iteration.
30
31      ---
32
33      ## ?? Setup & Diagnostics
34
35      ### 1. External System Requirements
36      The indexer depends on **FFmpeg** and **FFprobe** to parse media containers, extract
keyframes, and stitch clips.
37
38      *   **On Windows (PowerShell as Admin)**:
39          ```powershell
40          winget install Gyan.FFmpeg
41          ```
42          *Restart your terminal afterwards to register the path changes.*
43
44      ### 2. Install the package
45
46      The project is packaged with PEP 621 metadata in `pyproject.toml` (hatchling
47      build backend). Install it editable:
48
49      ```bash
50      # Runtime deps only:
51      pip install -e .
52      # Plus test + lint/type tooling (pytest, pytest-cov, httpx, ruff, mypy):
53      pip install -e .[dev]
54      ```
55
56      This registers the `indexer` console entry point (so `indexer --server`,
57      `indexer --help`, etc. work) and puts the `backend` and `training` packages on
58      your path.
59
60      > **PyTorch is installed separately.** `torch`/`torchvision` are intentionally
61      > **not** in `pyproject.toml`: their CPU/CUDA wheels need an out-of-band wheel index
62      > that PEP 621 cannot express. Install them yourself, matching your hardware:
63      > ```bash
64      > # NVIDIA GPU (CUDA 12.4):
65      > pip install "torch>=2.12.0" "torchvision>=0.27.0" --index-url
https://download.pytorch.org/whl/cu124
66      > # CPU only:
67      > pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
68      > ```
69
70      ### 3. Diagnostics Checker (`scripts/doctor.py`)

```

```

71     Run the diagnostics checker to verify your system dependencies, initialize a
72     default config, and (optionally) bootstrap the Python packages:
73     ```bash
74     python scripts/doctor.py
75     # or, via the CLI:
76     indexer --setup
77     ```
78     This script will:
79     *   Validate `ffmpeg` and `ffprobe` are present in your PATH.
80     *   Initialize a default `config.json` (if one does not exist) with thresholds for blur,
81     court HSV bounds, motion signals, YOLO velocity, chunking, and stitching.
82     *   Detect whether an NVIDIA GPU is available (via `nvidia-smi`) and select the matching
83     PyTorch wheel index automatically (`cu124` if GPU present, `cpu` otherwise).
84     *   Install everything from `requirements.txt`: `fastapi`, `uvicorn`, `opencv-python`,
85     `numpy`, `pydantic`, `jinja2`, `python-dotenv`, `google-genai`, `torch`, `torchvision`,
86     `supervision`.
87
88     > The default `yolo_hybrid` segmenter uses torchvision's COCO-pretrained Faster
89     R-CNN, whose weights are downloaded and cached automatically by torchvision on first run ?
90     nothing is bundled in the repo. (Pick a lighter/heavier detector via `indexing.detector_model`:
91     `fasterrcnn_resnet50_fpn` (default), `fasterrcnn_mobilenet_v3_large_fpn`, or
92     `retinanet_resnet50_fpn`.)
93
94     ---
95
96     ## ? Running the Application
97
98     ### 1. Launching the Local Web App Dashboard
99     To start the FastAPI server and open the web dashboard:
100     ```bash
101     python -m backend.main --server --port 8000
102     ```
103     Open your browser and navigate to [http://localhost:8000](http://localhost:8000).
104     *Simply input your local raw video file path in the sidebar to process, validate, index,
105     and compile highlights instantly.*
106
107     ### 2. Running in CLI Processing Mode
108     To process and index a video completely via terminal commands:
109     ```bash
110     python -m backend.main --video-path "C:/path/to/my_video.mp4"
111     ```
112     This outputs a validation summary and indexes all rallies directly into the local SQLite
113     database, using whichever segmenter is set as `default_segmenter` in `config.json`. The
114     shipped `config.json` sets `gemini` (needs `GEMINI_API_KEY`; without it the run silently
115     produces 0 segments and the observability report flags `silent_provider_noop`). Set it to
116     `yolo_hybrid`/`wasb_hybrid`/`motion` for the local paths.
117
118     To explicitly pick a segmenter for one run:
119     ```bash
120     python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter yolo_hybrid
121     python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter gemini
122     python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter motion
123     ```
124
125     ### 3. Rich CLI Debugging (`--debug-clip`)
126     To diagnose exactly why a video segment was registered as active/dead play, or inspect
127     detailed focus/continuity metrics at a particular time in seconds:
128     ```bash
129     python -m backend.main --video-path "C:/path/to/my_video.mp4" --debug-clip 125.5
130     ```
131     *Outputs granular validation check logs and motion ratios within a +/- 3 second frame
132     window around the timestamp.*
133
134     ### 4. Extracting a Trimmed Segment (`--trim-start` / `--trim-end`) ? also the quickest
135     watermark preview

```

```

120     Extract a slice of a recording to disk (re-encoded; and, with the default-on branding
watermark, stamped ***"Generated by Khelsutra.guru"*** in the corner):
121     ```bash
122     python -m backend.main --video-path "C:/path/to/my_video.mp4" \
123         --trim-start 600 --trim-end 660 --trim-output slice_10m_to_11m.mp4
124     ```
125     Output lands in `--output-dir` (default `output/`) unless `--trim-output` is an absolute
path. This is the fastest way to *see* the watermark on a real clip ? set
`stitching.watermark.enabled: false` in `config.json` to turn it off.
126
127     > To preview the exact overlay on any file without the pipeline:
128     > ```bash
129     > ffmpeg -i in.mp4 -vf "drawtext=text='Generated by Khelsutra.guru':font='DejaVu Sans':f
ontcolor=white@0.85:fontsize=h/28:x=w-tw-24:y=h-th-24:box=1:boxcolor=black@0.4:boxborderw=10"
-c:a copy out.mp4
130     > ```
131
132     ### 5. End-to-End "Process + Stitch" Script
133     `scripts/run_process_and_stitch.py` is an interactive convenience runner: it MD5-hashes
a hardcoded video path, reuses cached indexed segments if available (offering to re-index from
scratch), and then prompts for stitching parameters (`begin_padding`, `end_padding`,
`min_shot_count`) before compiling the final highlight reel. Useful when iterating on stitching
settings against a fixed large recording without re-running expensive segmentation.
134
135     ### 6. Cache Hygiene & Disk Cleanup
136     The trajectory detectors (`wasb_hybrid` / `tracknet_hybrid`) stage a copy of each video
into WSL and decode every frame to PNG ? easily **tens of GBs per 4K video**. To keep disk usage
bounded:
137
138     * **Automatic self-clean:** after a *successful* run the frame cache + staged copy are
deleted (`indexing.{wasb,tracknet}.keep_frames`, default `false`). On failure they're kept so a
re-run resumes from the checkpointed detector cache.
139     * **Manual sweep (dry-run-first):** `tools/cleanup_caches.py` reports and (with
`--apply`) reclaims what failed/old runs left behind, across both the WSL stage dir and
`output/`:
140     ```bash
141     python -m tools.cleanup_caches                # report what's reclaimable (safe;
deletes nothing)
142     python -m tools.cleanup_caches --apply        # delete the regenerable caches
143     python -m tools.cleanup_caches --keep-video GX010125 # protect one video's cache
(id or filename stem)
144     python -m tools.cleanup_caches --older-than 7 # only purge entries older than 7
days
145     ```
146     It treats frame caches, staged/proxy videos, and handoff/temp dirs as
**regenerable** (purgeable) and always keeps trajectory CSVs, the SQLite DB, and final reels.
Add `--include-wasbcache` to also drop detector resume caches. `--summary` prints a one-line
nudge (used by the optional SessionStart hook).
147
148     ---
149
150     ## ?? Threshold Tuning (`config.json`)
151     You can adjust the parameters of your validators, segmenters, AI provider, and stitcher
by editing `config.json` in the root folder. The default config emitted by `scripts/doctor.py`:
152
153     ```json
154     {
155         "sport": "badminton",
156         "ai_provider": "gemini",
157         "ai_model": "gemini-flash-latest",
158         "validation": {
159             "min_resolution_width": 1280,
160             "min_resolution_height": 720,
161             "min_fps": 29.9,
162             "min_blur_threshold": 20.0,

```

```

163         "max_scene_cuts_per_minute": 0.5,
164         "relevance": {
165             "court_green_lower": [35, 40, 40],
166             "court_green_upper": [85, 255, 255],
167             "court_pixels_min_ratio": 0.05
168         }
169     },
170     "indexing": {
171         "default_segmenter": "gemini",
172         "use_gpu": true,
173         "motion_threshold": 0.08,
174         "min_segment_duration": 3.0,
175         "dead_time_merge_gap": 2.0,
176         "chunk_duration": 90,
177         "chunk_overlap": 10,
178         "yolo_velocity_threshold": 0.15,
179         "yolo_frame_skip": 3,
180         "yolo_tracker": "bytetrack.yaml",
181         "yolo_prefetch_workers": 2,
182         "min_action_window_duration": 2.0,
183         "ai_handoff_padding": 2.0,
184         "ai_max_workers": 5
185     },
186     "output": {
187         "default_highlights_name": "highlights.mp4",
188         "db_name": "sports_indexer.db"
189     },
190     "stitching": {
191         "begin_padding": 0.5,
192         "end_padding": 0.5,
193         "use_cache": false,
194         "min_shot_count": 1,
195         "watermark": {
196             "enabled": true,
197             "text": "Generated by Khelsutra.guru",
198             "position": "bottom-right",
199             "opacity": 0.85,
200             "logo_path": "",
201             "font_path": ""
202         }
203     }
204 }
205 ```

```

207 Notable knobs:

```

208 * `validation.min_blur_threshold` ? defaults to `20.0` (deliberately permissive for
noisy indoor-arena GoPro footage; see `docs/archives/TEST_RUN_SUMMARY.md` for why).
209 * `indexing.default_segmenter` ? `yolo_hybrid` | `gemini` | `motion`.
210 * `indexing.use_gpu` ? when true and CUDA is available, YOLO runs on GPU.
211 * `indexing.yolo_velocity_threshold` ? fraction of frame width per second; lower
values catch slower play, higher values are stricter about what counts as "action".
212 * `indexing.yolo_frame_skip` ? process every Nth frame during YOLO tracking. `3` at 30
fps ? 10 samples/sec.
213 * `indexing.chunk_duration` / `chunk_overlap` ? how to split long videos for the YOLO
phase.
214 * `indexing.ai_handoff_padding` ? seconds of slack added around each YOLO candidate
window before sending to the AI provider.
215 * `indexing.ai_max_workers` ? parallel calls to the AI provider during phase 3.
216 * `stitching.min_shot_count` ? drop play segments whose estimated shot count is below
this threshold when compiling highlights. **Enforced on both the CLI runner and the
`/api/compile` endpoint** (segments that carry no `shot_count` metadata are exempt, so
explicitly-requested segment ids can't silently vanish under the default floor).
217 * `indexing.wasb.keep_frames` / `indexing.tracknet.keep_frames` ? after a
**successful** trajectory-detector run, the decoded frame cache and the staged video copy (the
dominant disk consumer ? ~GBs per 4K video) are deleted automatically. Default `false` (= auto-

```

```

delete); set `true` only for debugging. On failure they're always kept so a re-run resumes.
`indexing.{wasb,tracknet}.min_free_gb` warns before a run if WSL free space is below the
threshold (0 = off). See $ Cache hygiene below and `python -m tools.cleanup_caches`.
218 * `stitching.watermark` ? branding overlay burned into every highlight clip (on by
default). `enabled: false` turns it off; `text` is the fully-configurable string; `position` ?
{`bottom-right`, `bottom-left`, `top-right`, `top-left`}; `opacity` 0?1; optional `logo_path`
(transparent PNG). It ships with a bundled Apache-2.0 font (Roboto Slab) so the text renders
out-of-the-box without a system fontconfig setup (which Windows ffmpeg often lacks); set
font_path to a different .ttf` (your own, or another in `backend/assets/fonts/`) to
change the typeface. If the watermark encode still fails, the clip is re-encoded without it (so
the reel is never lost) and the ffmpeg reason is logged.
219
220 ---
221
222 ## ? Frequently Asked Questions (FAQ)
223
224 ### Q1: The server complains that `ffprobe` is not found.
225 Ensure FFmpeg is installed and added to your system environment variables. You can check
this by typing `ffmpeg -version` or `ffprobe -version` in your terminal. If running
`scripts/doctor.py` outputs a `[FAIL]`, re-install FFmpeg or manually copy the binary files to a
PATH folder.
226
227 ### Q2: Why does my video fail the Relevance check?
228 Our default relevance check scans for the typical indoor green badminton court floor
color space (HSV). If your court is blue, or uses standard wood grain flooring, simply adjust
the HSV values inside your `config.json` under `validation.relevance.court_green_lower` and
`court_green_upper` to match your court, or lower the `court_pixels_min_ratio` to `0.0` to
bypass.
229
230 ### Q3: How do I add a new custom validator to the pipeline?
231 1. Create a new python file in `backend/pipeline/validators/` (e.g. `my_check.py`).
232 2. Subclass `VideoValidator` and implement `validate(self, video_path, config,
context)`.
233 3. Register your class inside `backend/pipeline/validators/__init__.py` using
`registry.register(MyCheckValidator())`.
234 It's that simple! The app will automatically run it, display it in the CLI diagnostics
summary, and render it in the frontend checklist.
235
236 ### Q4: Why does the CLI or server output freeze or take a long time on large videos?
237 * Print Buffering: Standard Python buffers console output when stdout is
redirected or running in background shells on Windows. To get real-time unbuffered log
statements immediately, run commands with Python's unbuffered flag:
238 ```bash
239 python -u -m backend.main --video-path "C:/path/to/video.mp4"
240 ```
241 * Video Seeking Overhead: Seeking to random frames in high-bitrate long H.265/HEVC
videos using OpenCV's `cap.set(cv2.CAP_PROP_POS_FRAMES)` incurs massive I/O decoding delays. The
`motion` segmenter reads sequentially and skips frame decodes using container-level
`cap.grab()`, which delivers a 100x speedup on multi-gigabyte raw files. `yolo_hybrid`
sidesteps the problem entirely by extracting fixed chunks via ffmpeg first.
242
243 ### Q5: Why did the video fail with "No keyframes / packet timestamps could be
extracted"?
244 Some H.265/HEVC streams (e.g. GoPro footage) encode frames as GDR or CRA keyframes.
Standard FFmpeg frame-level skipping (`-skip_frame nokey`) fails to identify them, returning
empty logs. We resolved this by extracting packet indexes (`packet=pts_time,flags`) at the
container demuxer level instead, which is incredibly robust and executes instantly without
decoding a single pixel.
245
246 ### Q6: How can I quickly test the indexing without waiting to parse an entire multi-GB
video file?
247 You can pass the `--max-frames <N>` argument in CLI or `max_frames` in the POST API
payload. This restricts the segmenter frame loop to process only the first $N$ sub-sampled
frames, enabling you to test motion filters and court relevance checks in seconds:
248 ```bash

```



```

249     python -m backend.main --video-path "C:\Users\avidu\Videos\LargeTestVideo.MP4" --max-
frames 300
250     ```
251     *(At 5 sub-sampled frames per second, `--max-frames 300` analyzes exactly the first 60
seconds of video).*
252
253     ### Q7: How do I configure and run an AI-powered segmenter (Gemini / YOLO-hybrid)?
254     Both `yolo_hybrid` (default) and `gemini` need an API key for the AI provider. To use
Google Gemini:
255     1. Create a local `.env` file in the project root (this is already in `.gitignore` to
prevent secret leaks).
256     2. Set your Google AI Studio API key in the file:
257         ```env
258         GEMINI_API_KEY=your_key_here
259         ```
260     3. Run with whichever segmenter you want:
261         ```bash
262         # Default: cheap candidate filtering with YOLO, then precise boundaries via Gemini
263         python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter yolo_hybrid
264
265         # Full-video Gemini analysis (more expensive, highest semantic precision per call)
266         python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter gemini
267
268         # Pure CV baseline ? no API key needed
269         python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter motion
270         ```
271         *Or set `"default_segmenter": "yolo_hybrid"` (or `"gemini"/"motion"`) under
`"indexing"` in `config.json`.*
272
273     The AI provider used by `yolo_hybrid` and `gemini` is controlled by `ai_provider` +
`ai_model` at the top level of `config.json`. Today only `"gemini"` is registered; new providers
can be added under `backend/providers/`.
274
275     **Trajectory-based segmenters (`wasb_hybrid`, `tracknet_hybrid`).** The registry also
includes two shuttle-trajectory variants. Like `yolo_hybrid`, they cheaply filter the video down
to candidate windows *locally* and then route each window to the AI provider (`gemini`) for
precise boundaries ? so they need the same `GEMINI_API_KEY`. **If an observability report shows
`segmenter_actual: wasb_hybrid` (or `tracknet_hybrid`), that is a valid, registered segmenter ?
not an error.** Pick one with `--segmenter wasb_hybrid` or via `default_segmenter`.
(`wasb_hybrid` additionally requires the WSL WASB setup ? see
[docs/TRACKNET_WSL_SETUP.md](docs/TRACKNET_WSL_SETUP.md).)
276
277     ### Q8: How does the `yolo_hybrid` pipeline actually save money?
278     Sending a multi-gigabyte video to a hosted LLM is slow and expensive. `yolo_hybrid` runs
in four phases:
279
280     1. **Chunking** ? splits the video into overlapping windows (`chunk_duration`,
`chunk_overlap`) so we can process incrementally.
281     2. **Detection + tracking** ? torchvision person detection (Faster R-CNN) + ByteTrack
runs on each chunk locally (GPU-accelerated when available), measuring per-track velocity.
Frames whose normalised player velocity exceeds `yolo_velocity_threshold` are kept; the rest are
dropped. Adjacent active frames are merged into candidate windows, with
sub-`min_action_window_duration` windows discarded.
282     3. **AI handoff** ? only the surviving candidate windows (padded by `ai_handoff_padding`
seconds on each side) are extracted and sent to the AI provider for precise rally boundaries and
shot-count estimation. These calls run in parallel up to `ai_max_workers`.
283     4. **DB population** ? confirmed segments are written to `play_segments` along with
`shot_count`, `shot_count_confidence`, and a `detector` tag like `yolo-hybrid-gemini-2.5-flash`.
284
285     In practice this typically cuts AI-provider runtime/cost by an order of magnitude versus
uploading the full video to `gemini` directly, while preserving the LLM's strength at semantic
boundary detection.
286
287     ### Q9: How can I debug/test the Gemini Segmenter on a small portion of a video without
uploading a huge multi-GB file?

```

288 Ingesting and uploading multi-gigabyte video files to Google Cloud can be slow and expensive. We built a native **fast local trimming** developer debugging feature:

289 1. Open your `config.json` file.

290 2. Under the `"indexing"` configuration block, add a `"debug_duration"` key (in seconds):

```

291     ``json
292     "indexing": {
293         "default_segmenter": "gemini",
294         "debug_duration": 15.0,
295         ...
296     }
297     ``

```

298 3. When `"debug_duration"` is set, the Gemini segmenter will instantly run a zero-re-encode, sub-second `ffmpeg` slice to create a tiny temporary clip of the first 15 seconds, upload only that tiny clip to Google AI Studio (which takes <1 second to upload and <5 seconds to process), run the model query, and clean up the temporary files automatically. This allows you to iterate on model prompts, evaluate validation overlaps, and verify rally boundaries in under 10 seconds!

299

300 **Q10: Can I trust the per-segment metadata from the `motion` segmenter?**

301 **No.** The zero-dependency `motion` segmenter detects *timing* via pixel-difference, but its per-segment *metadata* is heuristic/synthetic ? `intensity`, `avg_speed_kmh`, the `serve/smash/foul/winner` *events*, and `server/receiver` are generated values, *not measured*. They exist so the schema is populated for demos; do not treat them as ground truth. For real shot counts and boundaries use `yolo_hybrid` (default) or `gemini`. If your observability report shows a `synthetic_motion_metadata` warning, this is why.

302

303 **Q11: I re-ran a video and my previous results changed/disappeared. Why?**

304 Videos are keyed by the *MD5* of the file's bytes. Re-processing the *same* file is treated as a fresh ingest: `add_video` is `INSERT OR REPLACE`, and the foreign-key cascade *deletes* the previous play/candidate segments before writing the new ones. So the earlier results are intentionally replaced, not merged. (A `reingest_replaced_prior` note in the report flags this.) Note also that re-encoding or re-downloading a video changes its bytes ? a new MD5 ? a separate record.

305

306 **Q12: What is the `report.json` / `report.md` / `summary.md` file next to my video?**

307 Every processing run now writes a *per-video observability report* next to the video (or to `report.output_dir` if set in `config.json`):

308 - `<name>.report.json` ? machine-readable: every stage, metric, and finding. The source of truth; diff two runs to see what changed.

309 - `<name>.report.md` ? the *technical* report: validation results, processing stages, and *findings/warnings*. Each warning names the problem and cites the FAQ entry to read. *Start here* when a run looks wrong.

310 - `<name>.summary.md` ? a *customer-friendly* summary: just the video metadata and highlights, no system internals.

311

312 To read the technical report: scan *Verdict* first (PASS / PASS-with-warnings / FAIL), then *Findings & warnings* ? each finding has a plain-English message, evidence, and a ? see README#Qn pointer. To turn reports off or relocate them, set `"report": {"enabled": false}` or `"report": {"output_dir": "..."} in config.json`.

313

314 **Q13: A validation check failed and indexing was halted ? what do I do?**

315 Open the report's *Processing stages ? validation* block (or the *VALIDATION SUMMARY* printed to the console). Each failed check states the exact reason. Common cases: *relevance* (the court color didn't match ? see [Q2](#q2-why-does-my-video-fail-the-relevance-check)); *resolution/fps* (below the configured minimums ? adjust `validation.min_*` in `config.json`); *format* (not an MP4 container); *blur* (footage too soft ? improve sharpness/lighting or lower `validation.min_blur_threshold`; a reading of `0.0` often means undecodable frames, so re-encode with `ffmpeg -i in.mp4 -c:v libx264 out.mp4` before lowering the threshold); *keyframes/PTS* (HEVC/GoPro keyframe quirks ? see [Q5](#q5-why-did-the-video-fail-with-no-keyframes--packet-timestamps-could-be-extracted)). Fix the underlying cause or relax the relevant threshold in `config.json`, then re-run.

316

317 **Q14: The report says fps was a "fallback" (not probed) ? what does that mean?**

```

318     The `Video` section shows `FPS | 30 (fallback)` and a `fps_fallback_used` warning when
the pipeline could not probe the real frame rate from the file and fell back to a hardcoded
default (30 fps). Because rally detection converts frames/seconds and scales motion thresholds
by fps, a wrong fps miscalibrates timing and motion sensitivity ? segment boundaries and
which motion counts as "play" can be off.
319
320     This usually means validation didn't run (it's the step that probes the file via
`ffprobe`), or `ffprobe` couldn't read the stream's `r_frame_rate`. To fix: confirm the real fps
?
321     ```bash
322     ffprobe -v error -select_streams v:0 -show_entries stream=r_frame_rate -of csv=p=0
"C:/path/to/video.mp4"
323     ```
324     ? make sure `ffprobe` is on your PATH (see [Q1](#q1-the-server-complains-that-ffprobe-
is-not-found)) and that the run validates the file, then re-run so the real fps is used. (When
fps shows `(probed)` instead, this is all fine.)
325
326     ---
327
328     ## ? Annotation & Evaluation
329
330     Once a video is processed, you can measure how well the pipeline detected its
rallies against hand-annotated ground truth. You annotate each rally's `[start, end)` time
range in a small CSV; the harness scores the detected rallies straight from the SQLite DB ? no
API calls, fully deterministic.
331
332     ```bash
333     python run_eval.py --scaffold --annotations rallies.csv          # make an empty
template
334     # ...fill in start,end rows, then:
335     python run_eval.py --video "C:/videos/match.mp4" --annotations rallies.csv --show-
failures
336     ```
337
338     It reports precision/recall/F1, mean IoU, and boundary error for two stages ? raw
detector `candidates` vs. AI-confirmed `final` segments ? which pinpoints whether a weak result
is a detection problem or a segmentation problem.
339
340     > Rally timestamps evaluate and tune segmentation; they do not train the shuttle
detector (that needs per-frame coordinate labels). See
**[docs/EVALUATION.md](docs/EVALUATION.md)** for the annotation format, CLI flags, and how to
read the output.
341
342     ---
343
344     ## ? Running the Test Suite
345     The test suite uses `pytest` and lives in `tests/`. From the project root:
346
347     ```bash
348     pip install pytest pytest-cov --user
349
350     # One-liner full suite (recommended before opening a PR) ? discoverable wrapper that
351     # runs the whole suite, reports a clear PASS/FAIL, and returns pytest's exit code:
352     python run_all_tests.py
353     python run_all_tests.py -k yolo -v          # any args pass straight through to pytest
354
355     # ...or call pytest directly:
356     pytest                                     # run everything
357     pytest tests/test_yolo_hybrid.py          # run a single file
358     pytest -k yolo --maxfail=1 -v             # filter by keyword, fail fast, verbose
359     pytest --cov=backend --cov-report=term-missing # coverage report (uses .coveragerc)
360     ```
361
362     > The optional `obsreport` package powers the per-video observability reports; its tests
363     > skip cleanly if it isn't installed. To exercise them: `pip install -e ../sports-

```

```

obsreport --no-build-isolation`.
364
365     The suite mocks external dependencies (the Gemini client, ffprobe/ffmpeg, OpenCV
`VideoCapture`, the YOLO model) so it runs offline and without a GPU. The tests cover:
366
367     * `tests/test_base.py` ? segmenter registry
368     * `tests/test_database.py` ? SQLite schema, video/segment insert/query
369     * `tests/test_geometry.py` ? bbox center / distance / velocity / IoU helpers
370     * `tests/test_video.py` ? ffprobe duration + ffmpeg chunk extraction helpers
371     * `tests/test_providers.py` + `tests/test_provider_gemini.py` ? AI provider registry
and the Gemini provider
372     * `tests/test_sport_profiles.py` ? sport profile registry + the badminton prompt
373     * `tests/test_gemini.py` ? `GeminiPlaySegmenter` (single-file and chunked paths)
374     * `tests/test_yolo_hybrid.py` ? `HybridYoloSegmenter` happy path, missing-API-key
bailout, and timestamp-validation drops
375     * `tests/test_tracknet_runner.py` + `tests/test_tracknet_hybrid.py` ? TrackNet WSL
runner (mocked subprocess) and the `tracknet_hybrid` segmenter
376     * `tests/test_eval_metrics.py` + `tests/test_eval_annotations.py` +
`tests/test_eval_harness.py` ? rally evaluation harness (interval matching, CSV loading, DB
scoring, CLI)
377
378     ---
379
380     ## ? Project Structure (high level)
381
382     ```
383     backend/
384         main.py                # FastAPI app + CLI orchestrator
385         results.py             # Run-result aggregation
386         pipeline/              # The processing pipeline
387         segmenters/            # Pluggable rally segmenters: motion, gemini, yolo_hybrid,
tracknet_hybrid, wasb_hybrid, fusion_hybrid
388         detectors/             # Shuttle/person detectors: tracknet, wasb, person
389         validators/            # Pluggable video validators (format, blur, scene-cut,
relevance, ...)
390         stitcher/compiler.py   # FFmpeg-based highlight compiler
391         api/static/            # Zero-build frontend served by FastAPI
392         infrastructure/        # SQLite layer (database.py) + execution policy
393         providers/             # Pluggable AI providers (gemini today)
394         sports/                # Pluggable sport profiles (badminton today)
395         eval/                  # Rally evaluation harness: metrics, annotations, scoring
396         annotations/           # Rally annotation providers
397         config/                # Config schema + model registry
398         features/              # Rally sequence features
399         reporting/             # Observability run signals
400         tools/                 # Export + rally slicing utilities
401         utils/                 # ffprobe/ffmpeg + bbox geometry helpers
402     docs/                      # Live docs (CODE_MAP, COMMERCIALIZATION, BACKLOG, ROADMAP,
guidelines, golden-set). Historical items in docs/archives/
403     tests/                     # pytest suite (mocked, runs offline)
404     scratch/                   # Throwaway experiments
405     training/                  # Offline detector training (see ROADMAP)
406     eval_baselines/           # Frozen evaluation baselines
407     scripts/
408         doctor.py              # Dependency & FFmpeg diagnostics + default config init
409         run_process_and_stitch.py # Interactive end-to-end runner
410     tools/
411         cleanup_caches.py      # Dry-run-first cache GC (WSL ~/clips + output/); see "Cache
Hygiene" above
412     run_all_tests.py           # Full test-suite wrapper (see "Running the Test Suite" above)
413     run_eval.py                # Rally evaluation CLI (see "Annotation & Evaluation" above)
414     run_regression.py          # Regression runner
415     pyproject.toml             # PEP 621 packaging (hatchling); `indexer` entry point + deps
416     config.json                # Runtime configuration (see "Threshold Tuning" above)
417     ```

```

14. user (2026-06-21T22:32:20.302Z)

```

backend/assets/fonts/README.md:Fonts shipped with the highlight **watermark**
(`stitching.watermark`), used by ffmpeg `drawtext`
CLAUDE.md:FastAPI + SQLite + ffmpeg + GPU CV/AI, with paid **Gemini** as the cloud AI. Sibling
CLAUDE.md:- Tests: `python run_all_tests.py` (offline, fully mocked). Python 3.8+; `ffmpeg`
required.
deploy/cloudrun/README.md:rally counts, `gpu_active_seconds`/`wall_seconds`/`bytes_out`, a reel
smoke-test (`ffmpeg -f null -`
deploy/cloudrun/README.md:- libx264 encode (parity, D16) + NVDEC decode-only; the image keeps
ffmpeg's default libx264.
deploy/digitalocean/README.md:./deploy/digitalocean/bootstrap.sh --gpu      # GPU box (installs
CUDA torch)
deploy/gcp/README.md:torch); reuse them on GCP:
deploy/gcp/README.md:./deploy/digitalocean/bootstrap.sh --gpu
# venv + torch cu124 (confirm cuda True)
deploy/gcp/README.md:. .venv/bin/activate && pip install -e .[storage-gcs]
# GCS backend
docs/ANALYZERS_AND_RECONCILERS.md:bounds. Analyzers are **pure** ? no I/O, no torch/cv2 ?
mirroring `fusion_features.py`'s discipline, so
docs/ANALYZERS_AND_RECONCILERS.md: torch / a second WSL pass ? gate behind their cue flags; all
deps stay MIT/Apache/BSD (ADR-002/005).
docs/ANALYZERS_AND_RECONCILERS.md:- **Offline, runnable in this dev container (no
GPU/torch/cv2):** pure analyzer math on synthetic
docs/archives/CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md:- **import-smoke**: minimal
deps only (no torch/cv2), runs `test_import_smoke.py`.
docs/archives/CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md:- **full-suite**: installs
system libs + CPU torch + full requirements + pytest/pytest-cov/httpx, then the coverage command
above with 70% floor (calibrated 2026-06-11 without obsreport).
docs/archives/CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md:**obsreport note**:
Reporting tests (harvest, spec rules, etc.) are present but skipped when the sibling package is
absent. Run `pip install -e ../sports-obsreport --no-build-isolation` locally for fuller
coverage.
docs/archives/CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md:Because the dev container
lacks GPU/torch/cv2/numpy (and per CLAUDE.md), all heavy verification is handed to the owner on
a real machine.
docs/archives/CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md:- Note: container limitation
(no pytest/numpy/torch). All green where executable. Owner must re-run full baseline on capable
machine before LGTM per protocol. Post #160 merge was clean (653 passed, 79.01% from LGTM
verification).
docs/archives/decisions/DECISIONS.md:1. **Drop `ultralitics`. Both the library and the
`yolov8n.pt` weights are **AGPL-3.0**; §13 makes them a blocker for a closed-source hosted
service. Swapping the codebase while keeping the `.pt` does **not** escape ? the weights are
AGPL too. Replace player/motion-gating detection with a permissive detector: **torchvision
Faster R-CNN / RetinaNet (BSD, default ? zero new deps, person = COCO class 0)**, escalating to
**YOLOX** or **RT-DETRv2 (Baidu repo, Apache-2.0)** if more speed/accuracy is needed.
**Ultralytics Enterprise License** is a paid fallback only (quote-only pricing).
docs/archives/decisions/DECISIONS.md:- Eliminates the only two hard legal blockers; the rest of
the stack (torch, opencv, fastapi, numpy, lapx, ffmpeg-as-subprocess, google-genai SDK) is
already permissive.
docs/archives/decisions/DECISIONS.md:- **Portability / single-runtime.** TPU needs XLA
(torch_xla / JAX) ? significant fragile NRE to port

```

15. assistant (2026-06-21T22:32:22.160Z)

Now let me check pyproject.toml for the entry point and packaging setup:

16. user (2026-06-21T22:32:22.598Z)

```

1      # PEP 621 packaging for the badminton/sports highlight indexer (DEFERRED #7).
2      #
3      # Build backend: hatchling. `backend/` is an intentional NAMESPACE package

```

```

4      # (no `backend/___init___py`; mypy.ini sets explicit_package_bases for the same
5      # reason), so the wheel target packages are listed EXPLICITLY below ? hatchling's
6      # auto-detection would otherwise miss the un-___init___d `backend` root.
7      #
8      # torch/torchvision are deliberately NOT listed in [project.dependencies]:
9      # their CPU/CUDA wheels need an out-of-band index (--index-url / --extra-index-url),
10     # which PEP 621 cannot express. Install them separately ? see [project.optional-
dependencies]
11     # `gpu` (documentation only) and the README. CI installs CPU torch on its own line.
12     [build-system]
13     requires = ["hatchling"]
14     build-backend = "hatchling.build"
15
16     [project]
17     name = "badminton-highlight-indexer"
18     version = "0.1.0"
19     description = "AI-assisted rally segmentation and highlight indexing for racket sports."
20     readme = "README.md"
21     requires-python = ">=3.10"
22     license = { text = "MIT" }
23     authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
24     keywords = ["badminton", "sports", "video", "highlights", "rally", "segmentation"]
25
26     # Runtime deps mirror requirements.txt, EXCEPT torch/torchvision (see header).
27     dependencies = [
28         "fastapi>=0.111.0",
29         "uvicorn>=0.30.1",
30         "opencv-python>=4.9.0.80",
31         "numpy>=1.26.4",
32         "pydantic>=2.7.1",
33         "jinja2>=3.1.4",
34         "python-dotenv>=1.0.0",
35         "google-genai>=2.4.0",
36         "supervision>=0.21.0,<0.30.0",
37         # Used directly on the serving path (backend/features/rally_seq.py: uniform_filter1d
/
38         # maximum_filter1d); previously present only TRANSITIVELY via supervision, so a
supervision
39         # bump or a pruned-transitive freeze could silently break inference. Declare it.
(BSD-3.)
40         "scipy>=1.10",
41     ]
42
43     [project.optional-dependencies]
44     # Test + lint/type tooling ? matches the CI lanes (.github/workflows/ci.yml).
45     dev = [
46         "pytest",
47         "pytest-cov",
48         "httpx",          # fastapi TestClient transport
49         "ruff",
50         "mypy",
51         "PyJWT[crypto]>=2.8", # M9 edge-auth: Cf-Access JWT verify (RS256) ? tests + CI
52     ]
53
54     # M9 edge-auth (Cloudflare Access). DEFAULT-OFF: only needed when
security.edge_auth_enabled=True.
55     # `jwt` is lazy-imported, so the base runtime works without this; install it for
hosted/multi-tenant.
56     auth = ["PyJWT[crypto]>=2.8"]
57
58     # Per-video observability reports (SOFT dependency: the pipeline degrades to a
59     # no-op if absent). Pinned to its git tag per BACKLOG. The private repo is
60     # fetched over SSH; if you lack access, simply omit this group ? reporting
61     # tests skip via pytest.importorskip. For local dev against a sibling checkout:
62     # pip install -e ../sports-obsreport --no-build-isolation

```

```

63     reporting = [
64         "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3",
65     ]
66
67     # `gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision
68     # CUDA wheels require `--index-url`/`--extra-index-url`, which PEP 621 cannot
69     # express, so they are NOT declarable here. Install them out-of-band instead:
70     # # CUDA 12.4 box:
71     # pip install torch>=2.12.0 torchvision>=0.27.0 --index-url
https://download.pytorch.org/whl/cu124
72     # # CPU-only box (also what CI uses):
73     # pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
74     gpu = []
75
76     # Storage backends (docs/DECOUPLED_COMPUTE/). The default `local` backend needs NONE of
77     # these. Cloud SDKs are imported lazily, so `import backend.storage` works without them;
78     # install the extra for the bucket you use. (boto3's S3 client covers AWS + Cloudflare
R2 +
79     # DO Spaces + MinIO via endpoint_url.) `gdrive://` needs NO extra ? it reads a locally
80     # MOUNTED Google Drive (Google Drive for Desktop) as plain files; see
backend/storage/gdrive.py.
81     storage-s3 = ["boto3>=1.34"]
82     storage-gcs = ["google-cloud-storage>=2.16"]
83
84     [project.scripts]
85     # `indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
86     indexer = "backend.main:main"
87
88     [tool.hatch.metadata]
89     # The `reporting` extra pins obsreport to a git tag (a PEP 508 *direct
90     # reference*); hatchling requires opting in to allow those in metadata.
91     allow-direct-references = true
92
93     [tool.hatch.build.targets.wheel]
94     # `backend` is a namespace package (no __init__.py); list both import roots
95     # explicitly so an editable/wheel install exposes `import backend` and
96     # `import training` regardless of cwd.
97     packages = ["backend", "training"]
98

```

17. user (2026-06-21T22:32:23.316Z)

```

./deploy/digitalocean/bootstrap.sh
./deploy/digitalocean/gpu_setup.sh
./deploy/digitalocean/mount_scratch.sh
./deploy/gcp/create_gpu_vm.sh
./deploy/gcp/ops-agent-startup.sh
./deploy/gcp/provision.sh
./deploy/gcp/teardown.sh
./scripts/bundle_ui.sh
./scripts/nightly_regression.ps1
./scripts/register_nightly_task.ps1

```

18. assistant (2026-06-21T22:32:25.693Z)

Let me check the test files to understand the testing setup:

19. user (2026-06-21T22:32:27.252Z)

```

-rw-r--r-- 1 avidu 197609 26459 Jun 22 04:00 tests/test_ablation.py
-rw-r--r-- 1 avidu 197609 3182 Jun 22 04:00 tests/test_annotation_providers.py
-rw-r--r-- 1 avidu 197609 33659 Jun 22 04:00 tests/test_api_endpoints.py
-rw-r--r-- 1 avidu 197609 7688 Jun 22 04:00 tests/test_app_home.py
-rw-r--r-- 1 avidu 197609 4428 Jun 22 04:00 tests/test_app_launcher.py

```

```
-rw-r--r-- 1 avidu 197609 15194 Jun 22 04:00 tests/test_async_jobs.py
-rw-r--r-- 1 avidu 197609 4833 Jun 22 04:00 tests/test_audio_features.py
-rw-r--r-- 1 avidu 197609 3320 Jun 22 04:00 tests/test_automated_provider.py
-rw-r--r-- 1 avidu 197609 8211 Jun 22 04:00 tests/test_base.py
-rw-r--r-- 1 avidu 197609 9612 Jun 22 04:00 tests/test_billing.py
-rw-r--r-- 1 avidu 197609 2538 Jun 22 04:00 tests/test_bind_and_host.py
-rw-r--r-- 1 avidu 197609 1122 Jun 22 04:00 tests/test_calibrate_local.py
-rw-r--r-- 1 avidu 197609 2032 Jun 22 04:00 tests/test_calibrate_wasb.py
-rw-r--r-- 1 avidu 197609 3668 Jun 22 04:00 tests/test_calibration.py
-rw-r--r-- 1 avidu 197609 7778 Jun 22 04:00 tests/test_cleanup_caches.py
-rw-r--r-- 1 avidu 197609 4131 Jun 22 04:00 tests/test_compiler.py
-rw-r--r-- 1 avidu 197609 14522 Jun 22 04:00 tests/test_compute_backend.py
-rw-r--r-- 1 avidu 197609 14961 Jun 22 04:00 tests/test_config.py
-rw-r--r-- 1 avidu 197609 17342 Jun 22 04:00 tests/test_config_deployment.py
-rw-r--r-- 1 avidu 197609 3249 Jun 22 04:00 tests/test_config_wasb.py
```

20. assistant (2026-06-21T22:32:29.189Z)

Let me check for the bootstrap.sh to see what it does:

21. user (2026-06-21T22:32:29.743Z)

```
1      #!/usr/bin/env bash
2      #
3      # Bootstrap a DigitalOcean / Paperspace Ubuntu box to run the rally pipeline.
4      #
5      #   ./deploy/digitalocean/bootstrap.sh           # CPU box (Droplet)           ? M0 /
M1-cpu
6      #   ./deploy/digitalocean/bootstrap.sh --gpu    # GPU box (Paperspace / GPU Droplet) ?
M1 / M3.5
7      #
8      # Mirrors .github/workflows/ci.yml (Ubuntu) so the box matches CI exactly.
9      # STORAGE-FREE + SECRET-FREE: proves the pipeline runs on the box via the mocked
10     # test suite. No bucket, no Gemini key, no WSL required. See
11     ../../docs/DECOUPLED_COMPUTE/.
12     set -euo pipefail
13     GPU=0
14     [ "${1:-}" = "--gpu" ] && GPU=1
15
16     echo ">> [1/4] System libraries (ffmpeg + opencv runtime + the watermark font)"
17     sudo apt-get update
18     # fonts-dejavu-core: the stitcher's watermark uses ffmpeg drawtext with DejaVu ? WITHOUT
it
19     # compile_highlights raises and NO reel is produced (found on the 2026-06-21 cloud dry
run: 22
20     # rallies detected but no reel). Matches deploy/cloudrun/Dockerfile. (Or set
stitching.watermark=false.)
21     sudo apt-get install -y python3 python3-venv python3-pip git ffmpeg libgl1 fonts-dejavu-
core
22     # opencv needs libgl12.0-0; the package name differs across Ubuntu releases (t64 on
24.04).
23     sudo apt-get install -y libgl12.0-0t64 || sudo apt-get install -y libgl12.0-0
24
25     echo ">> [2/4] Python venv (.venv)"
26     python3 -m venv .venv
27     # shellcheck disable=SC1091
28     . .venv/bin/activate
29     python -m pip install --upgrade pip
30
31     echo ">> [3/4] torch (${[ "$GPU" = 1 ] && echo CUDA || echo CPU}) + package (editable,
dev tooling)"
32     if [ "$GPU" = 1 ]; then
33         # CUDA 12.4 wheels. NOTE: verify your GPU arch is supported by this build.
34         # The Paperspace free M4000 is Maxwell (sm_52) ? old archs are sometimes dropped;
```



```

35     # if `torch.cuda.is_available()` is False on an M4000, use a P4000/RTX4000 or pin
36     # an older torch. (docs/DECOUPLED_COMPUTE/03 + 06.)
37     pip install torch torchvision --index-url https://download.pytorch.org/whl/cu124
38 else
39     pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
40 fi
41 # Runtime deps + pytest/ruff/mypy. The 'reporting' extra (private obsreport over SSH)
42 # is intentionally omitted ? it is a soft dep and its tests skip when absent.
43 pip install -e .[dev]
44
45 echo ">> [4/4] Device check"
46 python - <<'PY'
47 import torch
48 ok = torch.cuda.is_available()
49 print("torch", torch.__version__, "| cuda_available", ok,
50       "| device", (torch.cuda.get_device_name(0) if ok else "CPU only"))
51 PY
52
53 cat <<'EOF'
54
55 >> Bootstrap complete. Run the storage-free, secret-free smoke (mocked suite):
56
57     . .venv/bin/activate && python -m pytest -q
58
59     Expect ~900 passed. That proves the whole pipeline runs on this box with no GPU,
60     no WSL, no bucket and no API key. (On a GPU box the device check above should
61     print cuda_available True.)
62 EOF
63

```

22. assistant (2026-06-21T22:32:31.799Z)

Now let me check the tests for config/app_home/doctor to understand the conventions:

23. user (2026-06-21T22:32:32.239Z)

```

1     """Tests for the resolved application-data HOME (PACKAGING_PLAN.md P0a).
2
3     The load-bearing guarantee: with ``RALLY_APP_HOME`` UNSET, every resolver is byte-
4     identical to the
5     pre-P0a CWD-relative behaviour; with it SET, config/.env/output/DB all root under that
6     one home.
7     """
8
9     import os
10    from pathlib import Path
11
12    import pytest
13
14    from backend.utils import app_home as ah
15    from backend.config import loader
16
17    @pytest.fixture(autouse=True)
18    def _clear_app_home_env(monkeypatch):
19        """Each test starts with RALLY_APP_HOME unset unless it sets it explicitly."""
20        monkeypatch.delenv(ah.APP_HOME_ENV, raising=False)
21
22    # ---- default (env unset) is byte-identical to today ----
23
24    def test_app_home_defaults_to_cwd():
25        assert ah.app_home() == Path(".")
26
27

```

```

28     def test_default_config_path_unset_is_cwd_config_json():
29         # Pre-P0a this was the module constant Path("config.json").
30         assert ah.default_config_path() == Path("config.json")
31
32
33     def test_default_env_path_unset_is_cwd_dotenv():
34         assert ah.default_env_path() == Path(".env")
35
36
37     def test_resolve_output_dir_unset_is_verbatim_for_relative():
38         # env unset ? returned VERBATIM: no pathlib normalisation of ./ , separators, or
trailing slash
39         # (the pre-P0a code stored --output-dir as-is, and that string is echoed by
/api/config + reports).
40         for v in ("output", "custom", "./output", "out/clips", "output/"):
41             assert ah.resolve_output_dir(v) == v
42         assert ah.resolve_output_dir("") == "output" # empty ? the historical default
43
44
45     def test_env_file_to_load_unset_returns_none(monkeypatch, tmp_path):
46         # Even with a .env sitting in the CWD, the unset path must NOT prefer it ? bare
load_dotenv()
47         # walks up from the caller FILE, not the CWD, so None preserves that exact
behaviour.
48         monkeypatch.chdir(tmp_path)
49         (tmp_path / ".env").write_text("X=1\n", encoding="utf-8")
50         assert ah.env_file_to_load() is None
51
52
53     # ---- RALLY_APP_HOME set ? everything roots there ----
54
55     def test_app_home_honors_env(monkeypatch, tmp_path):
56         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
57         assert ah.app_home() == tmp_path
58
59
60     def test_app_home_expands_user(monkeypatch):
61         monkeypatch.setenv(ah.APP_HOME_ENV, "~/khehsutra-data")
62         assert ah.app_home() == Path.home() / "khehsutra-data"
63
64
65     def test_app_home_ignores_blank_env(monkeypatch):
66         monkeypatch.setenv(ah.APP_HOME_ENV, " ")
67         assert ah.app_home() == Path(".")
68
69
70     def test_config_and_env_paths_root_under_home(monkeypatch, tmp_path):
71         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
72         assert ah.default_config_path() == tmp_path / "config.json"
73         assert ah.default_env_path() == tmp_path / ".env"
74         assert ah.app_path("output", "x.db") == tmp_path / "output" / "x.db"
75
76
77     def test_resolve_output_dir_set(monkeypatch, tmp_path):
78         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
79         assert ah.resolve_output_dir("output") == str(tmp_path / "output")
80
81
82     def test_resolve_output_dir_absolute_passthrough(monkeypatch, tmp_path):
83         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
84         abs_dir = str(tmp_path / "elsewhere") if os.path.isabs(str(tmp_path)) else
os.path.abspath("elsewhere")
85         assert ah.resolve_output_dir(abs_dir) == abs_dir
86
87

```

```

88     def test_resolve_output_dir_empty_falls_back_to_home_output(monkeypatch, tmp_path):
89         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
90         assert ah.resolve_output_dir("") == str(tmp_path / "output")
91
92
93     # ---- default_os_app_home: the OS-conventional dir the launcher exports (NOT auto-
selected) ----
94
95     def test_default_os_app_home_windows(monkeypatch, tmp_path):
96         monkeypatch.setattr(ah.platform, "system", lambda: "Windows")
97         monkeypatch.setenv("APPDATA", str(tmp_path / "Roaming"))
98         assert ah.default_os_app_home() == tmp_path / "Roaming" / "Khelsutra"
99
100
101     def test_default_os_app_home_macos(monkeypatch):
102         monkeypatch.setattr(ah.platform, "system", lambda: "Darwin")
103         assert ah.default_os_app_home() == Path.home() / "Library" / "Application Support" /
"Khelsutra"
104
105
106     def test_default_os_app_home_linux_xdg(monkeypatch, tmp_path):
107         monkeypatch.setattr(ah.platform, "system", lambda: "Linux")
108         monkeypatch.setenv("XDG_DATA_HOME", str(tmp_path / "share"))
109         assert ah.default_os_app_home() == tmp_path / "share" / "Khelsutra"
110
111
112     def test_default_os_app_home_linux_default(monkeypatch):
113         monkeypatch.setattr(ah.platform, "system", lambda: "Linux")
114         monkeypatch.delenv("XDG_DATA_HOME", raising=False)
115         assert ah.default_os_app_home() == Path.home() / ".local" / "share" / "Khelsutra"
116
117
118     # ---- loader integration: load_config() roots on the app-home ----
119
120     def test_loader_reads_config_from_app_home(monkeypatch, tmp_path):
121         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
122         (tmp_path / "config.json").write_text('{"sport": "tennis"}', encoding="utf-8")
123         cfg = loader.load_config() # no explicit path ? default_config_path() under the
home
124         assert cfg.sport == "tennis"
125
126
127     def test_loader_no_config_in_home_uses_defaults(monkeypatch, tmp_path):
128         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path)) # empty dir, no config.json
129         cfg = loader.load_config()
130         # Defaults load cleanly (no crash, no file) ? same contract as a missing CWD
config.json.
131         assert cfg is not None
132         assert hasattr(cfg, "indexing")
133
134
135     def test_save_default_config_writes_under_app_home(monkeypatch, tmp_path):
136         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
137         written = loader.save_default_config() # no explicit path
138         assert written == tmp_path / "config.json"
139         assert written.exists()
140
141
142     def test_explicit_path_still_overrides_app_home(monkeypatch, tmp_path):
143         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path / "home"))
144         explicit = tmp_path / "explicit.json"
145         explicit.write_text('{"sport": "pickleball"}', encoding="utf-8")
146         cfg = loader.load_config(str(explicit))
147         assert cfg.sport == "pickleball"
148

```

```

149
150 # ---- env_file_to_load: app-home .env preferred ONLY when RALLY_APP_HOME is set ----
151
152 def test_env_file_to_load_set_and_present(monkeypatch, tmp_path):
153     monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
154     (tmp_path / ".env").write_text("X=1\n", encoding="utf-8")
155     assert ah.env_file_to_load() == tmp_path / ".env"
156
157
158 def test_env_file_to_load_set_but_absent_returns_none(monkeypatch, tmp_path):
159     monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path)) # home pinned, but no .env in it
160     assert ah.env_file_to_load() is None
161
162
163 # ---- resolve_output_dir passthrough for absolute / Windows drive-anchored inputs ----
164
165 def test_resolve_output_dir_set_absolute_passthrough(monkeypatch, tmp_path):
166     monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
167     abs_p = os.path.abspath(os.sep + "data" + os.sep + "reels")
168     assert ah.resolve_output_dir(abs_p) == abs_p
169
170
171 def test_resolve_output_dir_set_windows_drive_passthrough(monkeypatch, tmp_path):
172     monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
173     if os.name == "nt":
174         # drive-anchored ("C:out" / "C:\\out") must NOT be re-rooted under the app home
175         assert ah.resolve_output_dir("C:out") == "C:out"
176         assert ah.resolve_output_dir("C:\\out") == "C:\\out"
177     else:
178         pytest.skip("Windows drive-letter semantics only meaningful on nt")
179
180
181 # ---- the package-relative static mount (CWD-independent) ----
182
183 def test_main_static_dir_is_package_relative():
184     import backend.main as m
185     expected = os.path.join(os.path.dirname(os.path.abspath(m.__file__)), "api",
186 "static")
187     assert m._STATIC_DIR == expected
188     assert os.path.isabs(m._STATIC_DIR)
189

```

24. user (2026-06-21T22:32:32.449Z)

```

1     """Basic tests for the typed configuration package.
2
3     These ensure:
4     - Defaults are sensible and come from the models.
5     - File overrides work.
6     - Old config files with extra keys are tolerated.
7     - save_default_config produces loadable output.
8     """
9     import json
10    import tempfile
11    import os
12
13
14    from backend.config import load_config, save_default_config, AppConfig
15
16
17    def test_load_default_config():
18        # Force load without the project's config.json so we test the model built-in
19        defaults
20        cfg = load_config("/nonexistent/path/that/does/not/exist/12345/config.json")
21        assert isinstance(cfg, AppConfig)
22

```

```

21     assert cfg.sport == "badminton"
22     assert cfg.indexing.default_segmenter == "yolo_hybrid"
23     assert cfg.validation.min_blur_threshold == 20.0 # updated default from review
24     assert cfg.indexing.ai_max_workers == 5
25
26
27     def test_logging_config_default_quiets_http():
28         """verbose_http defaults to False so run.log is quiet unless explicitly turned
up. """
29         cfg = load_config("/nonexistent/path/12345/config.json")
30         assert cfg.logging.verbose_http is False
31
32
33     def test_skip_ai_handoff_default_off():
34         """Fully-local mode is opt-in: skip_ai_handoff defaults False so the AI handoff
runs. """
35         cfg = load_config("/nonexistent/path/12345/config.json")
36         assert cfg.indexing.skip_ai_handoff is False
37
38
39     def test_skip_ai_handoff_file_override():
40         with tempfile.TemporaryDirectory() as td:
41             p = os.path.join(td, "skip.json")
42             with open(p, "w") as fh:
43                 json.dump({"indexing": {"skip_ai_handoff": True}}, fh)
44             cfg = load_config(p)
45             assert cfg.indexing.skip_ai_handoff is True
46
47
48     def test_logging_config_file_override():
49         with tempfile.TemporaryDirectory() as td:
50             p = os.path.join(td, "logging.json")
51             with open(p, "w") as fh:
52                 json.dump({"logging": {"verbose_http": True}}, fh)
53             cfg = load_config(p)
54             assert cfg.logging.verbose_http is True
55
56
57     def test_fusion_config_defaults_reproduce_control():
58         """The fusion block defaults must be no-ops: wasb substrate, no cues, no gating, no
mask.
59
60         Gate A (min_crossings) defaults to 0 = OFF: the #146 served-default flip was
REVERTED
61         (eval?serve ? the +0.074 was on the harness's recall-oriented PROPOSAL windowing; on
the
62         served COARSE windowing Gate A is neutral/negative, ?0.008, dropping real rallies).
See
63         docs/QUALITY_ITERATIONS.md and the served litmus in
tests/test_served_gate_a_regression.py.
64         """
65         cfg = load_config("/nonexistent/path/12345/config.json")
66         f = cfg.indexing.fusion
67         assert f.substrate == "wasb"
68         assert f.cue_flags == {} # person cue OFF
69         assert f.min_crossings == 0 # Gate A OFF (default) ? #146 served flip
reverted
70         assert f.min_players == 0 # Gate B OFF
71         assert f.court_polygon is None # no mask
72         assert f.velocity_threshold == 0.02
73
74
75     def test_fusion_config_gate_a_opt_in_via_min_crossings():
76         """Gate A is opt-in: setting min_crossings explicitly enables it (the eval-only
operating

```

```

77         point), while the served default stays OFF (#146 flip reverted)."""
78         with tempfile.TemporaryDirectory() as td:
79             p = os.path.join(td, "optin.json")
80             with open(p, "w") as fh:
81                 json.dump({"indexing": {"fusion": {"min_crossings": 2}}}, fh)
82             cfg = load_config(p)
83             assert cfg.indexing.fusion.min_crossings == 2    # Gate A ON only when explicitly
set
84
85
86     def test_served_default_segmenter_is_gemini():
87         """The shipped config.json served default is the prior detector (gemini) ? the #146
flip
88         to fusion_hybrid was REVERTED as a confirmed served regression (eval?serve)."""
89         repo_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
90         cfg = load_config(os.path.join(repo_root, "config.json"))
91         assert cfg.indexing.default_segmenter == "gemini"
92         # ...and the served fusion default keeps Gate A OFF (no served gating).
93         assert cfg.indexing.fusion.min_crossings == 0
94
95
96     def test_fusion_config_file_override():
97         with tempfile.TemporaryDirectory() as td:
98             p = os.path.join(td, "fusion.json")
99             with open(p, "w") as fh:
100                 json.dump({"indexing": {"fusion": {

```

25. user (2026-06-21T22:32:32.630Z)

```

1         """Tests for scripts/doctor.py::init_default_config (PACKAGING_PLAN.md P0a).
2
3         doctor's --setup config write now follows
`backend.utils.app_home.default_config_path()` ? the
4         SAME resolver the server READS ? instead of the repo root, so the two can't disagree
once the app
5         is launched from an arbitrary CWD.
6         """
7
8         import importlib.util
9         import os
10        import sys
11
12        import pytest
13
14        from backend.utils import app_home as ah
15
16        # scripts/ isn't a package ? load the module by path (mirrors
tests/test_nightly_regression.py).
17        _SCRIPT = os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
18                                "scripts", "doctor.py")
19        _spec = importlib.util.spec_from_file_location("doctor", _SCRIPT)
20        doctor = importlib.util.module_from_spec(_spec)
21        sys.modules["doctor"] = doctor
22        _spec.loader.exec_module(doctor)
23
24
25        @pytest.fixture(autouse=True)
26        def _clear_app_home_env(monkeypatch):
27            monkeypatch.delenv(ah.APP_HOME_ENV, raising=False)
28
29
30        def test_init_writes_config_under_app_home(monkeypatch, tmp_path):
31            monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
32            doctor.init_default_config()
33            written = tmp_path / "config.json"

```

```

34         assert written.exists(), "config.json must land under RALLY_APP_HOME (the server's
read path)"
35         # And it is the SAME path the loader resolves ? the footgun this PR closes.
36         assert written == ah.default_config_path()
37
38
39     def test_init_skips_when_config_already_exists(monkeypatch, tmp_path):
40         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
41         existing = tmp_path / "config.json"
42         existing.write_text('{"sport": "tennis"}', encoding="utf-8")
43         doctor.init_default_config() # must NOT overwrite
44         assert existing.read_text(encoding="utf-8") == '{"sport": "tennis"}'
45
46
47     def test_init_unset_writes_to_cwd(monkeypatch, tmp_path):
48         # env unset ? CWD/config.json (the server, run from the same CWD, reads exactly
this).
49         monkeypatch.chdir(tmp_path)
50         doctor.init_default_config()
51         assert (tmp_path / "config.json").exists()
52
53
54     def test_init_is_noop_when_backend_import_fails(monkeypatch, tmp_path, capsys):
55         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
56
57         real_import = __builtins__["__import__"] if isinstance(__builtins__, dict) else
__builtins__.__import__
58
59         def _boom(name, *args, **kwargs):
60             if name == "backend.config" or name.startswith("backend.config"):
61                 raise ImportError("simulated missing backend.config")
62             return real_import(name, *args, **kwargs)
63
64         monkeypatch.setattr("builtins.__import__", _boom)
65         doctor.init_default_config() # must degrade to a no-op, never raise
66         assert not (tmp_path / "config.json").exists()
67         assert "Falling back to no-op" in capsys.readouterr().out
68

```

26. assistant (2026-06-21T22:32:34.529Z)

Now let me search for any existing uninstall manifests or uv usage:

27. user (2026-06-21T22:32:35.388Z)

docs/PACKAGING_PLAN.md:2. ****P2b-install**** (\$?) ? `uv tool install` / a generated cross-platform install script (now a clean ****torch-free**** LIGHT install thanks to P2e) + ship a truly-local default `config.json` + an uninstall manifest.

docs/PACKAGING_PLAN.md: | ****LIGHT**** (v1 first increment) | Non-tech India coach/parent (Win/Mac/Linux); any CPU box | wheel + bundled SPA + Gemini/motion/cpu-mock; ****no torch/GPU/weights****; ffmpeg via per-OS offer; defaults truly-local; user-supplied Gemini key | ****Cross-platform, no native component:**** `uv tool install` / a generated install script + a ****pure-Python launcher**** (`indexer --app` ? boot server ? `webbrowser.open 127.0.0.1:8000`, free-port fallback). Optional thin per-OS shortcut (`.bat`/`.command`/`.desktop`) wrapping the same command. First run: cpu-mock self-test ? 3-step wizard. |

docs/PACKAGING_PLAN.md: | P2b-install | ****Install path**** (`uv tool install` / a generated cross-platform install script; ship `config.json` defaulting truly-local; optional thin per-OS shortcut to `indexer --app`) | P2b-launcher | No (signing only if clickable binaries ? 09) | ? | ? |

docs/PACKAGING_PLAN.md:****Internal code anchors `[verified]`:****

```

`pyproject.toml:80-82,8-37,63-70`; `backend/main.py:10,64-74,144-146,275-315,646,729-736,756`;
`backend/config/loader.py:30,112,202`; `backend/config/presets.py`;
`backend/compute/__init__.py`; `backend/storage/*`;
`backend/pipeline/detectors/native_wasb_runner.py:17,89,110-112`;
`backend/pipeline/stitcher/compiler.py:253`; `backend/features/rally_seq.py:17`;

```

```
`backend/infrastructure/database.py:88-110` (user_version=6); `training/gen0/harness.py:174-177`
(`ship=False`); `deploy/cloudrun/Dockerfile:33-37,54`. **Internal docs:**
`docs/COMPUTE_DECOUPLED_SERVING/`, `docs/DESKTOP_APP_PLAN.md` (superseded),
`docs/UI_PROPOSAL.md`, `docs/I18N_PLAN.md`, `kheIsutra/docs/{LOCAL_APPLIANCE_ARCHITECTURE,ANNO
TE_AND_TRAIN, DRY_RUN_CUJ, FRIENDLY_AND_MULTILINGUAL}.md`. **External `[researched]`: **uv torch-
index docs; Frigate/Ollama/ComfyUI/LM Studio distribution; conda-forge/micromamba ToS; ONNX
Runtime; ffmpeg GPL-vs-LGPL.
```

28. assistant (2026-06-21T22:32:37.909Z)

Perfect! Now let me check the PACKAGING_PLAN.md more thoroughly:

29. user (2026-06-21T22:32:38.591Z)

```
1      # Batteries-Included App ? Packaging & Distribution Plan
2
3      > **Status:** `IN PROGRESS` ? 01/02 resolved (LIGHT-first; cross-platform webserver, no
native component; **WASB-complete north star**); **11 PRs merged 2026-06-21/22** (P0a?e, P1a/b,
P2-launcher/P2a/P2d/P2e); 03+ owner/counsel-gated . **Owner:** Avi Dullu . **Created:**
2026-06-21 . **Last updated:** 2026-06-22
4      > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
5      > **Tracking anchors:** $7 progress tracker is the source of truth; indexed in
`docs/README.md` + `docs/NEXT_STEPS.md`; memory `[[packaging-app-plan]]` + `[[next-session-
packaging-app]]`.
6      > **Relation to existing docs:** **supersedes** the narrative of
`docs/DESKTOP_APP_PLAN.md` (per locked D10 ? self-hosted webserver, not a desktop app); **builds
on** `docs/COMPUTE_DECOUPLED_SERVING/` (the profile/storage/compute seam) and is the
**distribution layer** under `kheIsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md`.
7      > **Honesty note:** claims tagged `[verified]` (checked at file:line), `[researched]`
(web/needs sign-off), `[design]` (proposed). Not legal advice ? licensing items need counsel.
8
9      ---
10
11     ## ? NEXT SESSION ? start here
12     **You are here (2026-06-22):** `origin/master` has the **LIGHT-tier engine fully bundle-
ready** ? `pip install` ? `indexer --app` boots and serves the real KheIsutra SPA single-origin,
**torch-free**, with resolved app-home state, an env-overridable ffmpeg, DNS-rebinding-guarded
localhost binding, single-instance locking, and safe DB upgrades. 11 PRs merged; 0 open PRs; 0
GCP VMs ($0). Ramp up via this doc's $7 tracker + memory `[[packaging-app-plan]]`.
13
14     **Remaining work, in priority order (each its own small PR; branch off `origin/master`,
explicit paths ? shared checkout):**
15     1. **P3 GPU validation + screencast (owner wants first; ~$0.50, well under the $100
cap).** Provision an L4 (`deploy/gcp/create_gpu_vm.sh`) ? install the **wheel** on the fresh GPU
box + the WASB env ? validate the packaged stack end-to-end (`indexer --app` ? bundled SPA ?
native-WASB ? reel) ? **screencast** ? `teardown.sh` (deletes) + confirm $0. ? Run as a FOCUSED
op (billable VM ? teardown is priority #1). ? Known blocker: native-WASB returned **0 rallies**
on a fresh build last session ? reproducing the 22-rally result needs the engine session's exact
tuned `worker infer detector_impl=native` config. Confirm cmd + cost before spend; `[[gcp-vm-
always-delete]]`.
16     2. **P2b-install** ($7) ? `uv tool install` / a generated cross-platform install script
(now a clean **torch-free** LIGHT install thanks to P2e) + ship a truly-local default
`config.json` + an uninstall manifest.
17     3. **P2c first-run wizard** ($7) ? the 3-step wizard UI (Gemini key / compute / videos)
in the **kheIsutra `web/` SPA**, rendered from `GET /api/capabilities`. **Cross-repo**
(`avidullu/kheIsutra`) ? coordinate like P1b (the SPA is gitignored-dist; refresh `backend/ui/`
via `scripts/bundle_ui.sh` after).
18     4. Then **P3-cloud** (Cloud Run dispatch, micromamba Dockerfile ? P3a) and **P4** (ONNX
export ? dissolve the dual-env; training + in-app annotation = the **north-star** completion).
19
20     ---
21
22     ## 0. TL;DR
23     Ship the engine as **ONE pip-installable artifact whose behaviour is selected entirely
```


by the already-merged decoupled-compute seam** (`deployment.profile` / `compute\_target` / Storage+Compute backends), fronted by a friendly launcher + a Windows installer ? **not** a frozen monolith, **not** an Electron/Tauri shell. The hard constraint that forces scope: torch is deliberately out of `pyproject` (needs `--index-url`) **and** native WASB needs a *second, incompatible* torch env (conda py3.8 + torch 1.11+cu113) shelled out via `\$WASB\_PYTHON` `[verified native\_wasb\_runner.py:89]` ? so no single venv/freeze can deliver the GPU-quality path. **The honest v1 is a LIGHT tier** (Gemini + motion + cpu-mock, *zero* torch/GPU) that preserves CUJ-1.5/8; the native-WASB GPU path (CUJ-6/7) ships **owner-only** until two blockers clear (WASB-C3 weight provenance + a reproducible truly-local config). **LIGHT is the FIRST shippable increment, not the terminal scope ? the sole completion goal (north star, L10) is the FULL quality path: native WASB + BYO training + in-app annotation.** Delivery is **cross-platform (Windows/macOS/Linux): the same local webserver powers the UI on all three, with NO native component** (L9). **Five small engine prerequisites must land before any packaging push** (§7 P0). The single most actionable finding: **all mutable state is CWD-relative today and `--setup`/server disagree on `config.json`** ? a silent data-loss footgun that P0 must fix first.

24

25 **## 1. Problem & goal**

26 ****Goal:**** a non-technical user (the primary persona: an India coach/parent on Windows ? `docs/UI\_PROPOSAL.md` §2) **downloads and runs** the indexer **without sourcing the repo**. The app spins up a **local web service** for (a) inference video?rallies?reel, (b) BYO **training**, and (c) **rally annotation** if needed ? runnable **locally and against cloud services** ? and **preserves every completed CUJ** (§ CUJ map).

27

28 ****Why now:**** scaffolding + the decoupled-compute serving seam (M1?M8) are merged and default-OFF; the SPA + i18n + the dry-run CUJ are done. What's missing is the *distribution* story: today the only path is `git clone` ? `pip install -e .` ? out-of-band torch ? manual ffmpeg ? set up the WASB conda env ? build the SPA from a *different repo*. This plan collapses that into download-and-run.

29

30 ****Read to get current:**** `pyproject.toml`, `backend/main.py` (`main()` + `--server`), `backend/config/{loader,presets,models}.py`, `backend/storage/\*`, `backend/compute/\*`, `backend/pipeline/detectors/native\_wasb\_runner.py`, `deploy/cloudrun/Dockerfile`, `docs/COMPUTE\_DECOUPLED\_SERVING/README.md`, `khehsutra/web/` + `khehsutra/docs/LOCAL\_APPLIANCE\_ARCHITECTURE.md`.

31

32 **## 2. Decisions locked (do not relitigate)**

33

#	Decision	Source / date	Implication for packaging
L1	**Pure DETECT?WINDOW?STITCH core never branches on profile**;	one build, profile-selected COMPUTE_DECOUPLED_SERVING D1	Local vs cloud is **one config flip**, not a fork ? build ON this, don't reinvent.
L2	**Profiles** = truly-local / cloud-serving / cpu-mock via `PROFILE_PRESETS`	M1 #279	[verified backend/config/presets.py] The app exposes a single "where does compute run" choice.
L3	**Compute = GCP only** (Cloud Run GPU Jobs); DigitalOcean dropped	ADR-013, 2026-06-20	Cloud tier targets Cloud Run + BYO-GPU-VM only.
L4	**Edge-auth required before any non-localhost exposure** (Cloudflare Access + in-app JWT verify)	D9 / M9	[verified backend/api/auth.py wired default-OFF main.py:294] Bind stays `127.0.0.1` by default; `0.0.0.0` only under an edge-auth profile.
L5	**Delivery = self-hosted webserver the user spins up ? NOT a desktop app**	D10, 2026-06-20 (supersedes `DESKTOP_APP_PLAN.md`); **01 confirmed 2026-06-21**	The launcher boots the server + opens the user's *own browser* at localhost ? that IS the delivery, no embedded webview.

41 | L6 | **MIT / Apache-2.0 / BSD only** for every dependency ? code, data, AND weights | ratified 2026-06-09 | Every *new packaging* dep must clear this (see §3).

42 | L7 | **Paid LLMs (Gemini) = inference/serving only, never a training data source**;

commercial models trained on paid-LLM labels are ship-blocked | guardrail + `gate\_verdict`

[verified harness.py:177 ship=False] | The wizard's Gemini-key step must say so; training UI must surface the ship-gate.

43 | L8 | **User-switchable profiles** ("your machine or our cloud") with explicit confirm ? cloud spend never silent | D13 / D15 | "Push to cloud" toggle needs confirm + INR estimate from `backend/billing.py`.

44 | **L9** | **Cross-platform (Windows + macOS + Linux); NO native component** ? the same

```

local webserver powers the UI on all three | 01, 2026-06-21 | Distribution = `uv`/pip + a
**pure-Python launcher** (`webbrowser.open`). Reject native-installer-as-runtime,
Electron/Tauri, and LGPL tray libs. OSes differ only in ffmpeg/torch *acquisition*. |
45 | **L10** | **North star = the FULL quality path is the SOLE completion goal** ? LIGHT
is the first increment; DONE requires native WASB + BYO training + in-app annotation | 02,
2026-06-21 | §9 DoD splits "v1 shippable" from "north-star DONE"; P3/P4
(WASB/ONNX/train/annotate) are completion-gating, not optional. |
46
47     ## 3. Foundation ? research / prior art
48
49     **Distribution channels `[researched]`** ? `uv` is the only channel that can both
install a pinned Python *and* route torch's `--index-url`; `pipx`/plain `pip` can't express the
torch index. Frozen binaries (PyInstaller/Nuitka) with torch+CUDA are 3?5 GB, can't split CUDA
libs, and **cannot host the second WASB env** ? rejected for the GPU path. Prior art (Frigate
NVR ? closest analog: ffmpeg + ML + web + Docker; Ollama; ComfyUI; LM Studio) converges on **two
patterns that fit us:** (1) a **container image** for the server/GPU profile, (2) a **first-run
download + a friendly launcher** for the local app.
50
51     **Licensing landmines `[verified]` (the load-bearing §3 findings):**
52     - **WASB-C3 is the dominant blocker.** The WASB-SBDT *code* is MIT, but the badminton
*checkpoint* provenance is **unresolved** for commercial redistribution ? `gate_verdict`
hardcodes `ship=False` `[verified training/gen0/harness.py:174-177]`. **Do not conflate MIT-code
with cleared-weights.** A public download must **not** auto-fetch the weights until counsel
signs off (03).
53     - **ffmpeg is GPL via the `libx264` encoder** `[verified
backend/pipeline/stitcher/compiler.py:253]`. The license-clean path is **not to redistribute
ffmpeg** ? resolve a system/winget install instead. `imageio-ffmpeg` also lacks `ffprobe` (which
the engine needs) `[researched]`.
54     - **Anaconda ToS exposure:** the Cloud Run image uses `Miniconda3` + `conda tos accept` ?
pkgs/main`/`pkgs/r` `[verified deploy/cloudrun/Dockerfile:33-37]` ? the ?200-employee ToS trap.
Replace with **micromamba + conda-forge** (BSD, no ToS) for the cloud image *and* any local
managed WASB env; record an ADR.
55     - **New packaging deps that clear L6 `[researched]`:** `uv` (Apache/MIT), Inno Setup
(permissive), `onnxruntime` (MIT), `micromamba` (BSD-3), Docker/OCI (Apache-2.0), boto3/google-
cloud-* (Apache-2.0), stdlib `webbrowser`. **Forbidden:** `pystray` (LGPL-3.0). **Build-time-
only:** `Nuitka` 4.x is **AGPL-3.0+** (compiled output has a runtime exception) ? usable as a
tool, never bundled. `obsreport` (private git dep) stays a truly-optional extra, never bundled.
56
57     ## 4. Design / architecture
58
59     ### 4.1 One build, profile-selected (the reuse map)
60     - **Reuse:** hatchling build + `indexer = backend.main:main` console script `[verified
pyproject.toml:80-82]`; `PROFILE_PRESETS` `[verified presets.py]`; `get_compute_backend(config)`
returns `CloudRunCompute` only for `compute_target=='cloud_run'` else `None`=in-process
`[verified backend/compute/__init__.py]`; `backend/storage/*` (local/gcs/s3/gdrive); the in-
process `ThreadPoolExecutor(max_workers=1)` job worker `[verified backend/api/job_worker.py]` ?
**server and worker are the same process for serving** `[verified main.py:729-736]`, so a
packaged launcher just runs `indexer --server`.
61     - **New:** the app defaults to **truly-local** ; the user toggles profile in Settings
(rendered from a new `/api/capabilities`) with explicit confirm (L8) + INR estimate before any
cloud job.
62
63     ### 4.2 How one build runs local + cloud
64     The local FastAPI server is the **control plane in every profile.** `truly-local` runs
detect/stitch in-process. `cloud-serving` flips `compute_target=cloud_run` ? the local control
plane dispatches a **Cloud Run GPU Job** (the *same* image, forced inline) and bucket-polls a
done-marker; `storage=gcs` routes I/O. `cpu-mock` gives a $0 deterministic self-test on first
run.
65
66     ### 4.3 How the UI is bundled (collapses the cross-repo + reverse-proxy problem)
67     Build `khelsutra/web` (`npm ci && npm run build`) **at package time** and vendor
`web/dist/` into the wheel via a **hatchling force-include** ? a package dir (e.g.
`backend/ui/`), served by FastAPI with an **SPA history-fallback** route. The SPA already calls
**same-origin `BASE='/api'`** `[verified khelsutra/web client.ts:7]` ? **zero SPA code change**
for single-origin serving. Retire the legacy CWD-relative `backend/api/static` mount `[verified

```

```

main.py:71-72]`; resolve paths via `importlib.resources`. `dist/` is gitignored ? the build must
record a bundle hash + an `/api/version` handshake + a contract test pinning SPA?engine.
68
69     ### 4.4 Dual-torch / weights / ffmpeg ? concretely
70     - **Dual-torch:** do **not** host both ABIs in one venv or freeze. v1 LIGHT carries **no
torch at all**. The GPU/WASB HEAVY tier is a **separately-provisioned second env** ? short term
via **micromamba + conda-forge** reached by `$WASB_PYTHON` exactly as `native_wasb_runner.py:89`
already does; **strategic fix = export WASB to ONNX** (the `DetectorRunner` ABC already names an
`ONNXRunner` writing the identical `Frame,Visibility,X,Y` CSV) so the WASB path needs **neither**
torch 1.11 nor 2.x ? `onnxruntime` (MIT, tens of MB, CPU+CUDA+DirectML+CoreML) dissolves the
dual-env. Owner-gated on a real-GPU byte-parity re-validation.
71     - **Weights:** never bake `wasb_badminton_best.pth.tar` into the artifact (size + C3).
Fetch on first run, **SHA-256-pinned + resumable**, into the app-data cache, setting
`$WASB_WEIGHTS`. **Until C3 clears, the public download must NOT auto-fetch ? owner-only flag.**
72     - **ffmpeg:** route **every** `ffmpeg`/`ffprobe` call through **one resolver** (config
path ? env ? `shutil.which` ? fail-loud); first-run setup (extend `scripts/doctor.py`, which
already prints the winget command) detects-and-offers `winget install Gyan.FFmpeg`. An LGPL
ffmpeg bundle (BtbN,openh264/NVENC) is a documented fallback that also requires swapping the
`libx264` default ? tracked, not v1.
73
74     ### 4.5 Tiers
75     | Tier | Audience | Contents | Install UX |
76     |-----|-----|-----|-----|
77     | **LIGHT** (v1 first increment) | Non-tech India coach/parent (Win/Mac/Linux); any CPU
box | wheel + bundled SPA + Gemini/motion/cpu-mock; **no torch/GPU/weights**; ffmpeg via per-OS
offer; defaults truly-local; user-supplied Gemini key | **Cross-platform, no native component:**
`uv tool install` / a generated install script + a **pure-Python launcher** (`indexer --app` ?
boot server ? `webbrowser.open 127.0.0.1:8000`, free-port fallback). Optional thin per-OS
shortcut (`.bat`/`.command`/`.desktop`) wrapping the same command. First run: cpu-mock self-test
? 3-step wizard. |
78     | **HEAVY / local-GPU** (owner + opt-in power user) | Owner w/ golden corpus + CUDA box
| LIGHT + micromamba WASB env (py3.8/torch1.11) via `$WASB_PYTHON`; main torch out-of-band via
uv; weights SHA-256-pinned. **Gated on C3 + reproducible config**; ONNX supersedes this env |
opt-in "Enable local GPU" step; resumable progress (multi-GB on Indian bandwidth); post-install
`torch.cuda.is_available()` validate; falls back to LIGHT. |
79     | **CLOUD / server** | GCP deploy + BYO-GPU-VM | the `deploy/cloudrun/Dockerfile` image
(micromamba-ified); cloud-serving + gcs; behind Cloudflare Access | `gcloud run jobs create` per
`deploy/cloudrun/README.md`, or `deploy/gcp` VM scripts. Bind?`0.0.0.0` (config-gated) + edge-
auth mandatory. |
80
81     ## 5. Threat model / risk table (local appliance)
82     | Asset | Protected? | Residual risk / control |
83     |-----|-----|-----|
84     | Gemini API key | Partially | Today `GET /api/config` returns the **whole `AppConfig`
unauthenticated[verified main.py:144-146]. **P0** redacts + adds a serialization test + a
Host-header/localhost-origin guard (DNS-rebinding vector against `127.0.0.1`). Key lives in app-
home `.env` (or OS keychain), never in config; scrubbed on uninstall. |
85     | User videos + **human rally labels** | Yes (local) | MD5 video?label join breaks on
re-encode ? never re-encode after annotating; add export/backup (P2d). |
86     | Cloud spend | Yes | Profile flip needs explicit confirm + INR estimate + per-video cap
(L8); never silent. |
87     | SQLite index | Yes | DB migration on upgrade: backup-before-migrate + downgrade guard
(P2d). |
88
89     ## 6. Honest limits ? what this does NOT do
90     - The **first increment** (LIGHT) **does not run native WASB** ? CUJ-6/7 is absent from
the **initial** download (CPU/Gemini quality ? GPU-WASB quality). Per **L10 this is temporary, not
terminal** ? the north-star completion goal explicitly includes native WASB + training +
annotation (P3/P4). Don't market the first increment as the finished product.
91     - A green offline suite does **not** prove the **built artifact** works (hidden imports,
namespace-package resolution with no `backend/__init__.py`, missing data files like fonts) ? P0
adds a built-artifact smoke lane.
92     - "Push to cloud" is **BYO-GCP** today (no hosted-Khelsutra Storage/Compute client
exists) ? D13 marketing currently overstates a hosted option (O4).
93     - Training is **not** a closed loop: `worker train` is an offline CLI, `gate_verdict`

```

```

refuses commercial ship, and in-process jobs don't survive restart `[verified main.py:646
fail_stale_jobs]`. Never advertise annotate?train as done.
94
95     ## 7. Deliverables & progress tracker ? source of truth
96     Legend: ? Todo ? In progress ? Done ? Gated. **One small PR per row**, branched
from `origin/master`, no stacking, default-OFF.
97
98     | ID | Deliverable | Depends on | Gated? | Status | PR |
99     |----|-----|-----|-----|-----|----|
100    | **P0a** | **App-data HOME** (`RALLY_APP_HOME` env + per-OS default via
`default_os_app_home`) threaded through `default_config_path()`,
`load_dotenv(env_file_to_load())`, `resolve_output_dir` (output/DB), package-relative static
mount; **unified `doctor.py` write with the server read**. Byte-identical when unset; 4-lens
adversarial review folded. | ? | No | ? | [#293](https://github.com/Khelsutra/badminton-
highlight-indexer/pull/293) |
101    | P0b | **Redact `GET /api/config`** (`backend/utils/redact.py`) + serialization-safety
test (incl. a full-`AppConfig` over-masking guard). *(Host-header guard moved to P0c ? same
exposure concern.)* | ? | No | ? | [#295](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/295) |
102    | P0c | **Config/profile-driven bind + DNS-rebinding Host-header guard**
(`TrustedHostMiddleware` + `_resolve_bind_host`; default `127.0.0.1`;
`0.0.0.0`+`RALLY_ALLOWED_HOSTS` under edge-auth) + a startup WARN for the edge-auth/allowlist
footgun. Byte-identical default; adversarial review folded. | ? | No | ? |
[#296](https://github.com/Khelsutra/badminton-highlight-indexer/pull/296) |
103    | P0d | **Declare `scipy`** in `[project.dependencies]` (serving use `[verified
features/rally_seq.py:17]`, was transitive-only) + **license-scan CI lane** (pip-licenses; fails
on GPL/AGPL/LGPL/SSPL) + **built-wheel smoke lane** (build ? install ? `indexer --help` from a
non-repo cwd). Both lanes green on CI. | ? | No | ? |
[#297](https://github.com/Khelsutra/badminton-highlight-indexer/pull/297) |
104    | P0e | **Single-instance OS lock** (`backend/utils/single_instance.py`, fcntl/msvcrt)
on `

```

```

113 | P2b-install | **Install path** (`uv tool install` / a generated cross-platform install
script; ship `config.json` defaulting truly-local; optional thin per-OS shortcut to `indexer
--app`) | P2b-launcher | No (signing only if clickable binaries ? 09) | ? | ? |
114 | P2c | **In-UI 3-step first-run wizard** (Gemini key / Compute / Videos), rendered from
`/api/capabilities`, en/kn/hi | P1b | No | ? | ? | ? |
115 | P2d | **DB upgrade contract** ? `SCHEMA_VERSION` + downgrade guard
(`SchemaTooNewError`) + backup-before-migrate (`<db>.bak-v<old>`, WAL-complete) +
`backup_database()` export. Byte-identical for fresh/current DBs. *(uninstall manifest deferred
to P2b-install.)* | ? | No | ? | [#307](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/307) |
116 | P2e | **Lazy-torch for the LIGHT tier** ? deferred the one eager `import torch`
(`person_detector.py`) so `import backend.main` works WITHOUT torch (the LIGHT-tier promise,
enforced by a blocked-torch subprocess test). | ? | No | ? |
[#308](https://github.com/Khelsutra/badminton-highlight-indexer/pull/308) |
117 | **P3a** | **micromamba + conda-forge** replaces Miniconda+`conda` to accept` in
`deploy/cloudrun/Dockerfile` (+ ADR); reuse for the local managed WASB env | ? | No | ? | ? |
118 | P3b | **WASB weight fetcher** (SHA-256-pinned, resumable) ? `$WASB_WEIGHTS` ? **owner-
only flag; public must NOT fetch until C3** | P3a | ? 03 (counsel) | ? | ? |
119 | P3c | **"Push to cloud" toggle** (flip cloud-serving + confirm + INR estimate + per-
video cost cap) + wire worker usage-emission into the M8 ledger | P1b | Owner (cloud spend) | ?
| ? |
120 | P3d | **M9 edge-auth verified path** (JWKS refresh for ~6-week rotation, Cloud Run
ingress lock) | ? | ? counsel (public signup) | ? | ? |
121 | **P4a** | **ONNX export of WASB** + implement the ABC `ONNXRunner` (identical CSV) +
owner byte-parity/golden gate on a real GPU ? *dissolves the dual-env* | ? | ? owner GPU | ? | ?
|
122 | P4b | **Training-in-UI decision** (durable out-of-process long-job runner +
progress/cancel, surface `gate_verdict ship=False`) ? **recommend DEFER from v1** | ? | Owner |
? | ? |
123 | P4c | **Annotation decision** ? bundle VLC 3.0.x plugin vs build the **in-browser
HTML5 annotator** emitting the same rally CSV ? **recommend in-browser**, owner-gated | ? |
Owner | ? | ? |
124
125 ## 8. Open questions ? owner / external (blocking gates first)
126 1. **01 ? RESOLVED 2026-06-21:** a **webserver spun up on Windows/macOS/Linux** to power
the UI is the delivery ? **no native component** (no Electron/Tauri/embedded-webview, no native-
installer-as-runtime, no LGPL tray). Optional thin per-OS shortcut just runs the same command.
`pystray`/LGPL stays rejected; stdlib `webbrowser` only.
127 2. **02 ? RESOLVED 2026-06-21:** v1 = **LIGHT first**, **but the sole completion goal
(north star, L10) is the FULL path** ? native WASB + BYO training + in-app annotation. DoD $9
splits "v1 shippable" from "north-star DONE"; P3/P4 are completion-gating, not optional.
128 3. **03 ? WASB-C3 counsel sign-off:** may the checkpoint be redistributed / auto-fetched
into a public build at all? Determines whether a public HEAVY tier is **ever** possible vs WASB-
on-cloud-only.
129 4. **04 ? Cloud in a downloaded app = hosted-Khelsutra (none built) or BYO-GCP (the only
thing built)?** Decide before marketing the "push to cloud" toggle (D13 currently overstates
hosted).
130 5. **05 ? Training in v1 UI?** Recommend defer (synth=mock, real=GPU+dual-env,
`ship=False`).
131 6. **06 ? Annotation in v1?** in-browser HTML5 annotator (recommended) vs bundle VLC vs
defer.
132 7. **07 ? Ship AI-drafted kn/hi to real users before native-speaker review, or gate the
trilingual UI on review?**
133 8. **08 ? Shipped DEFAULT `config.json` / profile** = truly-local (so the box never
surprise-bills on Gemini nor returns 0 rallies on WASB-without-key)?
134 9. **09 ? Code-signing budget** (Windows Authenticode now, macOS Developer-ID later) so
non-tech users don't hit SmartScreen/AV prompts.
135
136 ## 9. Definition of done
137 - [ ] **01/02/08 decided in writing** (scope + delivery shape + default profile).
138 - [ ] P0a/P0e merged: launching `indexer --server` from **any** CWD finds the same
config/DB; `/api/config` leaks nothing; all CI lanes (incl. the new license-scan + built-
artifact lanes) green at the coverage floor.
139 - [ ] P1a/P1b merged: FastAPI serves the **real SPA** single-origin with refresh-
survival; capabilities/videos/reels return real data; contract test guards SPA?engine skew.

```

```

140     - [ ] P2a?P2d merged: a **clean Windows VM ? installer ? reel downloaded (en+hi)** with
**no Python/git/ffmpeg pre-installed**; uninstall is clean; upgrade preserves data.
141     - [ ] **Every completed CUJ in the map below verified preserved** (CUJ-6/7 explicitly
owner-only and flagged, not silently dropped).
142     - [ ] Verified on a **clean Windows AND macOS AND Linux** box (the same webserver-spin-
up path on all three).
143
144     **North-star DONE (sole completion goal, L10) ? v1 is the first increment toward this,
NOT the finish line:**
145     - [ ] **Native WASB ships in the downloadable app** on Win/Mac/Linux ? via the
micromamba env and/or the **ONNX runner** (P4a) that dissolves the dual-env ? passing the owner
byte-parity gate, with **WASB-C3 cleared** (O3).
146     - [ ] **BYO training runnable from the app** (durable out-of-process long-job runner;
`gate_verdict ship=False` surfaced honestly) ? P4b.
147     - [ ] **In-app rally annotation** (recommended: in-browser HTML5 annotator emitting the
same rally CSV) ? P4c.
148     - [ ] **CUJ-6/7 preserved in the DEFAULT download**, no longer owner-only.
149
150     ### CUJ preservation map `[verified inventory]`
151     | CUJ | Status | How preserved in the app |
152     |-----|-----|-----|
153     | **CUJ-1** Local Gemini ? rallies ? reel (en+hi) | DONE | LIGHT ships Gemini + full
`/api/process?jobs?query?compile?highlights` on the in-process worker, storage=local; verbatim,
needs key + resolved ffmpeg. |
154     | **CUJ-2** In-UI `gs://` bucket ingest | DONE | `acquire_local_input` + gcs backend
already accept `gs://`; preserved when the user opts into a cloud storage backend (+ ADC). |
155     | **CUJ-3** Per-rally selection + query bar | DONE | Pure SPA + `/api/query` +
`/api/compile`; preserved by bundling the built SPA single-origin + the contract test. |
156     | **CUJ-4** Reel download-to-local | DONE | `/api/compile` + `GET /api/highlights/...`
unchanged; resolved ffmpeg provides the encoder; app-home output dir keeps reels findable. |
157     | **CUJ-5** Trilingual SPA (en+kn+hi), offline | DONE | Catalogs + OFL Noto fonts
compiled into the bundle (no CDN); air-gap renderable. *(kn/hi native review pending ? 07.)* |
158     | **CUJ-6/7** Worker GCS?GPU WASB?GCS + cold-start dual-env | DONE (engine) |
**HEAVY/cloud tiers ONLY** (owner + provisioned GPU) via micromamba WASB env / Cloud Run image ?
**deliberately absent from the LIGHT default**; gated on C3 + reproducible config. |
159     | **CUJ-8** CLI single-shot (`indexer --video-path --segmenter`) | DONE | Console +
module entrypoints untouched by packaging; available every tier (segmenter set limited by tier).
|
160     | **CUJ-9** Annotate ? train | PARTIAL | `worker train` CLI + rally-annotator CSV bridge
remain runnable; honesty fences (synth=mock, `ship=False`) must surface in any UI; recommend
**deferred** from v1 UI (P4b/c). |
161
162     ## 10. References
163     **Internal code anchors `[verified]`:** `pyproject.toml:80-82,8-37,63-70`;
`backend/main.py:10,64-74,144-146,275-315,646,729-736,756`;
`backend/config/loader.py:30,112,202`; `backend/config/presets.py`;
`backend/compute/__init__.py`; `backend/storage/`;
`backend/pipeline/detectors/native_wasb_runner.py:17,89,110-112`;
`backend/pipeline/stitcher/compiler.py:253`; `backend/features/rally_seq.py:17`;
`backend/infrastructure/database.py:88-110` (user_version=6); `training/gen0/harness.py:174-177`
(`ship=False`); `deploy/cloudrun/Dockerfile:33-37,54`. **Internal docs:**
`docs/COMPUTE_DECOUPLED_SERVING/`, `docs/DESKTOP_APP_PLAN.md` (superseded),
`docs/UI_PROPOSAL.md`, `docs/I18N_PLAN.md`, `kshelsutra/docs/{LOCAL_APPLIANCE_ARCHITECTURE,ANNOTA
TE_AND_TRAIN,DRY_RUN_CUJ,FRIENDLY_AND_MULTILINGUAL}.md`. **External `[researched]`:** uv torch-
index docs; Frigate/Ollama/ComfyUI/LM Studio distribution; conda-forge/micromamba ToS; ONNX
Runtime; ffmpeg GPL-vs-LGPL.
164
165     ---
166
167     ### Changelog
168     - 2026-06-21 ? Created from an exhaustive research workflow (29 agents) + per-claim re-
verification at file:line.
169     - 2026-06-21 ? Owner resolved O1 + O2: **cross-platform (Win/Mac/Linux) webserver, no
native component** (L9); **LIGHT-first with a WASB-complete north star as the sole completion
goal** (L10). Updated $0/$2/$4.5/$6/$7-P2b/$8/$9 accordingly. Remaining gates: O3 (WASB-C3

```

counsel), 04 (hosted vs BYO-cloud), 05?09.
170

30. assistant (2026-06-21T22:32:41.009Z)

Perfect! Now let me compile the comprehensive findings. Let me verify a few more details on the backend/config/startup.py for the checks:

31. user (2026-06-21T22:32:41.270Z)

```
1      """Startup checks ? per-profile coherence (M5) + exposure-safety posture (M9 hardening).
2
3      A deployment profile (`truly-local` / `cloud-serving` / `cpu-mock`) implies a
coherent
4      runtime. These checks fail/warn fast at process startup ? a loud signal *before* any
5      (expensive) job runs ? when the active profile is inconsistent with the environment:
6
7      * cloud-serving with a non-cloud storage backend ? a hard **error** (the one
config bug we
8      can detect with certainty + zero false-positive risk: a cloud profile that can't
reach a
9      bucket can never serve). This is the README §4.3 "loud error before any job".
10     * a GPU mismatch (e.g. `device='cuda'` on a box where CUDA isn't visible, or
`device='auto'`
11     falling back to CPU) ? a **warning**, never an error: the probe is best-effort
(torch may live
12     only in the detector subprocess env ? see `presets.probe_cuda_available`), and the
detector's
13     own healthcheck is the real authority. A warning surfaces it early without false-
failing.
14     * creds that can't be confirmed (no `GOOGLE_APPLICATION_CREDENTIALS` / not on
Cloud Run) ?
15     a **warning**: Application Default Credentials can come from the metadata server /
gcloud, which
16     we can't probe offline, so the storage backend's own init is the authority; we just
flag it.
17
18     **Exposure-safety posture (server only, `include_exposure=True`)** When the engine is
*exposed*
19     behind an edge gate (the owner flips `security.edge_auth_enabled` on for the
Cloudflare-Access
20     boundary ? the alpha-shareable "tunnel-to-a-box" topology), the gate is only half the
safety story.
21     A half-configured exposure starts *looking* healthy then leaks / 500s per request, so we
verify the
22     rest of the posture at **boot**: the `allowed_video_root` path-jail is set (else an
authenticated
23     tenant could read an arbitrary local file ? **error**), the CF team-domain + AUD are
present (else
24     no token can verify ? **error**, lifting the current first-request failure to boot), and
the
25     `auth` extra is importable (else every request 500s ? **warning**). Only the HTTP
**server** passes
26     `include_exposure=True`; the worker reads `--video-uri` (not untrusted tenant
paths), so it is
27     never "exposed" and stays unaffected.
28
29     **DEFAULT-OFF (both dimensions)** with no profile selected (`profile == ""`) the
profile checks are
30     a strict **no-op** ? exactly the pre-M5 behaviour; with `edge_auth_enabled=False`
(today's default)
31     the exposure checks are a strict **no-op** too. So nothing changes for today's manual-
knob deployments.
32     """
33
```

```

34     from __future__ import annotations
35
36     import logging
37     import os
38     from dataclasses import dataclass
39     from typing import Any, Mapping, Optional
40
41     LEVEL_ERROR = "error"
42     LEVEL_WARN = "warn"
43
44     # Storage backends that count as "a cloud bucket" for cloud-serving coherence.
45     _CLOUD_STORAGE = ("gcs", "s3")
46     # STRONG env signals that Google ADC *might* resolve. Deliberately NOT
GOOGLE_CLOUD_PROJECT /
47     # CLOUDSDK_CONFIG ? those are often set without usable creds (a plain project id / a
config dir
48     # with no active login), so they would suppress the heads-up falsely. (s3 creds are
intentionally
49     # left to boto3's default chain ? no parallel heads-up, by design.)
50     _GCP_ENV_HINTS = ("GOOGLE_APPLICATION_CREDENTIALS", "K_SERVICE", "GCE_METADATA_HOST")
51
52
53     @dataclass(frozen=True)
54     class StartupCheck:
55         """One per-profile finding. ``level`` is ``error`` (fatal) or ``warn`` (heads-
up)."""
56
57         level: str
58         code: str          # stable, greppable code (e.g. "STORAGE_INCOHERENT")
59         message: str
60
61
62     class StartupCheckError(RuntimeError):
63         """Raised by ``enforce_startup_checks`` when an active profile fails a FATAL
check."""
64
65         def __init__(self, failures: list[StartupCheck]):
66             self.failures = failures
67             super().__init__(
68                 "deployment profile startup check failed: "
69                 + "; ".join(f"[{c.code}] {c.message}" for c in failures)
70             )
71
72
73     def _creds_discoverable(env: Mapping[str, str]) -> bool:
74         """True if *some* Google ADC source is hinted by the environment. Best-effort ? a
False here
75         only downgrades to a WARNING, never a hard failure (the storage init is the
authority)."""
76         return any(env.get(k) for k in _GCP_ENV_HINTS)
77
78
79     def _auth_extra_available() -> bool:
80         """True if PyJWT (the ``auth`` extra) is importable ? probed WITHOUT importing it
(find_spec),
81         so this stays cheap + side-effect-free. Module-level so tests can monkeypatch it
deterministically
82         (whether PyJWT is installed in the test env must not change the WARN's presence)."""
83         try:
84             import importlib.util
85             return importlib.util.find_spec("jwt") is not None
86         except Exception: # pragma: no cover - find_spec only raises on a broken parent
package
87             return False
88

```



```

89
90     def _exposure_checks(config: Any, env: Mapping[str, str]) -> list[StartupCheck]:
91         """Exposure-safety posture for an *exposed* HTTP server. Gated on
92         ``security.edge_auth_enabled``
93         (default False ? ``[]`` = strict no-op, today's behaviour). When edge auth is ON ?
94         the "I am exposing this engine behind the CF-Access gate" signal ? the gate alone isn't
95         enough: verify the
96         rest of the posture at boot so a half-configured exposure fails fast instead of
97         leaking / 500-ing
98         per request. Config-only (no IO beyond the ``find_spec`` auth-extra probe); ``env``
99         is the
100         caller's environment view (so the host-allowlist check is testable)."""
101         sec = getattr(config, "security", None)
102         if not sec or not getattr(sec, "edge_auth_enabled", False):
103             return [] # DEFAULT-OFF: edge auth off ? no posture enforcement (byte-identical
104             to today).
105         out: list[StartupCheck] = []
106
107         # (1) The path-jail is load-bearing once authenticated tenants can hit /api/process:
108         WITHOUT it,
109         # a tenant can point the engine at an ARBITRARY local file (allowed_video_root unset
110         = "any local
111         # file", the single-user default). Fatal for an exposed engine.
112         root = getattr(sec, "allowed_video_root", None)
113         if not root or not str(root).strip():
114             out.append(StartupCheck(
115                 LEVEL_ERROR, "EXPOSED_NO_VIDEO_ROOT",
116                 "security.edge_auth_enabled is ON (engine exposed) but
117                 security.allowed_video_root is "
118                 "unset ? an authenticated tenant could make /api/process read an arbitrary
119                 local file. "
120                 "Set allowed_video_root to a jail directory before exposing the engine.",
121             ))
122
123         # (2) Edge auth cannot verify a token without the team domain + AUD. Today that only
124         fails on the
125         # FIRST request (auth.resolve_tenant); lift it to BOOT so a misconfigured gate never
126         starts
127         # "healthy" then 401s every request.
128         team = (getattr(sec, "cf_team_domain", "") or "").strip()
129         aud = (getattr(sec, "cf_access_aud", "") or "").strip()
130         if not team or not aud:
131             missing = " + ".join(n for n, v in (("cf_team_domain", team), ("cf_access_aud",
132             aud))) if not v)
133             out.append(StartupCheck(
134                 LEVEL_ERROR, "EXPOSED_EDGE_AUTH_INCOMPLETE",
135                 f"security.edge_auth_enabled is ON but {missing} is not configured ? the Cf-
136                 Access JWT "
137                 "cannot be verified (every request would fail). Set security.cf_team_domain
138                 "
139                 "(e.g. https://<team>.cloudflareaccess.com) and security.cf_access_aud (the
140                 CF Access "
141                 "application AUD tag).",
142             ))
143
144         # (3) The auth extra (PyJWT[crypto]) is lazy-imported by
145         auth.verify_cf_access_token. If it's
146         # missing, every request 500s at verify time ? a boot WARN beats a per-request 500.
147         Warning, not
148         # fatal: the auth module's own ImportError is the final authority.
149         if not _auth_extra_available():
150             out.append(StartupCheck(
151                 LEVEL_WARN, "EXPOSED_AUTH_EXTRA_MISSING",

```

```

136             "security.edge_auth_enabled is ON but PyJWT[crypto] is not importable ? Cf-
Access "
137             "verification will fail at request time. Install the 'auth' extra (pip
install -e .[auth]).",
138         ))
139
140     # (4) P0c: an exposed engine binds 0.0.0.0, but the Host-header (DNS-rebinding)
guard defaults to
141     # loopback-only when RALLY_ALLOWED_HOSTS is unset (see main._allowed_hosts_from_env)
? it would
142     # then 400 every request the edge forwards with its public Host. Boot WARN beats a
silent 400-all.
143     if not (env.get("RALLY_ALLOWED_HOSTS", "") or "").strip():
144         out.append(StartupCheck(
145             LEVEL_WARN, "EXPOSED_HOST_ALLOWLIST_LOOPBACK",
146             "security.edge_auth_enabled is ON but RALLY_ALLOWED_HOSTS is unset ? the
Host-header guard "
147             "defaults to loopback-only and will 400 forwarded requests. Set
RALLY_ALLOWED_HOSTS to your "
148             "public host(s) (or '*' when the edge gate validates the host).",
149         ))
150
151     return out
152
153
154     def run_startup_checks(
155         config: Any,
156         *,
157         cuda_available: Optional[bool] = None,
158         env: Optional[Mapping[str, str]] = None,
159         include_exposure: bool = False,
160     ) -> list[StartupCheck]:
161         """Return the startup findings for ``config``.
162
163         ``cuda_available`` is the caller's best-effort GPU probe (``None`` = not probed ?
GPU checks
164         are skipped; the API server passes ``None``, the worker passes
``presets.probe_cuda_available``).
165         ``include_exposure`` adds the exposure-safety posture (``_exposure_checks``) ? the
HTTP **server**
166         passes ``True``; the worker leaves it ``False`` (it is never "exposed"). Both
dimensions are
167         DEFAULT-OFF: no profile ? no profile checks; ``edge_auth_enabled=False`` ? no
exposure checks.
168         Pure: no IO (beyond the ``find_spec`` auth probe), no torch import ? the caller owns
the GPU probe.
169         """
170         e = os.environ if env is None else env
171         checks: list[StartupCheck] = []
172
173         # Exposure-safety posture ? INDEPENDENT of deployment.profile (a tunnel-to-a-box
exposure often
174         # runs with no profile set), gated on edge_auth_enabled (default-OFF). Server-only.
175         if include_exposure:
176             checks.extend(_exposure_checks(config, e))
177
178         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
179         if not profile:
180             return checks # no profile ? only the exposure posture (itself off unless edge
auth is on).
181
182         storage_backend = getattr(getattr(config, "storage", None), "backend", "local")
183         indexing = getattr(config, "indexing", None)
184         detector_impl = getattr(indexing, "detector_impl", "auto")
185         device = getattr(getattr(indexing, "wasb", None), "device", "cuda")

```

```

186
187     if profile == "truly-local":
188         # truly-local reads bytes from the local FS or a *mounted* Drive ? a cloud
backend here is
189         # an inconsistent (but not fatal) choice worth surfacing.
190         if storage_backend not in ("local", "gdrive"):
191             checks.append(StartupCheck(
192                 LEVEL_WARN, "STORAGE_UNUSUAL",
193                 f"profile 'truly-local' usually uses local/gdrive storage, but
storage_backend="
194                 f"'{storage_backend}'. Reads will go to a remote backend.",
195             ))
196         # GPU heads-up (warning only ? the detector healthcheck is the authority; the
probe is
197         # best-effort and may be False even when a subprocess detector env has CUDA).
198         if cuda_available is False:
199             if device == "cuda":
200                 checks.append(StartupCheck(
201                     LEVEL_WARN, "CUDA_NOT_VISIBLE",
202                     "device='cuda' but no CUDA GPU is visible to this process. If the
box truly "
203                     "has no GPU the detector healthcheck will hard-fail ? set
indexing.wasb.device="
204                     "'auto' for the CPU fallback (M4).",
205                 ))
206             elif device == "auto":
207                 checks.append(StartupCheck(
208                     LEVEL_WARN, "CPU_FALLBACK",
209                     "device='auto' and no CUDA GPU is visible to THIS process ? auto may
resolve "
210                     "to CPU (much slower). If torch lives only in the detector
subprocess env this "
211                     "can be a false alarm; the runner's DeviceContext/healthcheck is the
authority.",
212                 ))
213
214     elif profile == "cloud-serving":
215         # The ONE fatal coherence check: a cloud profile that can't reach a bucket can't
serve.
216         if storage_backend not in _CLOUD_STORAGE:
217             checks.append(StartupCheck(
218                 LEVEL_ERROR, "STORAGE_INCOHERENT",
219                 f"profile 'cloud-serving' requires a cloud storage backend "
220                 f"('{','.join(_CLOUD_STORAGE)}'), but storage_backend='{storage_backend}'.
"
221                 "A cloud job cannot read/write a local-only backend.",
222             ))
223         elif storage_backend == "gcs" and not _creds_discoverable(e):
224             checks.append(StartupCheck(
225                 LEVEL_WARN, "CREDS_UNCONFIRMED",
226                 "no GOOGLE_APPLICATION_CREDENTIALS / Cloud Run env detected; relying on
Application "
227                 "Default Credentials. The GCS backend will fail loudly at init if ADC
can't resolve.",
228             ))
229         if device == "cpu":
230             checks.append(StartupCheck(
231                 LEVEL_WARN, "CLOUD_CPU_DEVICE",
232                 "profile 'cloud-serving' runs on a GPU (L4) but
indexing.wasb.device='cpu' ? "
233                 "detection will not use the GPU.",
234             ))
235         elif cuda_available is False and device in ("cuda", "auto"):
236             checks.append(StartupCheck(
237                 LEVEL_WARN, "CLOUD_CUDA_NOT_VISIBLE",

```

```

238             "profile 'cloud-serving' expects a GPU but no CUDA GPU is visible to
this process.",
239         ))
240
241     elif profile == "cpu-mock":
242         # The mock profile is for CI/regression ? a real detector here defeats the
purpose.
243         if detector_impl != "stub":
244             checks.append(StartupCheck(
245                 LEVEL_WARN, "MOCK_NONSTUB_DETECTOR",
246                 f"profile 'cpu-mock' normally uses detector_impl='stub', but
detector_impl="
247                 f"'{detector_impl}'. A real detector defeats the GPU-free mock.",
248             ))
249
250     return checks
251
252
253     def enforce_startup_checks(
254         config: Any,
255         *,
256         cuda_available: Optional[bool] = None,
257         env: Optional[Mapping[str, str]] = None,
258         raise_on_error: bool = True,
259         include_exposure: bool = False,
260         logger: Optional[logging.Logger] = None,
261     ) -> list[StartupCheck]:
262         """Run the checks, LOG each (errors at ERROR, warnings at WARNING), and ? unless
263         ``raise_on_error=False`` ? raise ``StartupCheckError`` if any FATAL check tripped.
264
265         ``include_exposure`` (server-only) adds the exposure-safety posture. Returns the
full findings
266         list (so a caller can inspect/surface them). A no-op with no logging and no raise
when both
267         dimensions are off (``run_startup_checks`` returns ``[]``)."""
268         log = logger or logging.getLogger(__name__)
269         checks = run_startup_checks(
270             config, cuda_available=cuda_available, env=env,
include_exposure=include_exposure)
271         for c in checks:
272             if c.level == LEVEL_ERROR:
273                 log.error("[STARTUP] %s: %s", c.code, c.message)
274             else:
275                 log.warning("[STARTUP] %s: %s", c.code, c.message)
276         errors = [c for c in checks if c.level == LEVEL_ERROR]
277         if errors and raise_on_error:
278             raise StartupCheckError(errors)
279         return checks
280

```

32. assistant (2026-06-21T22:33:29.483Z)

Excellent! Now I have all the information I need. Let me compile the comprehensive findings:

P2b-Install Packaging PR ? Comprehensive Surface Map

1. **Launcher app-home wiring** (backend/main.py `--app`)

****Code path for `--app` argument handling:**** `backend/main.py:867-1092`

In `main()` at line ****867?879****, the `--app` argument is defined. At ****1064?1092****, the launcher branch:

```
```python
```

```

if args.app:
 bind_host = "127.0.0.1"
 port = _resolve_app_port(bind_host, args.port)
 if port != args.port:
 print(f"[APP] Port {args.port} is busy ? using {port} instead.")
 url = f"http://{bind_host}:{port}/"
 print(f"[APP] Starting KhelSutra ? opening {url} in your browser ...")
 threading.Thread(target=_open_browser_when_ready, args=(url, url + "api/health"),
 daemon=True).start()
...

CRITICAL FINDING: The launcher does **NOT** auto-export `RALLY_APP_HOME` to the OS app-data
dir when unset. Instead:

- **Line 939:** `global_config = load_app_config()` is called with NO arguments ? resolves
`default_config_path()` (from `backend/utils/app_home.py:53`)
- **Line 947:** `global_config.output.output_dir = resolve_output_dir(args.output_dir)` accepts
the CLI `--output-dir` (default `output`)
- **When `RALLY_APP_HOME` is unset:** `default_config_path()` returns `Path("config.json")`
(CWD-relative) and `resolve_output_dir("output")` returns `output` (CWD-relative) ? **byte-
identical to pre-P0a behaviour**
- **When `RALLY_APP_HOME` is set:** both root under that home

State directory resolution: With **no `RALLY_APP_HOME` and no config.json**, a fresh
`indexer --app` run in an arbitrary CWD:
- Reads/writes config from/to **`./config.json`** (relative to CWD)
- Writes output (DB, reels) to **`./output/`** (relative to CWD)
- Creates/checks single-instance lock at **`./output/sports_indexer.db.lock`**

Server-side config loading: `load_app_config()` at **main.py:939** with no args ?
`backend/config/loader.py:108` ? calls `default_config_path()` (app_home-aware). The server
**does not call this early in main(); it's called inside the `--app` / `--server` / CLI-video-
path branches to initialize `global_config`.

```

## 2. **\*\*Fresh-install config defaults\*\***

**\*\*Missing-file branch in `backend/config/loader.py:108?138`:\*\***

```

```python
if cfg_path.exists():
    try:
        with open(cfg_path, "r", encoding="utf-8") as f:
            file_data = json.load(f)
    except Exception as exc:
        logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial data.")

```

...file_data = {} if not found/invalid

...

****Behavior:**** When no config.json file exists, `load\_config()` ****does NOT error****. Instead, it
****falls back to `AppConfig()` defaults**** (line 153: `defaults = \_get\_builtin\_defaults()` ?
`AppConfig().model\_dump(mode="json")`).

****DEFAULT pydantic field values**** (from `backend/config/models.py`):

Field	Default	Citation
`indexing.default_segmenter`	`"yolo_hybrid"`	models.py:128
`indexing.use_gpu`	`True`	models.py:129
`deployment.profile`	`""` (empty string, no profile)	models.py:362
`deployment.compute_target`	`""` (empty)	models.py:365
`indexing.skip_ai_handoff`	`False`	models.py:202

```
| `stitching.min_shot_count` | `1` | models.py:259 |  
| `output.db_name` | `"sports_indexer.db"` | models.py:232 |
```

****Key finding:**** The shipped config.json (repo root, `config.json`) currently sets
`default\_segmenter: "gemini"` (README.md:103, test_config.py:91), ****NOT**** `yolo\_hybrid`. A fresh
install with no config.json will use the pydantic default (`yolo\_hybrid`), ****not**** the repo's
shipped override.

3. ****Truly-local profile preset****

****Profile names and presets**** from `backend/config/presets.py:27?63`:

```
```python  
PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")

PROFILE_PRESETS = {
 "truly-local": {
 "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
 "indexing": {"skip_ai_handoff": True},
 "storage": {"backend": "local"},
 },
 "cpu-mock": {
 "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
 "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
 "storage": {"backend": "local"},
 },
 # ... cloud-serving omitted here
}
```
```

****Exact `truly-local` preset:****

- `deployment.profile`: `"truly-local"`
- `deployment.compute\_target`: `"local\_gpu"`
- `indexing.skip\_ai\_handoff`: `True` (fully local, zero cloud)
- `indexing.detector\_impl`: (NOT set in preset ? inherits default `"auto"`)
- `storage.backend`: `"local"`
- `indexing.use\_gpu`: (NOT set in preset ? inherits default `True`)

****How a profile is applied:**** In `backend/config/loader.py:154?199`:

```
```python  
profile = _resolve_explicit_profile(file_data) # env > config.json
if profile:
 merged = _deep_merge(_deep_merge(defaults, preset_for(profile)), file_data)
 # profile can be applied ONLY via:
 # (1) DEPLOYMENT_PROFILE env var (loader.py:62)
 # (2) config.json deployment.profile field (loader.py:72?76)
```
```

****CRITICAL:**** A profile is ****NOT**** selectable purely via config.json's `deployment.profile` field
alone ? the loader must ****explicitly recognize it****. If config.json sets an unknown profile
name, the loader warns and applies no preset (loader.py:160?164). ****Env var `DEPLOYMENT\_PROFILE`
takes precedence over config.json**** (line 62).

4. ****`indexer --setup` / doctor.py****

****What `--setup` does today**** (`backend/main.py:884?896`):

```
```python  
if args.setup:
 doctor_path = os.path.join(
```

```

 os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
 "scripts", "doctor.py",
)
 print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
 subprocess.run([sys.executable, doctor_path])
 return
...

```

**\*\*`scripts/doctor.py` behaviour\*\* (lines 45?72):**

```

```python
def init_default_config():
    from backend.config import save_default_config
    from backend.utils.app_home import default_config_path
    config_path = str(default_config_path()) # RALLY_APP_HOME if set, else "config.json"
    if os.path.exists(config_path):
        print(f"[INFO] {config_path} already exists. Skipping initialization.")
        return
    try:
        saved_path = save_default_config(config_path)
        print(f"[SUCCESS] config.json initialized with default settings at {saved_path}.")
    except Exception as e:
        print(f"[ERROR] Failed to create config.json via loader: {e}")
...

```

- ****Writes config.json:**** YES, via ``backend/config/loader.py:save_default_config()`` (line 202?212)
- ****Path:**** Uses ``default_config_path()`` ? ****RALLY_APP_HOME** if set, else CWD/config.json** (app_home.py:53?58)
- ****NOT repo root:**** Correctly writes to the same path the server reads (****P0a fix verified****, test_doctor.py:30?36)
- ****FFmpeg check:**** YES ? ``check_command("ffmpeg")`` at doctor.py:89, prints per-OS hint (winget, brew, apt) at lines 107?114
- ****Packaged in wheel:**** The wheel packages ``["backend", "training"]`` (pyproject.toml:97); ****`scripts/` is NOT listed**** ? ****`doctor.py` is NOT included in the wheel****. ``indexer --setup`` ****cannot call doctor.py from an installed wheel**** ? it requires the repo.

****BLOCKER:**** For a packaged install, ``doctor.py`` must be either:

1. Moved into ``backend/`` and made callable as a module entry point, OR
2. Bundled via a hatchling entry point

5. ****README / documented install steps****

****Current documented procedure**** (README.md:33?83):

```
```bash
```

### 1. **FFmpeg (Windows, admin PowerShell):**

```
winget install Gyan.FFmpeg
```

### 2. **Install the package (editable):**

```
pip install -e .
```

#### **Plus test/lint tooling:**

```
pip install -e .[dev]
```

### 3. **PyTorch separately (not in pyproject.toml):**

#### **GPU (CUDA 12.4):**

```
pip install "torch>=2.12.0" "torchvision>=0.27.0" --index-url
https://download.pytorch.org/whl/cu124
```

#### **CPU:**

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
```

## 4. Diagnostics:

```
python scripts/doctor.py
```

**or:**

```
indexer --setup

```

No mention of `uv` or `uv tool install` ? those are **\*\*planned for P2b\*\*** but not yet documented.

---

## 6. **\*\*Uninstall footprint ? all mutable state\*\***

Every piece of state a fresh install creates that uninstall must scrub:

Item	Location	Determined by	Notes
<b>**config.json**</b>	<code>`default_config_path()`</code>	<code>`RALLY_APP_HOME/config.json`</code> (else CWD)	<code>`backend/utils/app_home.py:53?58`</code>   User edits; preserve if desired
<b>**env**</b> (secrets)	<code>`default_env_path()`</code>	<code>`RALLY_APP_HOME/.env`</code> (else CWD)	<code>`backend/utils/app_home.py:61?63`</code>   Contains <code>`GEMINI_API_KEY`</code> , <code>`WASB_PYTHON`</code> , etc.
<b>**SQLite DB**</b>	<code>`output.output_dir / output.db_name`</code>	default <code>`"output/sports_indexer.db"`</code>	<code>`backend/config/models.py:232?235`</code>   MD5-keyed video records
<b>**DB lock file**</b>	<code>`&lt;db_path&gt;.lock`</code>	<code>`"output/sports_indexer.db.lock"`</code>	<code>`backend/main.py:958`</code> (acquire_lock call)   Single-instance guard
<b>**output/ directory**</b>	<code>`output.output_dir`</code>	default <code>`"output"`</code> (CWD-relative if <code>`RALLY_APP_HOME`</code> unset)	<code>`backend/config/models.py:235`</code>   Compiled reels, reports
<b>**uploads/ directory**</b>	<code>`security.upload_dir`</code>	default <code>`"output/uploads"`</code>	<code>`backend/config/models.py:289`</code>   Chunked upload landing zone
<b>**Console script**</b>	<code>`indexer`</code> entry point	<code>`pyproject.toml:86`</code>	Registered by <code>`pip install`</code>
<b>**Wheel/package**</b>	Site-packages	<code>`badminton-highlight-indexer`</code>	pip   All Python code, SPA bundle, fonts
<b>**Cached weights**</b>	<code>`\$WASB_WEIGHTS`</code> path	(not auto-fetched in P2b)	<code>`backend/pipeline/detectors/native_wasb_runner.py`</code> (future P3b)   Only if HEAVY tier used

**\*\*Uninstall script must:\*\***

1. Remove the console script: ``pip uninstall badminton-highlight-indexer``
2. Prompt user for removal of mutable state (offer to preserve config.json + DB for re-install):
  - ``default_config_path()`` (app\_home-aware)
  - ``default_env_path()`` (delete ``.env``, not config.json)
  - ``output_dir`` (reels, DB, reports)
  - Lock file

---

## 7. **\*\*Existing install tooling\*\***

**\*\*Grep results for install/setup scripts:\*\***

File	What it does
<b>**deploy/digitalocean/bootstrap.sh**</b>	Ubuntu/DigitalOcean: apt-get ffmpeg/python/libs ? venv ? pip torch (cu124 or cpu) ? <code>`pip install -e .[dev]`</code> ? torch test. <b>**Does NOT write config.json**</b> . Mocks/offline suite proof.
<b>**scripts/doctor.py**</b>	Validates ffmpeg; inits config.json (app-home aware); offers per-OS ffmpeg install; runs pip from requirements.txt (GPU-auto-detect). <b>**NOT in wheel**</b> .
<b>**pyproject.toml [project.scripts]**</b>	<code>`indexer = "backend.main:main"`</code> (line 86). <b>**ONLY entry point**</b> . No install script, no uninstall manifest.
<b>**pyproject.toml [project.dependencies]**</b>	No torch, no ffmpeg. torch is out-of-band (needs <code>`--index-url`</code> ).



**\*\*No existing:\*\*** uv usage, uninstall manifest, multi-entry-point, or shell/PowerShell wrappers.

---

## 8. **\*\*Tests layout for config/doctor/app\_home\*\***

Test files for this surface:

File	Purpose	Citation
<b>**tests/test_app_home.py**</b>	<code>`app_home()`</code> , <code>`default_config_path()`</code> , <code>`resolve_output_dir()`</code> , env handling, OS-app-home defaults. 188 lines.	Per the layout convention: <code>`test_&lt;module&gt;.py`</code>
<b>**tests/test_config.py**</b>	Load defaults (no file), file overrides, extra-keys tolerance, profile application, <code>`save_default_config()`</code> . 200+ lines.	Existing convention
<b>**tests/test_doctor.py**</b>	Config write under app_home, skip-if-exists, CWD fallback, import degradation. 68 lines.	Uses <code>`importlib.util`</code> to load doctor.py as a module
<b>**tests/test_config_deployment.py**</b>	Profile application, compute_target assignment, startup checks (M5).	Existing convention

**\*\*New P2b tests should follow:\*\*** ``test_p2b_install*.py`` or integrate into existing files (e.g., ``test_app_home.py`` for path resolution, ``test_doctor.py`` for packaging validation).

---

## ?? Flags for P2b Blockers

- \*\*doctor.py not in wheel:\*\*** ``scripts/`` is excluded from ``[tool.hatch.build.targets.wheel] packages``. ``indexer --setup`` will fail from an installed wheel. **\*\*Must move doctor logic into the engine or create a new entry point.\*\***
- \*\*Config.json not in wheel by default:\*\*** The shipped ``config.json`` (repo root) is not explicitly bundled. A fresh ``pip install`` gets **\*\*pydantic defaults, NOT the repo's shipped overrides\*\***. **\*\*Default ``default_segmenter`` in wheel differs from repo\*\*** (``yolo_hybrid`` vs ``"gemini"``).
- \*\*Profile application requires explicit selection:\*\*** A profile is NOT auto-applied just by setting ``deployment.profile`` in ``config.json`` ? it must also be recognized in ``PROFILE_PRESETS``. Unknown profiles generate a **\*\*warning, not an error\*\***, and no preset is applied.
- \*\*--app does not auto-set RALLY\_APP\_HOME:\*\*** The launcher boots a server but does **\*\*not export RALLY\_APP\_HOME\*\*** to the OS app-data dir when unset. State lands in CWD (pre-P0a behaviour). The **\*\*packaged launcher shell script or entry point must export it\*\***.
- \*\*Truly-local default state:\*\*** With **\*\*no RALLY\_APP\_HOME and no config.json\*\***, a fresh wheel install defaults to:
  - ``default_segmenter: "yolo_hybrid"`` (pydantic default, not shipped config's ``"gemini"``)
  - ``skip_ai_handoff: False`` (must be set to ``True`` in truly-local preset or config)
  - ``profile: ""`` (no profile; manual knobs)

**\*\*For truly-local first run, must either ship a config.json inside the wheel OR prompt the wizard to set it.\*\***
- \*\*Uninstall manifest missing:\*\*** No existing mechanism to track/remove mutable state (DB, config, .env, output dir).

---

## Summary

The **\*\*install/first-run surface is fully mapped\*\***. The P2b PR must address: **\*\* (1) moving doctor into the wheel, (2) shipping a truly-local default config.json, (3) exporting RALLY\_APP\_HOME from the launcher, (4) building an uninstall manifest, and (5) documenting the ``uv tool install`` path. All file:line citations provided above for implementation.**